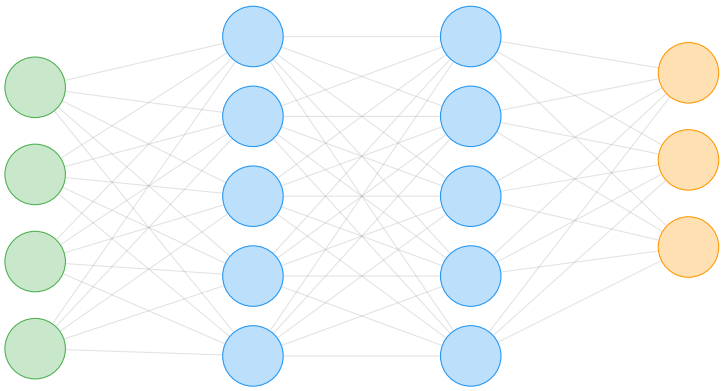
Neural Networks

A Comprehensive Guide for Artificial Intelligence Students



G. Qu et al.

Yunnan University
School of Intelligent Science and Technology
Kunming, China

March 2026

License

This work is licensed under a Creative Commons "Attribution-NonCommercial-ShareAlike 4.0 International" license.

About This Book

This comprehensive textbook on Neural Networks is designed for students and practitioners in Artificial Intelligence. It covers the fundamental concepts, mathematical foundations, and practical implementations of modern neural network architectures.

Main Features:

- Comprehensive coverage from basic perceptrons to advanced transformers
- Detailed mathematical derivations and explanations
- Practical examples using PyTorch framework
- GPU acceleration and CUDA programming concepts
- Real-world applications and case studies

Authors: G. Qu et al.

Institution: Yunnan University, School of Intelligent Science and Technology

Year: 2026

Abstract

Neural networks have revolutionized the field of artificial intelligence, enabling breakthroughs in computer vision, natural language processing, speech recognition, and many other domains. This textbook provides a comprehensive introduction to neural networks, from fundamental concepts to state-of-the-art architectures.

The book is organized into twelve chapters, covering:

1. **Mathematical Foundations:** Linear algebra, calculus, and probability theory essentials for understanding neural networks.
2. **Activation Functions:** Comprehensive analysis of sigmoid, tanh, ReLU, and advanced activation functions with guidance on selection criteria.
3. **Loss Functions:** Detailed exploration of cross-entropy, mean squared error, and task-specific loss functions.
4. **Neural Network Architectures:** Deep Neural Networks (DNN), Convolutional Neural Networks (CNN), and Recurrent Neural Networks (RNN).
5. **Core Operations:** Convolution, pooling, and normalization operations with mathematical rigor.
6. **Advanced Mechanisms:** Skip connections and channel attention mechanisms including residual learning.
7. **Optimization:** Gradient descent algorithms, optimizers (SGD, Adam, AdamW), and learning rate schedulers.
8. **Classic Architectures:** From LeNet and AlexNet to ResNet, VGG, and modern architectures.
9. **Computational Analysis:** Parameter counting and FLOPs computation for network efficiency analysis.
10. **Training and Frameworks:** Practical training procedures, hyperparameter tuning, and PyTorch implementation with CUDA concepts.
11. **Dataset Processing:** Data collection, annotation, augmentation, and splitting strategies.
12. **Transformer Architecture:** Self-attention mechanism, multi-head attention, encoder-decoder architecture, and modern large language models.

Each chapter includes detailed mathematical derivations, practical examples, and references to seminal and recent research papers. Upon completing this textbook, readers will be equipped to design, implement, and train neural networks for various applications.

Keywords: Neural Networks, Deep Learning, Machine Learning, CNN, RNN, Transformer, Attention, Self-Attention, Optimization, Backpropagation, PyTorch, Computer Vision, Natural Language Processing, Large Language Models

Acknowledgements

We would like to express our sincere gratitude to all those who have contributed to the development of this textbook.

First and foremost, we thank the pioneering researchers whose groundbreaking work laid the foundation for modern neural networks, including Yann LeCun, Geoffrey Hinton, Yoshua Bengio, and many others whose contributions have shaped the field of deep learning.

We are grateful to our colleagues at Yunnan University for their valuable feedback and suggestions during the development of this material.

Special thanks to the open-source community, particularly the developers of PyTorch, NumPy, and other tools that have made deep learning accessible to researchers and practitioners worldwide.

Finally, we thank our families and friends for their patience and support throughout this project.

The Authors
March 2026

To all the curious minds who seek to understand
the mathematics of learning machines,
and to the future researchers who will push
the boundaries of artificial intelligence.

- Authors -

Contents

Table of contents 18

List of figures 19

List of tables 21

List of listings 23

1 Mathematical Foundations 1
- 1.1 Introduction . . . 1
- 1.2 Linear Algebra . . . 1
  - 1.2.1 Vectors and Matrices . . . 1
  - 1.2.2 Matrix Operations . . . 2
  - 1.2.3 Norms . . . 2
  - 1.2.4 Eigenvalues and Singular Values . . . 3
    - 1.2.4.1 Why SVD Matters: Geometric Interpretation . . . 3
    - 1.2.4.2 Low-Rank Approximation: The Eckart–Young Theorem . . . 3
    - 1.2.4.3 Eigendecomposition and Spectral Theorem . . . 4
    - 1.2.4.4 Condition Number and Numerical Stability . . . 4
    - 1.2.4.5 Positive Definite Matrices and Cholesky Decomposition . . . 5
    - 1.2.4.6 Determinant, Trace, and Jacobian Determinant . . . 6
  - 1.2.5 Tensors . . . 6
- 1.3 Calculus . . . 7
  - 1.3.1 Derivatives and Gradients . . . 7
  - 1.3.2 Higher-Order Derivatives . . . 7
    - 1.3.2.1 Taylor Expansion: The Theoretical Bridge . . . 8
    - 1.3.2.2 Convexity and the Loss Landscape . . . 8
    - 1.3.2.3 Lipschitz Continuity and Gradient Clipping . . . 9
  - 1.3.3 Jacobian Matrix . . . 10
- 1.4 Probability Theory . . . 10
  - 1.4.1 Basic Probability . . . 10
  - 1.4.2 Common Distributions . . . 11
  - 1.4.3 Bayesian Probability . . . 11
  - 1.4.4 Maximum Likelihood Estimation . . . 12
  - 1.4.5 Bayesian Inference and MAP Estimation . . . 13
  - 1.4.6 Conjugate Priors and Exponential Families . . . 13
- 1.5 Optimization . . . 13
  - 1.5.1 Optimization Problem . . . 13
  - 1.5.2 Gradient Descent . . . 14
  - 1.5.3 Challenges in Optimization . . . 14
- 1.6 Information Theory . . . 15
  - 1.6.1 Entropy . . . 15
    - 1.6.1.1 Proof: $D_{KL}(p\|q) \geq 0$ via Jensen's Inequality . . . 16
    - 1.6.1.2 Mutual Information . . . 16
  - 1.6.2 Cross-Entropy and KL Divergence . . . 17
- 1.7 Chapter Summary . . . 18
- 1.8 Further Reading . . . 18
- 1.9 Exercises . . . 19

2 Activation Functions 21
2.1 Introduction . . . 21
2.2 Why Nonlinearity? The Universal Approximation Theorem . . . 21
2.2.1 The Theorem . . . 21
2.2.2 Intuitive Explanation . . . 22
2.2.3 Caveats and Practical Implications . . . 22
2.3 Historical Development of Activation Functions . . . 22
2.4 Neural Network Neurons . . . 23
2.5 Linear Activation Functions . . . 23
2.5.1 Identity Function . . . 23
2.6 Sigmoid Family . . . 24
2.6.1 Sigmoid (Logistic) . . . 24
2.6.1.1 Mathematical Analysis: Why Sigmoid Causes Vanishing Gradients . . . 25
2.6.1.2 Derivative Derivation . . . 25
2.6.2 Hyperbolic Tangent (Tanh) . . . 26
2.7 Rectified Linear Unit (ReLU) Family . . . 26
2.7.1 ReLU . . . 27
2.7.1.1 Why ReLU Prevents Vanishing Gradients: Mathematical Analysis 27
2.7.1.2 ReLU, Sparsity, and Biological Motivation . . . 28
2.7.2 Leaky ReLU . . . 29
2.7.3 Parametric ReLU (PReLU) . . . 29
2.7.4 Exponential Linear Unit (ELU) . . . 29
2.7.5 Swish / SiLU . . . 30
2.7.6 Mish . . . 30
2.7.7 StarReLU . . . 31
2.7.8 GELU (Gaussian Error Linear Unit) . . . 32
2.7.8.1 Probabilistic Interpretation: GELU, Dropout, and the Gaussian CDF . . . 32
2.7.8.2 Derivative of GELU . . . 33
2.8 Special Activation Functions . . . 33
2.8.1 Softmax . . . 33
2.9 Choosing an Activation Function . . . 34
2.10 Practical Considerations . . . 35
2.10.1 Weight Initialization . . . 35
2.10.2 Batch Normalization with Activations . . . 35
2.11 Implementation in PyTorch . . . 35
2.12 Chapter Summary . . . 36
2.13 Further Reading . . . 36
2.14 Exercises . . . 37

3 Loss Functions 39
3.1 Introduction . . . 39
3.2 The Role of Loss Functions . . . 39
3.3 Regression Loss Functions . . . 39
3.3.1 Mean Squared Error (MSE) . . . 39
3.3.1.1 Derivation: MSE as Maximum Likelihood under Gaussian Noise 40
3.3.1.2 Gradient Analysis . . . 40
3.3.1.3 Connection to Variance and R^2 . . . 41
3.3.2 Mean Absolute Error (MAE) . . . 41
3.3.2.1 Derivation: MAE as MLE under Laplace Noise . . . 41

3.3.2.2 Comparison: MSE vs MAE . . . 41
3.3.3 Huber Loss: The Best of Both Worlds . . . 42
3.4 Classification Loss Functions . . . 42
3.4.1 From Likelihood to Cross-Entropy: A Complete Derivation Chain . . . 43
3.4.1.1 Step 1: The Generative Model . . . 43
3.4.1.2 Step 2: Maximum Likelihood . . . 43
3.4.1.3 Step 3: Negative Log-Likelihood = Cross-Entropy . . . 43
3.4.1.4 Step 4: Connection to Information Theory . . . 43
3.4.1.5 Step 5: The Beautiful Gradient . . . 44
3.4.2 Binary Cross-Entropy . . . 44
3.4.3 Categorical Cross-Entropy . . . 44
3.5 Advanced Loss Functions . . . 45
3.5.1 Focal Loss . . . 45
3.5.1.1 Motivation: The Class Imbalance Problem . . . 45
3.5.1.2 The Modulating Factor . . . 45
3.5.1.3 Analysis of the Modulating Factor . . . 45
3.5.1.4 Alpha-Balanced Focal Loss . . . 46
3.5.1.5 Focal Loss Gradient . . . 46
3.5.2 Label Smoothing . . . 46
3.5.2.1 The Mechanism . . . 47
3.5.2.2 Why Does Label Smoothing Work? Three Perspectives . . . 47
3.5.2.3 Practical Impact . . . 48
3.6 Contrastive and Self-Supervised Loss Functions . . . 48
3.6.1 Contrastive Loss . . . 48
3.6.2 Triplet Loss . . . 48
3.6.3 InfoNCE Loss . . . 49
3.6.3.1 Connection to Mutual Information . . . 49
3.6.3.2 The Role of Temperature τ . . . 49
3.6.4 Dice Loss . . . 49
3.6.4.1 Derivation: Dice Coefficient and Jaccard Index . . . 50
3.6.5 KL Divergence Loss . . . 50
3.7 Choosing a Loss Function . . . 51
3.8 Chapter Summary . . . 51
3.9 Further Reading . . . 52

4 Neural Network Architectures **53**
4.1 Introduction . . . 53
4.2 Deep Neural Networks (DNN) . . . 53
4.2.1 Architecture . . . 53
4.3 Convolutional Neural Networks (CNN) . . . 53
4.3.1 Convolutional Layer . . . 54
4.3.2 Receptive Field Analysis . . . 54
4.3.2.1 Single-Layer Receptive Field . . . 54
4.3.2.2 Multi-Layer Receptive Field . . . 55
4.3.2.3 Effective Receptive Field . . . 55
4.3.3 1×1 Convolution: Cross-Channel Mixing . . . 55
4.3.4 Transposed Convolution (Deconvolution) . . . 56
4.4 Recurrent Neural Networks (RNN) . . . 56
4.4.1 Architecture . . . 56
4.4.2 Vanishing Gradient in RNNs: A Mathematical Proof . . . 57
4.4.2.1 Backpropagation Through Time (BPTT) . . . 57

4.4.2.2 Spectral Analysis . . . 57
4.5 Long Short-Term Memory (LSTM) . . . 58
4.5.1 LSTM Gates . . . 58
4.5.2 Why LSTM Solves the Vanishing Gradient . . . 58
4.5.2.1 Gradient Flow Through the Cell State . . . 59
4.5.2.2 Comparison: RNN vs LSTM Gradient . . . 59
4.6 Gated Recurrent Unit (GRU) . . . 60
4.6.1 GRU Equations . . . 60
4.7 Choosing Architecture . . . 60
4.8 Chapter Summary . . . 60
4.9 Further Reading . . . 61

5 Core Operations in Neural Networks 63
5.1 Introduction . . . 63
5.2 Convolution Operation . . . 63
5.2.1 2D Convolution . . . 63
5.2.2 Grouped and Depthwise Convolution . . . 63
5.2.3 Dilated (Atrous) Convolution . . . 64
5.3 Pooling Operations . . . 64
5.3.1 Max Pooling . . . 64
5.3.2 Average Pooling . . . 65
5.3.3 Global Pooling . . . 65
5.4 Normalization Techniques . . . 65
5.4.1 Batch Normalization . . . 65
5.4.2 Layer Normalization . . . 66
5.4.3 Instance Normalization . . . 66
5.4.4 Group Normalization . . . 67
5.5 Regularization Operations . . . 67
5.5.1 Dropout . . . 67
5.5.1.1 Why Dropout Works: Bayesian Interpretation . . . 68
5.5.1.2 Variants of Dropout . . . 68
5.6 Combining Operations . . . 68
5.6.1 Basic Conv Block . . . 68
5.7 Chapter Summary . . . 69
5.8 Further Reading . . . 70

6 Skip Connections and Attention Mechanisms 71
6.1 Introduction . . . 71
6.2 Skip Connections . . . 71
6.2.1 Residual Connections (ResNet) . . . 71
6.2.2 Why Residual Connections Work: Theoretical Analysis . . . 71
6.2.2.1 Gradient Flow Perspective . . . 71
6.2.2.2 Identity Mapping Hypothesis . . . 72
6.2.3 Pre-activation ResNet . . . 72
6.2.4 Bottleneck Architecture . . . 72
6.2.5 ResNeXt: Grouped Convolutions in Residual Blocks . . . 73
6.2.6 Stochastic Depth (DropPath) . . . 73
6.2.7 Dense Connections (DenseNet) . . . 73
6.3 Attention Mechanisms . . . 74
6.3.1 Self-Attention (Scaled Dot-Product) . . . 74
6.3.1.1 Why Scale by $\sqrt{d_k}$? . . . 74
6.3.1.2 Computational Complexity . . . 74

6.3.2 Multi-Head Attention . . . 75
6.3.3 Cross-Attention . . . 75
6.3.4 Positional Encoding . . . 75
6.3.4.1 Sinusoidal Positional Encoding . . . 76
6.3.4.2 Modern Alternatives . . . 76
6.3.5 Squeeze-and-Excitation (SE) Attention . . . 76
6.3.6 CBAM: Convolutional Block Attention Module . . . 77
6.3.7 Window Attention (Swin Transformer) . . . 77
6.3.8 Efficient Attention Variants . . . 77
6.4 Chapter Summary . . . 78
6.5 Further Reading . . . 79

7 Optimizers and Learning Rate Schedulers **81**
7.1 Gradient Descent Optimization . . . 81
7.1.1 Batch Gradient Descent . . . 81
7.1.2 Stochastic Gradient Descent (SGD) . . . 81
7.1.3 Mini-Batch Gradient Descent . . . 81
7.2 Gradient Descent with Momentum . . . 81
7.3 Adaptive Learning Rate Methods . . . 82
7.3.1 AdaGrad . . . 82
7.3.2 RMSProp . . . 82
7.3.3 Adam (Adaptive Moment Estimation) . . . 82
7.3.4 AdamW (Adam with Weight Decay) . . . 83
7.3.5 AdaDelta . . . 83
7.3.6 Unified View of Adaptive Methods . . . 84
7.3.7 Convergence Rate Analysis . . . 84
7.4 Gradient Clipping . . . 84
7.4.1 Value Clipping . . . 84
7.4.2 Norm Clipping . . . 85
7.5 Learning Rate Schedulers . . . 85
7.5.1 Step Decay . . . 85
7.5.2 Exponential Decay . . . 85
7.5.3 Cosine Annealing . . . 85
7.5.4 Warmup . . . 85
7.5.5 Warmup + Cosine Decay . . . 86
7.5.6 ReduceLROnPlateau . . . 86
7.5.7 OneCycleLR . . . 86
7.6 Optimizer Selection . . . 87
7.7 Chapter Summary . . . 87
7.8 Further Reading . . . 87

8 Classic Network Architectures **89**
8.1 Evolution of Neural Networks . . . 89
8.2 Perceptron and Early Neural Networks . . . 89
8.2.1 The Perceptron (1958) . . . 89
8.3 LeNet (1998) . . . 89
8.4 AlexNet (2012) . . . 89
8.5 VGGNet (2014) . . . 90
8.6 GoogLeNet / Inception (2014) . . . 91
8.7 ResNet (2015) . . . 91
8.8 DenseNet (2017) . . . 92
8.9 Transformer (2017) . . . 92

8.10 MobileNet (2017) . . . 93
8.11 SENet (2018) . . . 93
8.12 EfficientNet (2019) . . . 94
8.13 Vision Transformer (ViT) (2020) . . . 94
8.14 U-Net (2015) . . . 95
8.15 ShuffleNet (2018) . . . 95
8.16 Chapter Summary . . . 96
8.17 Further Reading . . . 97

9 Parameter and Computation Analysis **99**
9.1 Importance of Model Efficiency . . . 99
9.2 Parameter Counting . . . 99
9.2.1 Fully Connected Layer . . . 99
9.2.2 Convolutional Layer . . . 99
9.2.3 Batch Normalization . . . 100
9.3 FLOPs (Floating Point Operations) . . . 100
9.3.1 FLOPs for FC Layer . . . 100
9.3.2 FLOPs for Conv Layer . . . 100
9.4 Practical Examples . . . 101
9.4.1 LeNet-5 Parameter Count . . . 101
9.4.2 AlexNet Parameter Count . . . 101
9.5 Transfer Learning and Efficiency . . . 101
9.6 Detailed Model Breakdown . . . 102
9.6.1 ResNet-50 Parameter Analysis . . . 102
9.6.2 Transformer Model Scaling . . . 102
9.7 Activation Memory Analysis . . . 103
9.7.1 Activation Memory Formula . . . 103
9.8 Model Compression and Quantization . . . 104
9.8.1 Quantization . . . 104
9.8.2 Other Compression Techniques . . . 104
9.9 Chapter Summary . . . 104
9.10 Further Reading . . . 105

10 Training Process and Frameworks **107**
10.1 Introduction . . . 107
10.2 The Training Loop . . . 107
10.2.1 Formal Description . . . 107
10.2.2 Why Minibatch SGD Works . . . 108
10.3 Hyperparameters . . . 108
10.3.1 Learning Rate . . . 109
10.3.1.1 Effect of Learning Rate . . . 109
10.3.1.2 Learning Rate Range Test . . . 109
10.3.1.3 Learning Rate Warmup . . . 110
10.3.2 Batch Size . . . 110
10.3.2.1 Batch Size Effects . . . 110
10.3.2.2 Linear Scaling Rule . . . 110
10.4 Regularization . . . 110
10.4.1 Dropout . . . 110
10.4.1.1 Forward Pass with Dropout . . . 111
10.4.1.2 Inverted Dropout . . . 111
10.4.1.3 Ensemble Interpretation . . . 111
10.4.1.4 Dropout as Approximate Bayesian Inference . . . 112

10.4.1.5 Dropout Variants . . . 112
10.4.2 Weight Decay . . . 113
10.4.2.1 L2 Regularization as MAP Estimation . . . 113
10.4.2.2 Weight Decay vs. L2 Regularization: AdamW . . . 113
10.4.3 Other Regularization Techniques . . . 113
10.4.3.1 Gradient Clipping . . . 113
10.4.3.2 Label Smoothing . . . 114
10.4.3.3 Stochastic Depth . . . 114
10.5 Early Stopping . . . 114
10.5.1 Algorithm . . . 114
10.5.2 Early Stopping as Regularization . . . 115
10.6 Backpropagation . . . 115
10.6.1 The Computational Graph Perspective . . . 115
10.6.1.1 Forward Pass . . . 115
10.6.1.2 Backward Pass . . . 115
10.6.2 Example: Two-Layer Network . . . 116
10.6.3 Automatic Differentiation . . . 116
10.6.4 Common Training Pitfalls . . . 117
10.7 Model Checkpointing . . . 117
10.8 Mixed Precision Training . . . 117
10.8.1 Motivation . . . 117
10.8.2 The Loss Scaling Problem . . . 118
10.8.3 BF16: The Modern Alternative . . . 118
10.9 PyTorch Framework . . . 118
10.10 CUDA and GPU Training . . . 120
10.10.1 GPU Programming Model . . . 120
10.10.2 Distributed Training . . . 121
10.10.3 Gradient Accumulation . . . 121
10.11 Training Diagnostics . . . 121
10.11.1 Loss Curves . . . 122
10.11.2 Gradient Statistics . . . 122
10.12 Chapter Summary . . . 122
10.13 Further Reading . . . 123

11 Dataset Processing **125**
11.1 Introduction . . . 125
11.2 Data Collection . . . 125
11.2.1 Data Sources . . . 125
11.2.2 Data Quality Criteria . . . 125
11.3 Data Annotation . . . 126
11.3.1 Annotation Types . . . 126
11.3.2 Annotation Tools . . . 126
11.4 Data Splitting . . . 127
11.4.1 Basic Splits . . . 127
11.4.2 Stratified Splitting . . . 127
11.4.3 Cross-Validation . . . 128
11.5 Data Augmentation . . . 128
11.5.1 Image Augmentations . . . 128
11.5.2 Mixup: Convex Combinations of Samples . . . 129
11.5.3 CutMix: Pasting Patches . . . 130
11.5.4 Advanced Augmentation Policies . . . 131

11.6 Text Data Processing . . . 131
11.6.1 Tokenization . . . 131
11.6.2 Byte Pair Encoding (BPE) . . . 131
11.7 Audio Data Processing . . . 132
11.7.1 Spectrogram Computation . . . 132
11.8 Data Normalization . . . 132
11.8.1 Image Normalization . . . 133
11.9 Data Loading . . . 133
11.10 Handling Data Imbalance . . . 134
11.10.1 Imbalance Ratios . . . 134
11.10.2 Techniques for Imbalanced Data . . . 134
11.11 Data Quality and Versioning . . . 135
11.11.1 Data Versioning . . . 135
11.11.2 Data Provenance . . . 135
11.12 Evaluation Metrics . . . 136
11.12.1 Classification Metrics . . . 136
11.12.2 Segmentation Metrics . . . 136
11.13 Chapter Summary . . . 136
11.14 Further Reading . . . 137

12 Transformer Architecture **139**
12.1 Introduction . . . 139
12.2 The Attention Mechanism . . . 139
12.2.1 Background: RNN Limitations . . . 139
12.2.2 Scaled Dot-Product Attention . . . 139
12.2.3 Self-Attention . . . 140
12.3 Multi-Head Attention . . . 140
12.4 Transformer Architecture . . . 140
12.4.1 Feed-Forward Networks . . . 140
12.4.2 Layer Normalization . . . 140
12.4.3 Residual Connections . . . 141
12.5 Positional Encoding . . . 141
12.6 Encoder-Decoder Architecture . . . 141
12.7 Vision Transformer (ViT) . . . 143
12.8 Large Language Models . . . 143
12.8.1 GPT Architecture . . . 143
12.8.2 BERT Architecture . . . 143
12.8.3 Modern LLMs . . . 143
12.9 Efficiency Considerations . . . 144
12.9.1 Attention Complexity . . . 144
12.9.2 Memory Optimization . . . 144
12.10 Chapter Summary . . . 144
12.11 Further Reading . . . 145

13 Modern Training Techniques **147**
13.1 Introduction . . . 147
13.2 Pre-training and Fine-tuning . . . 147
13.2.1 Pre-training Objectives . . . 147
13.2.2 Fine-tuning Strategies . . . 147
13.3 LoRA: Low-Rank Adaptation . . . 148
13.4 Transfer Learning . . . 149
13.4.1 Feature Transfer . . . 149

13.4.2 Progressive Freezing . . . 149
13.5 Multi-Task Learning . . . 149
13.5.1 Hard Parameter Sharing . . . 149
13.5.2 Soft Parameter Sharing . . . 149
13.5.3 Task Balancing . . . 150
13.6 Continual Learning . . . 150
13.6.1 Catastrophic Forgetting . . . 150
13.6.2 EWC Regularization . . . 150
13.7 Self-Supervised Learning . . . 150
13.7.1 Contrastive Learning . . . 150
13.7.2 Masked Image Modeling . . . 151
13.8 Distillation and Compression . . . 151
13.8.1 Knowledge Distillation . . . 151
13.8.2 Self-Distillation . . . 151
13.9 Large Batch Training . . . 151
13.9.1 Linear Scaling Rule . . . 151
13.9.2 Warmup and Scaling . . . 151
13.10 Efficient Training Techniques . . . 152
13.10.1 Gradient Checkpointing . . . 152
13.10.2 Offloading . . . 152
13.11 Debugging Training . . . 152
13.11.1 Common Issues and Solutions . . . 152
13.11.2 Training Recipes . . . 152
13.12 Chapter Summary . . . 153
13.13 Further Reading . . . 153

Bibliography **155**

A Mathematical Reference **165**
A.1 Linear Algebra . . . 165
A.1.1 Matrix Operations . . . 165
A.1.2 Norms . . . 165
A.1.3 Eigenvalue Decomposition . . . 165
A.1.4 Singular Value Decomposition (SVD) . . . 166
A.2 Calculus . . . 166
A.2.1 Common Derivatives . . . 166
A.2.2 Chain Rule . . . 166
A.2.3 Jacobian and Hessian . . . 167
A.2.4 Taylor Expansion . . . 167
A.2.5 Lagrange Multipliers . . . 167
A.3 Probability and Statistics . . . 167
A.3.1 Bayes' Theorem . . . 167
A.3.2 Distributions . . . 168
A.3.3 Expectation and Variance . . . 168
A.3.4 KL Divergence and Cross-Entropy . . . 168
A.3.5 Information Theory . . . 168

B Quick Reference Tables **169**
B.1 Activation Functions . . . 169
B.2 Loss Functions . . . 169
B.3 Common Datasets . . . 169

C PyTorch Quick Reference **171**
C.1 Common Operations 171
C.2 Model Creation 171
C.3 Training Utilities 172

D Glossary **173**
D.1 Architecture Terms 173
D.2 Training Terms 173
D.3 Layer and Operation Terms 174
D.4 Performance Metrics 174
D.5 Other Common Abbreviations 175

List of Figures

1.1 Singular value spectrum . . . 4
1.2 Condition number and loss contours . . . 5
1.3 Gradient descent on a quadratic loss surface . . . 15
1.4 Schematic of loss landscape with local and global minima . . . 15
1.5 Jensen's inequality visualization . . . 17

2.1 Intuition behind universal approximation . . . 22
2.2 Singe neuron computation . . . 24
2.3 Sigmoid function and its derivative . . . 24
2.4 Tanh function and its derivative . . . 26
2.5 ReLU function and its derivative . . . 27
2.6 Comparison of ReLU variants . . . 30
2.7 Swish activation function . . . 31
2.8 Comparison of activation functions . . . 34

3.1 Regression loss functions compared . . . 42
3.2 Focal loss modulation . . . 46

4.1 Deep Neural Network architecture . . . 53
4.2 Convolution operation . . . 54
4.3 RNN architecture . . . 56
4.4 LSTM cell structure . . . 58

5.1 Convolution with padding . . . 64
5.2 Max pooling 2×2, stride 2 . . . 65

6.1 Residual connection . . . 71
6.2 Self-attention mechanism . . . 74

7.1 Learning rate schedules . . . 86

10.1 Learning rate range test. The steepest descent point α^* provides a good starting learning rate. . . . 109
10.2 Computational graph for a two-layer network. Solid arrows represent forward computation; dashed red arrows represent backward gradient flow. . . . 116
10.3 Training loss curves: healthy training (blue) vs. overfitting (red). Solid lines: training loss. Dashed lines: validation loss. . . . 122

11.1 Annotation granularity: from image-level labels to pixel-level segmentation masks. 126
11.2 Examples of image augmentations: horizontal flip, rotation, random crop, and color jitter. . . . 128

12.1 Transformer layer with residual connections and layer normalization. . . . 141

List of Tables

1.1 Convexity Properties and Their Implications for Optimization . . . 9
1.2 Common Probability Distributions in Deep Learning . . . 11
1.3 Common Conjugate Prior Relationships . . . 14

2.1 Historical Timeline of Activation Function Development . . . 23
2.2 Activation Function Selection Guide . . . 34
2.3 Activation Function Quick Reference . . . 36

3.1 MSE vs MAE: Comprehensive Comparison . . . 41
3.2 Focal Loss Modulating Factor $(1 - p_t)^\gamma$ . . . 46
3.3 Loss Function Selection Guide . . . 51
3.4 Complete Loss Function Decision Framework . . . 52

4.1 Gradient Flow: Vanilla RNN vs LSTM . . . 59
4.2 Architecture Selection Guide . . . 60

5.1 Normalization Comparison . . . 66
5.2 Normalization Methods: Unified View . . . 67
5.3 Dropout Variants . . . 68

6.1 Positional Encoding Methods . . . 76
6.2 Attention Complexity Comparison . . . 78

7.1 Adaptive Optimizer Family . . . 84
7.2 Optimizer Selection Guide . . . 87

8.1 EfficientNet Family . . . 94
8.2 Architecture Comparison . . . 96

9.1 ResNet-50 Layer-by-Layer Parameter Count . . . 102
9.2 Transformer Model Sizes . . . 103
9.3 Quantization Levels . . . 104

10.1 Hyperparameter Ranges and Default Values . . . 111
10.2 Dropout Variants and Their Properties . . . 112
10.3 Comparison of Differentiation Methods . . . 117
10.4 Diagnosing Training Failures . . . 117
10.5 Floating Point Format Comparison . . . 118
10.6 Distributed Training Strategies . . . 121

11.1 Tokenization Strategies . . . 132

12.1 Comparison of Modern LLM Architectures . . . 144

13.1 Common Training Issues and Solutions . . . 152

B.1 Activation Function Reference . . . 169
B.2 Loss Function Reference . . . 169
B.3 Benchmark Datasets . . . 170

Listings

2.1 Activation Functions in PyTorch 35

5.1 Conv Block in PyTorch 69

6.1 Multi-Head Self-Attention in PyTorch 77

7.1 Optimizers in PyTorch 83

10.1 Complete Training Loop in PyTorch 118
10.2 Gradient Accumulation in PyTorch 121

11.1 Mixup and CutMix in PyTorch 130
11.2 Efficient Data Loading in PyTorch 133

12.1 PyTorch Multi-Head Attention Implementation 142

13.1 LoRA Implementation in PyTorch 148

C.1 Common PyTorch tensor operations 171
C.2 PyTorch model definition patterns 171
C.3 Common training utilities 172

Chapter 1

Mathematical Foundations

1.1 Introduction

Neural networks are built upon a solid mathematical foundation that combines linear algebra, calculus, probability theory, and optimization. This chapter provides the essential mathematical prerequisites for understanding how neural networks learn and make predictions. We will cover these topics in the context of their applications in deep learning, providing both theoretical insights and practical intuitions.

The mathematics of neural networks can be viewed as the composition of linear transformations with nonlinear activation functions, optimized through gradient-based methods. Understanding this framework requires fluency in vector and matrix operations, differentiation of multivariate functions, and stochastic optimization.

1.2 Linear Algebra

Linear algebra provides the language for representing and manipulating data in neural networks. Neural networks fundamentally operate through linear transformations (matrix multiplications) combined with nonlinear activation functions.

1.2.1 Vectors and Matrices

A vector is an ordered collection of numbers that can represent a point in n-dimensional space. In neural networks, input data, hidden representations, and parameters are all represented as vectors or higher-dimensional tensors.

Definition 1.1 (Vector)**.** A vector $\mathbf{x} \in \mathbb{R}^n$ is an ordered tuple of n real numbers:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = (x_1, x_2, \dots, x_n)^\top \tag{1.1}$$

where x_i denotes the i-th component of the vector.

Definition 1.2 (Matrix)**.** A matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ is a two-dimensional array of m rows and n columns:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \tag{1.2}$$

where a_{ij} denotes the element in the i-th row and j-th column.

Example 1.1 (Neural Network Layer as Matrix Operation)**.** A single fully-connected

layer in a neural network performs the operation:

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b} \tag{1.3}$$

where:

- $\mathbf{x} \in \mathbb{R}^{d_{in}}$ is the input vector
- $\mathbf{W} \in \mathbb{R}^{d_{out} \times d_{in}}$ is the weight matrix
- $\mathbf{b} \in \mathbb{R}^{d_{out}}$ is the bias vector
- $\mathbf{y} \in \mathbb{R}^{d_{out}}$ is the output vector

If $d_{in} = 784$ (MNIST image) and $d_{out} = 128$, then $\mathbf{W}$ contains $784 \times 128 = 100,352$ parameters.

1.2.2 Matrix Operations

- **Matrix Addition:** $\mathbf{C} = \mathbf{A} + \mathbf{B}$, where $c_{ij} = a_{ij} + b_{ij}$
- **Scalar Multiplication:** $c\mathbf{A}$, where every element is multiplied by c
- **Matrix Multiplication:** $\mathbf{C} = \mathbf{AB}$, where $c_{ij} = \sum_k a_{ik} b_{kj}$
- **Transpose:** $(\mathbf{A}^\top)_{ij} = a_{ji}$
- **Element-wise (Hadamard) Product:** $(\mathbf{A} \odot \mathbf{B})_{ij} = a_{ij} \cdot b_{ij}$

Property 1.1 (Matrix Multiplication Properties)**.** For matrices $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$ of appropriate dimensions:

$$\mathbf{A}(\mathbf{BC}) = (\mathbf{AB})\mathbf{C} \quad \text{(associativity)} \tag{1.4}$$

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC} \quad \text{(distributivity)} \tag{1.5}$$

$$(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top \quad \text{(transpose of product)} \tag{1.6}$$

Note: $\mathbf{AB} \neq \mathbf{BA}$ in general (non-commutativity).

1.2.3 Norms

Norms measure the "size" or "length" of vectors and matrices.

Definition 1.3 (Vector Norms)**.** For a vector $\mathbf{x} \in \mathbb{R}^n$:

- L_1 norm (Manhattan norm): $\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$
- L_2 norm (Euclidean norm): $\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$
- L_p norm: $\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p\right)^{1/p}$
- L_∞ norm (max norm): $\|\mathbf{x}\|_\infty = \max_i |x_i|$

In neural networks, the L_2 norm is commonly used in regularization (weight decay), while the L_1 norm promotes sparsity.

Example 1.2 (Weight Decay Regularization)**.** Weight decay adds a penalty term to the loss function:

$$\mathcal{L}_{reg}(\mathbf{W}) = \mathcal{L}(\mathbf{W}) + \lambda \|\mathbf{W}\|_F^2 \tag{1.7}$$

where $\|\mathbf{W}\|_F = \sqrt{\sum_{i,j} w_{ij}^2}$ is the Frobenius norm (the L_2 norm of the flattened matrix).

1.2.4 Eigenvalues and Singular Values

Eigenvalues and singular values are fundamental for understanding linear transformations, dimensionality reduction, and network analysis.

Definition 1.4 (Eigenvalue and Eigenvector)**.** For a square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, if there exists a scalar λ and nonzero vector $\mathbf{v}$ such that:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v} \tag{1.8}$$

then λ is called an eigenvalue and $\mathbf{v}$ is the corresponding eigenvector.

Definition 1.5 (Singular Value Decomposition)**.** For any matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$, the Singular Value Decomposition (SVD) is:

$$\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top \tag{1.9}$$

where:

- $\mathbf{U} \in \mathbb{R}^{m\times m}$ and $\mathbf{V} \in \mathbb{R}^{n\times n}$ are orthogonal matrices
- $\Sigma \in \mathbb{R}^{m\times n}$ is a diagonal matrix containing singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$

The singular values are related to eigenvalues by $\sigma_i = \sqrt{\lambda_i(\mathbf{A}^\top\mathbf{A})}$.

SVD is crucial for Principal Component Analysis (PCA), which is used in understanding the dimensionality of neural network representations.

1.2.4.1 Why SVD Matters: Geometric Interpretation

To understand why SVD is so fundamental, consider the geometric action of a matrix $\mathbf{A}$. The equation $\mathbf{y} = \mathbf{A}\mathbf{x}$ can be decomposed into three sequential transformations via SVD:

1. $\mathbf{V}^\top$: Rotate (or reflect) the input vector into the "right singular basis."
2. Σ: Scale along each axis by the corresponding singular value σ_i.
3. $\mathbf{U}$: Rotate into the "left singular basis" for the output space.

This geometric view reveals that the singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$ describe exactly how much $\mathbf{A}$ stretches space along each principal direction. In neural networks, the weight matrix $\mathbf{W}$ of a layer performs exactly this kind of transformation on the input activations.

1.2.4.2 Low-Rank Approximation: The Eckart–Young Theorem

One of the most powerful applications of SVD in deep learning is low-rank approximation. Given a rank-r approximation $\hat{\mathbf{A}}_r = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$, the Eckart–Young theorem guarantees:

Theorem 1.1 (Eckart–Young–Mirsky, 1936/1949)**.** For any matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ with SVD $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$, the best rank-$r$ approximation in both the spectral norm and Frobenius norm is given by keeping the top r singular values:

$$\hat{\mathbf{A}}_r = \arg\min_{\operatorname{rank}(\mathbf{B})\leq r} \|\mathbf{A} - \mathbf{B}\|_F = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top \tag{1.10}$$

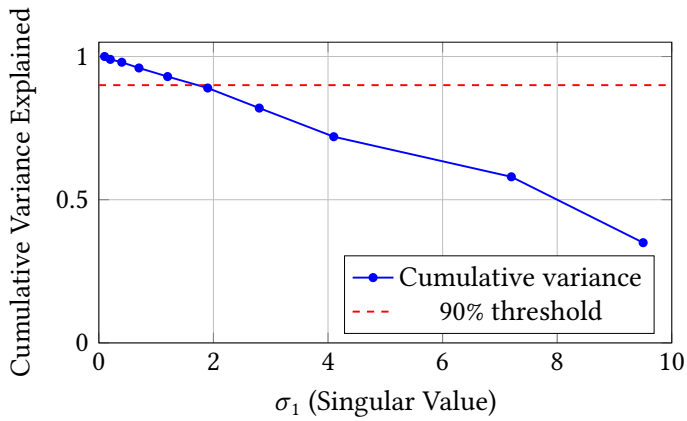


Figure 1.1 – Typical singular value spectrum of a neural network weight matrix. Most variance is captured by the top few singular values, motivating low-rank approximations.

with approximation error $\|\mathbf{A} - \hat{\mathbf{A}}_r\|_F = \sqrt{\sum_{i=r+1}^{\min(m,n)} \sigma_i^2}$.

This theorem underpins modern techniques such as **LoRA** (Low-Rank Adaptation) for efficient fine-tuning of large language models [1], where the weight update $\Delta\mathbf{W}$ is parameterized as a product $\mathbf{BA}$ with rank $r \ll \min(m, n)$, reducing trainable parameters from mn to $(m + n)r$.

1.2.4.3 Eigendecomposition and Spectral Theorem

For symmetric matrices (which arise frequently as Hessian matrices $\mathbf{H}$ in optimization), the eigendecomposition takes a special, simplified form:

Theorem 1.2 (Spectral Theorem)**.** For any real symmetric matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ ($\mathbf{A} = \mathbf{A}^\top$), there exists an orthogonal matrix $\mathbf{Q}$ and a diagonal matrix Λ such that:

$$\mathbf{A} = \mathbf{Q}\Lambda\mathbf{Q}^\top = \sum_{i=1}^{n} \lambda_i \mathbf{q}_i \mathbf{q}_i^\top \tag{1.11}$$

where $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$ are real eigenvalues and $\mathbf{q}_i$ are orthonormal eigenvectors.

The Hessian $\mathbf{H} = \nabla^2\mathcal{L}(\boldsymbol{\theta})$ of a loss function is always symmetric (by Clairaut's theorem on equality of mixed partials). Its eigenvalues determine the local geometry of the loss surface:

- All $\lambda_i > 0$: local minimum (positive definite)
- All $\lambda_i < 0$: local maximum (negative definite)
- Mixed signs: saddle point (indefinite)

1.2.4.4 Condition Number and Numerical Stability

The condition number of a matrix measures how sensitive the solution of $\mathbf{Ax} = \mathbf{b}$ is to perturbations in $\mathbf{b}$:

Definition 1.6 (Condition Number)**.** The condition number of a matrix $\mathbf{A}$ (with respect to the L_2 norm) is:

$$\kappa(\mathbf{A}) = \|\mathbf{A}\|_2 \cdot \|\mathbf{A}^{-1}\|_2 = \frac{\sigma_{\max}(\mathbf{A})}{\sigma_{\min}(\mathbf{A})} \tag{1.12}$$

For a symmetric positive definite matrix, $\kappa(\mathbf{A}) = \lambda_{\max}/\lambda_{\min}$.

In gradient descent, the condition number of the Hessian $\kappa(\mathbf{H})$ directly bounds the convergence rate. For a quadratic objective $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2}\boldsymbol{\theta}^\top \mathbf{H}\boldsymbol{\theta} - \mathbf{b}^\top \boldsymbol{\theta}$, gradient descent with learning rate α converges if and only if:

$$0 < \alpha < \frac{2}{\lambda_{\max}} \tag{1.13}$$

and the optimal learning rate is $\alpha^* = \frac{2}{\lambda_{\max}+\lambda_{\min}}$, giving a convergence rate:

$$\left(\frac{\kappa(\mathbf{H}) - 1}{\kappa(\mathbf{H}) + 1}\right)^2 \tag{1.14}$$

per iteration. When $\kappa(\mathbf{H}) \gg 1$ (ill-conditioned), convergence becomes extremely slow—this is a fundamental motivation for adaptive optimizers like Adam (Chapter 7), which effectively rescale the gradient along each dimension.

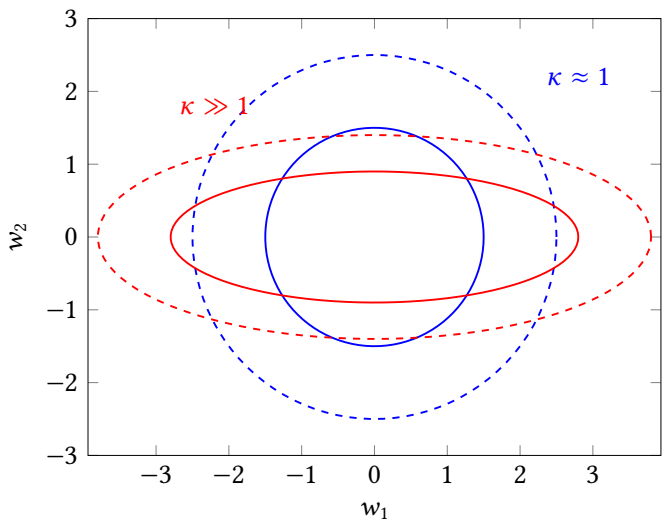


Figure 1.2 – Loss contours for well-conditioned (circular, $\kappa \approx 1$) and ill-conditioned (elliptical, $\kappa \gg 1$) problems. Gradient descent zigzags on ill-conditioned landscapes.

1.2.4.5 Positive Definite Matrices and Cholesky Decomposition

Definition 1.7 (Positive Definite)**.** A symmetric matrix $\mathbf{A}$ is positive definite if $\mathbf{x}^\top \mathbf{A}\mathbf{x} > 0$ for all nonzero $\mathbf{x}$.

For covariance matrices (which are always positive semidefinite), the Cholesky decomposition provides an efficient factorization:

Theorem 1.3 (Cholesky Decomposition)**.** For any symmetric positive definite $\mathbf{A} \in \mathbb{R}^{n\times n}$:

$$\mathbf{A} = \mathbf{L}\mathbf{L}^\top \tag{1.15}$$

where $\mathbf{L}$ is a lower triangular matrix with positive diagonal entries. Computation requires $\frac{n^3}{6}$ FLOPs, half that of LU decomposition.

The Cholesky decomposition is essential for:

- **Sampling from multivariate Gaussians:** If $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$, then $\mathbf{x} = \boldsymbol{\mu} + \mathbf{L}\mathbf{z}$ where $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and $\Sigma = \mathbf{L}\mathbf{L}^\top$.
- **Kalman filters** and **variational inference** in probabilistic deep learning.
- **Solving linear systems** $\mathbf{A}\mathbf{x} = \mathbf{b}$ in optimization subroutines.

1.2.4.6 Determinant, Trace, and Jacobian Determinant

Two scalar summaries of a matrix that appear frequently in deep learning:

Property 1.2 (Key Matrix Properties)**.** For an $n \times n$ matrix $\mathbf{A}$ with eigenvalues $\lambda_1, \dots, \lambda_n$:

$$\det(\mathbf{A}) = \prod_{i=1}^{n} \lambda_i = \text{signed volume scaling factor} \tag{1.16}$$

$$\operatorname{tr}(\mathbf{A}) = \sum_{i=1}^{n} \lambda_i = \sum_{i=1}^{n} a_{ii} \tag{1.17}$$

$$\det(\mathbf{A}) = \exp(\operatorname{tr}(\log \mathbf{A})) \quad \text{(matrix log identity)} \tag{1.18}$$

The **Jacobian determinant** is crucial in normalizing flows and variational autoencoders (VAEs). If $\mathbf{y} = g(\mathbf{x})$ is an invertible transformation, then by the change of variables formula:

$$p_{\mathrm{Y}}(\mathbf{y}) = p_{\mathrm{X}}(g^{-1}(\mathbf{y})) \cdot \left| \det \frac{\partial g^{-1}(\mathbf{y})}{\partial \mathbf{y}} \right| = p_{\mathrm{X}}(\mathbf{x}) \cdot \left| \det \frac{\partial g(\mathbf{x})}{\partial \mathbf{x}} \right|^{-1} \tag{1.19}$$

Normalizing flows design g so that this determinant is cheap to compute, enabling exact likelihood evaluation. Variational autoencoders and normalizing flows for generative modeling are discussed later in this book.

1.2.5 Tensors

Neural networks, especially convolutional neural networks, operate on higher-dimensional arrays called tensors.

Definition 1.8 (Tensor)**.** A tensor is a generalization of vectors and matrices to higher dimensions:

- 0D tensor: scalar
- 1D tensor: vector
- 2D tensor: matrix
- 3D tensor: e.g., RGB images of shape (H, W, C)
- 4D tensor: batches of images of shape (N, H, W, C)
- 5D tensor: volumes of shape (D, H, W, C)

Example 1.3 (Tensor Shapes in CNNs)**.** For a batch of N color images of size $H \times W$ with 3 channels (RGB):

- Input tensor: $\mathbf{X} \in \mathbb{R}^{N \times H \times W \times 3}$

- After convolution with C filters: $\mathbf{Y} \in \mathbb{R}^{N \times H' \times W' \times C}$

PyTorch convention: (N, C, H, W) TensorFlow convention: (N, H, W, C)

1.3 Calculus

Calculus, particularly differential calculus, is essential for understanding how neural networks learn through gradient-based optimization.

1.3.1 Derivatives and Gradients

Definition 1.9 (Derivative)**.** The derivative of a function $f : \mathbb{R} \to \mathbb{R}$ at point x is defined as:

$$f'(x) = \frac{df}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h} \tag{1.20}$$

Definition 1.10 (Gradient)**.** For a scalar function $f : \mathbb{R}^n \to \mathbb{R}$, the gradient is a vector of partial derivatives:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix} \tag{1.21}$$

The gradient points in the direction of steepest ascent.

Property 1.3 (Rules of Differentiation)**.**

- Sum rule: $\frac{d}{dx}(f+g) = f' + g'$
- Product rule: $\frac{d}{dx}(fg) = f'g + fg'$
- Chain rule: $\frac{d}{dx} f(g(x)) = f'(g(x)) \cdot g'(x)$
- Quotient rule: $\frac{d}{dx}\left(\frac{f}{g}\right) = \frac{f'g - fg'}{g^2}$

Example 1.4 (Chain Rule in Neural Networks)**.** For a two-layer network $f(\mathbf{x}) = \mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$, the gradient computation uses the chain rule extensively. If we want $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_1}$ for loss $\mathcal{L}$, we have:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_2} \cdot \frac{\partial \mathbf{z}_2}{\partial \mathbf{z}_1} \cdot \frac{\partial \mathbf{z}_1}{\partial \mathbf{W}_1} \tag{1.22}$$

where $\mathbf{z}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1$ and $\mathbf{z}_2 = \mathbf{W}_2\mathbf{z}_1 + \mathbf{b}_2$.

1.3.2 Higher-Order Derivatives

Definition 1.11 (Hessian Matrix)**.** For a function $f : \mathbb{R}^n \to \mathbb{R}$, the Hessian matrix contains all second-order partial derivatives:

$$\mathbf{H} = \nabla^2 f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix} \tag{1.23}$$

The Hessian is symmetric if all second derivatives are continuous.

The Hessian is used in Newton's method and for understanding curvature of the loss landscape.

1.3.2.1 Taylor Expansion: The Theoretical Bridge

The Taylor expansion is the single most important mathematical tool connecting first-order and second-order optimization methods. For a twice-differentiable function $f : \mathbb{R}^n \to \mathbb{R}$ around point $\mathbf{a}$:

Definition 1.12 (Multivariate Taylor Expansion)**.**

$$f(\mathbf{a} + \boldsymbol{\delta}) = f(\mathbf{a}) + \nabla f(\mathbf{a})^\top \boldsymbol{\delta} + \frac{1}{2}\boldsymbol{\delta}^\top \mathbf{H}(\mathbf{a})\boldsymbol{\delta} + \mathcal{O}(\|\boldsymbol{\delta}\|^3) \tag{1.24}$$

where $\nabla f(\mathbf{a})$ is the gradient and $\mathbf{H}(\mathbf{a}) = \nabla^2 f(\mathbf{a})$ is the Hessian evaluated at $\mathbf{a}$.

Why this matters for neural networks: Nearly all optimization theory can be understood through the lens of local Taylor approximation.

Example 1.5 (Gradient Descent as First-Order Taylor Approximation)**.** Setting $\boldsymbol{\delta} = -\alpha \nabla f(\mathbf{a})$ in the first-order expansion:

$$f(\mathbf{a} - \alpha \nabla f(\mathbf{a})) \approx f(\mathbf{a}) - \alpha \|\nabla f(\mathbf{a})\|^2 \tag{1.25}$$

This guarantees descent as long as $\alpha > 0$ and the linear approximation is valid—i.e., α is not too large.

Example 1.6 (Newton's Method as Second-Order Approximation)**.** Minimizing the second-order Taylor approximation:

$$f(\mathbf{a} + \boldsymbol{\delta}) \approx f(\mathbf{a}) + \nabla f(\mathbf{a})^\top \boldsymbol{\delta} + \frac{1}{2}\boldsymbol{\delta}^\top \mathbf{H}\boldsymbol{\delta} \tag{1.26}$$

Setting the gradient with respect to $\boldsymbol{\delta}$ to zero:

$$\nabla f(\mathbf{a}) + \mathbf{H}\boldsymbol{\delta}^* = 0 \implies \boldsymbol{\delta}^* = -\mathbf{H}^{-1}\nabla f(\mathbf{a}) \tag{1.27}$$

This is Newton's update. It converges quadratically near a minimum but requires $\mathcal{O}(n^3)$ per step for the matrix inverse—prohibitive for deep networks with millions of parameters. Modern quasi-Newton methods (L-BFGS) and approximate second-order methods (K-FAC, Shampoo) attempt to capture curvature information at manageable cost.

1.3.2.2 Convexity and the Loss Landscape

Convexity determines whether gradient descent can find the global minimum:

Definition 1.13 (Convex Function)**.** A function f is **convex** if for all $\mathbf{x}, \mathbf{y}$ and $\lambda \in [0, 1]$:

$$f(\lambda \mathbf{x} + (1 - \lambda)\mathbf{y}) \leq \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}) \tag{1.28}$$

Definition 1.14 (Strongly Convex Function)**.** A twice-differentiable f is μ-strongly convex if:

$$\nabla^2 f(\mathbf{x}) \succeq \mu \mathbf{I} \quad \text{(all eigenvalues of Hessian} \geq \mu > 0) \tag{1.29}$$

The bad news: Neural network loss landscapes are *non-convex*—the Hessian has both positive and negative eigenvalues everywhere. However, a remarkable empirical finding is that **most local minima in overparameterized networks have similar loss values** [2], and the dominant challenge is saddle points rather than poor local minima.

Table 1.1 – Convexity Properties and Their Implications for Optimization

Property	Condition	Implication
Convex	$\nabla^2 f \succeq 0$	All local minima are global
Strictly convex	$\nabla^2 f \succ 0$	Unique global minimum
μ-strongly convex	$\nabla^2 f \succeq \mu \mathbf{I}$	Linear convergence rate
L-smooth	$\lVert\nabla^2 f\rVert \leq L$	Descent guaranteed for $\alpha < 2/L$
PL condition	–	Linear convergence even if non-convex

The Polyak–Łojasiewicz (PL) condition is a relaxation of strong convexity that applies to some non-convex problems:

$$\frac{1}{2}\lVert\nabla f(\mathbf{x})\rVert^2 \geq \mu(f(\mathbf{x}) - f^*) \tag{1.30}$$

where f^* is the optimal value. Many neural network training objectives are conjectured to approximately satisfy this condition, explaining why gradient descent often finds good solutions despite non-convexity.

1.3.2.3 Lipschitz Continuity and Gradient Clipping

Definition 1.15 (L-Lipschitz Continuous)**.** A function f is L-Lipschitz continuous if:

$$|f(\mathbf{x}) - f(\mathbf{y})| \leq L\lVert\mathbf{x} - \mathbf{y}\rVert \quad \forall \mathbf{x}, \mathbf{y} \tag{1.31}$$

Equivalently, $\lVert\nabla f(\mathbf{x})\rVert \leq L$ for differentiable f.

In neural networks, the Lipschitz constant of each layer bounds how much a small change in the input can affect the output. For a linear layer $\mathbf{y} = \mathbf{W}\mathbf{x}$, the Lipschitz constant equals $\lVert\mathbf{W}\rVert_2 = \sigma_{\max}(\mathbf{W})$.

Gradient clipping in RNNs and transformers limits the gradient norm:

$$\mathbf{g} \leftarrow \begin{cases} \mathbf{g} & \text{if } \lVert\mathbf{g}\rVert \leq \tau \\ \frac{\tau}{\lVert\mathbf{g}\rVert}\mathbf{g} & \text{otherwise} \end{cases} \tag{1.32}$$

This prevents gradient explosion, which is particularly problematic for recurrent architectures and large language models.

Example 1.7 (Gradient Descent vs. Newton's Method)**.**

- Gradient descent: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha\nabla\mathcal{L}(\mathbf{w}_t)$
- Newton's method: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha\mathbf{H}^{-1}\nabla\mathcal{L}(\mathbf{w}_t)$

Newton's method uses second-order information (curvature) for potentially faster convergence, but computing the Hessian is expensive for large networks.

1.3.3 Jacobian Matrix

Definition 1.16 (Jacobian)**.** For a vector-valued function $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$, the Jacobian is an $m \times n$ matrix of partial derivatives:

$$\mathbf{J} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix} \tag{1.33}$$

Each row i is the gradient of f_i.

The Jacobian generalizes the gradient to vector-valued functions, which is essential for backpropagation through complex computational graphs.

Example 1.8 (Jacobian in Backpropagation)**.** In backpropagation, we compute gradients of scalar outputs (like loss) with respect to vector inputs. If $\mathbf{y} = f(\mathbf{x})$ and L is a scalar loss, then:

$$\frac{\partial L}{\partial \mathbf{x}} = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}}\right)^\top \frac{\partial L}{\partial \mathbf{y}} = \mathbf{J}^\top \nabla_{\mathbf{y}} L \tag{1.34}$$

The backward pass efficiently computes this product without explicitly forming the full Jacobian.

1.4 Probability Theory

Probability theory provides the framework for understanding uncertainty, making predictions, and designing loss functions.

1.4.1 Basic Probability

Definition 1.17 (Probability)**.** For a random experiment with sample space Ω, a probability function P assigns to each event $A \subseteq \Omega$ a number $P(A)$ such that:

- $0 \leq P(A) \leq 1$ for all events A
- $P(\Omega) = 1$
- If $A_1, A_2, \ldots$ are mutually exclusive, then $P(\bigcup_i A_i) = \sum_i P(A_i)$

Definition 1.18 (Random Variable)**.** A random variable X is a function that maps outcomes of a random experiment to numerical values:

- Discrete: $P(X = x)$ for countable values
- Continuous: described by probability density function $p(x)$

- **Expected Value:** $E[X] = \sum_x x \cdot P(X = x)$ or $\int x \cdot p(x)dx$
- **Variance:** $\mathrm{Var}(X) = E[(X - E[X])^2] = E[X^2] - (E[X])^2$
- **Standard Deviation:** $\sigma_X = \sqrt{\mathrm{Var}(X)}$

1.4.2 Common Distributions

Table 1.2 – Common Probability Distributions in Deep Learning

Distribution	Formula	Applications
Bernoulli	$P(X = 1) = p, P(X = 0) = 1 - p$	Binary classification
Categorical	$P(X = k) = p_k$	Multi-class classification
Gaussian (Normal)	$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$	Regression, weight initialization
Multinomial	Generalization of Bernoulli	Language modeling

Example 1.9 (Softmax Distribution)**.** The softmax function converts a vector of logits $\mathbf{z} \in \mathbb{R}^K$ into a probability distribution:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \tag{1.35}$$

Properties:

- $\sum_{i=1}^{K} \text{softmax}(z_i) = 1$
- $\text{softmax}(z_i) \in (0, 1)$ for all i
- $\arg\max_i \text{softmax}(z_i) = \arg\max_i z_i$

Used for multi-class classification output.

1.4.3 Bayesian Probability

Definition 1.19 (Bayes' Rule)**.**

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)} \tag{1.36}$$

In the context of machine learning:

$$P(\text{hypothesis}|\text{data}) = \frac{P(\text{data}|\text{hypothesis}) \cdot P(\text{hypothesis})}{P(\text{data})} \tag{1.37}$$

- $P(\text{hypothesis})$: Prior probability
- $P(\text{data}|\text{hypothesis})$: Likelihood
- $P(\text{hypothesis}|\text{data})$: Posterior probability

Bayesian methods are used in Bayesian neural networks for uncertainty estimation and in Bayesian optimization for hyperparameter tuning.

1.4.4 Maximum Likelihood Estimation

Maximum Likelihood Estimation (MLE) is the statistical principle underlying most loss functions in deep learning.

Definition 1.20 (Maximum Likelihood Estimation)**.** Given a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ drawn i.i.d. from an unknown distribution, MLE finds the parameter θ that maximizes the probability of observing the data:

$$\theta_{\text{MLE}} = \arg\max_{\theta} \prod_{i=1}^{N} p(y_i \mid \mathbf{x}_i; \theta) \tag{1.38}$$

Equivalently (taking the negative log for numerical stability):

$$\theta_{\text{MLE}} = \arg\min_{\theta} \underbrace{-\sum_{i=1}^{N} \log p(y_i \mid \mathbf{x}_i; \theta)}_{\text{Negative Log-Likelihood (NLL)}} \tag{1.39}$$

The negative log-likelihood (NLL) *is* the loss function. Different model assumptions lead to different loss functions:

Example 1.10 (MSE as MLE under Gaussian Noise)**.** Assume $y = f(\mathbf{x}; \theta) + \epsilon$ where $\epsilon \sim \mathcal{N}(0, \sigma^2)$. Then:

$$p(y \mid \mathbf{x}; \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - f(\mathbf{x}; \theta))^2}{2\sigma^2}\right) \tag{1.40}$$

$$-\log p(y \mid \mathbf{x}; \theta) = \frac{(y - \hat{y})^2}{2\sigma^2} + \frac{1}{2}\log(2\pi\sigma^2) \tag{1.41}$$

Since the constant $\frac{1}{2}\log(2\pi\sigma^2)$ does not depend on θ, minimizing NLL is equivalent to minimizing MSE. This explains why MSE is the "natural" loss for regression—it corresponds to the assumption of additive Gaussian noise.

The gradient of the MSE loss:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial \hat{y}} = \frac{2}{N}(\hat{y} - y) \tag{1.42}$$

is proportional to the residual, meaning large errors receive proportionally large gradients—this is both a strength (fast convergence near the optimum) and a weakness (sensitivity to outliers).

Example 1.11 (Cross-Entropy as MLE for Categorical Distributions)**.** For K-class classification, model the output as a categorical distribution:

$$p(y = k \mid \mathbf{x}; \theta) = \hat{y}_k = \text{softmax}(\mathbf{z})_k \tag{1.43}$$

The negative log-likelihood becomes:

$$\mathcal{L}_{\text{NLL}} = -\sum_{k=1}^{K} y_k \log \hat{y}_k = -\log \hat{y}_c \tag{1.44}$$

where c is the true class. This is exactly the cross-entropy loss. The gradient simplifies beautifully:

$$\frac{\partial \mathcal{L}}{\partial z_k} = \hat{y}_k - y_k \tag{1.45}$$

The gradient is simply the difference between prediction and target—the "error signal." This clean form is one reason softmax + cross-entropy is the standard for classification.

1.4.5 Bayesian Inference and MAP Estimation

While MLE finds the single best parameter, Bayesian inference treats the parameter θ itself as a random variable with a prior distribution $p(\theta)$.

Definition 1.21 (Maximum A Posteriori (MAP) Estimation)**.**

$$\theta_{\text{MAP}} = \arg\max_{\theta} \; p(\theta \mid \mathcal{D}) = \arg\max_{\theta} \; \underbrace{\log p(\mathcal{D} \mid \theta)}_{\text{log-likelihood}} + \underbrace{\log p(\theta)}_{\text{log-prior}} \tag{1.46}$$

Example 1.12 (Weight Decay as MAP with Gaussian Prior)**.** If we place a Gaussian prior $\theta_j \sim \mathcal{N}(0, \lambda^{-1})$ on each weight, then:

$$\log p(\theta) = \sum_j \log \frac{1}{\sqrt{2\pi/\lambda}} \exp\left(-\frac{\lambda\theta_j^2}{2}\right) = -\frac{\lambda}{2}\|\theta\|_2^2 + \text{const} \tag{1.47}$$

Thus, MAP estimation with a Gaussian prior is equivalent to MLE with L_2 regularization (weight decay):

$$\theta_{\text{MAP}} = \arg\min_{\theta} \; \mathcal{L}(\theta) + \frac{\lambda}{2}\|\theta\|_2^2 \tag{1.48}$$

Similarly, a Laplace prior $p(\theta_j) \propto \exp(-\lambda|\theta_j|)$ gives L_1 regularization, promoting sparsity.

1.4.6 Conjugate Priors and Exponential Families

Many common distributions in deep learning belong to the **exponential family**:

$$p(\mathbf{x} \mid \boldsymbol{\eta}) = h(\mathbf{x}) \exp\left(\boldsymbol{\eta}^\top T(\mathbf{x}) - A(\boldsymbol{\eta})\right) \tag{1.49}$$

where $\boldsymbol{\eta}$ are natural parameters, $T(\mathbf{x})$ are sufficient statistics, and $A(\boldsymbol{\eta})$ is the log-partition function.

Property 1.4 (Conjugate Priors)**.** A prior $p(\boldsymbol{\eta})$ is conjugate to a likelihood $p(\mathbf{x} \mid \boldsymbol{\eta})$ if the posterior $p(\boldsymbol{\eta} \mid \mathbf{x})$ belongs to the same distribution family as the prior. Key examples:

1.5 Optimization

Neural network training is fundamentally an optimization problem: finding parameters that minimize a loss function.

1.5.1 Optimization Problem

Problem 1.1 (Neural Network Training)**.** Given:

Table 1.3 – Common Conjugate Prior Relationships

Likelihood	Prior	Posterior	Application
Bernoulli	Beta	Beta	Click-through rate
Multinomial	Dirichlet	Dirichlet	Topic models (LDA)
Gaussian (known σ^2)	Gaussian	Gaussian	Bayesian regression
Gaussian (known μ)	Inverse-Gamma	Inverse-Gamma	Variance estimation
Poisson	Gamma	Gamma	Count data

- A neural network $f(\mathbf{x}; \theta)$ with parameters θ
- A dataset $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$
- A loss function $\mathcal{L}(\theta)$

Find:

$$\theta^* = \arg\min_\theta \mathcal{L}(\theta) \tag{1.50}$$

where typically:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell(f(\mathbf{x}_i; \theta), \mathbf{y}_i) \tag{1.51}$$

1.5.2 Gradient Descent

Algorithm 1 Batch Gradient Descent

1. Initialize parameters $\theta^{(0)}$
2. For $t = 0, 1, 2, \ldots$:
   (a) Compute gradient: $\mathbf{g}_t = \nabla\mathcal{L}(\theta^{(t)})$
   (b) Update parameters: $\theta^{(t+1)} = \theta^{(t)} - \alpha\mathbf{g}_t$
3. Stop when convergence criteria are met

The learning rate $\alpha > 0$ controls the step size. Choosing an appropriate learning rate is crucial for successful training.

1.5.3 Challenges in Optimization

- **Local Minima:** The loss landscape is non-convex, containing many local minima. However, for large neural networks, local minima are often not a significant problem; saddle points are more common.

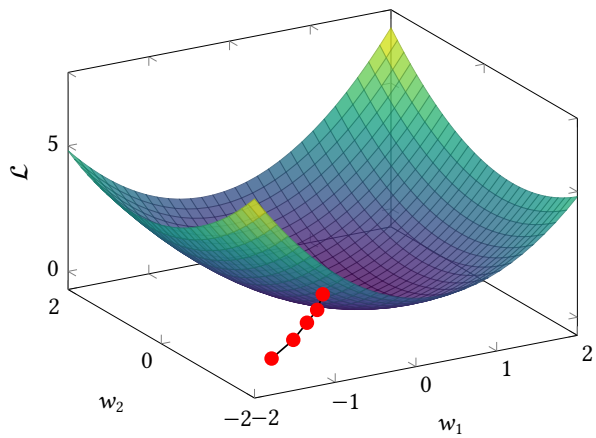


Figure 1.3 – Gradient descent on a quadratic loss surface

- **Vanishing/Exploding Gradients:** In deep networks, gradients can become very small (vanish) or very large (explode), making training difficult.
- **Plateaus:** Flat regions where gradients are near zero, leading to slow progress.
- **Loss Landscapes:** Neural network loss landscapes are complex, with many local minima, saddle points, and flat regions.

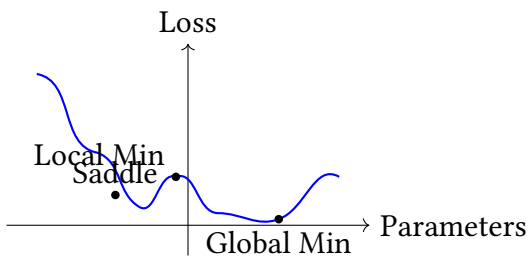


Figure 1.4 – Schematic of loss landscape with local and global minima

1.6 Information Theory

Information theory provides a framework for understanding loss functions, particularly cross-entropy.

1.6.1 Entropy

Definition 1.22 (Shannon Entropy)**.** For a discrete random variable X with probability distribution $p(x)$:

$$H(X) = -\sum_x p(x) \log p(x) = E[-\log p(X)] \tag{1.52}$$

Entropy measures the "uncertainty" or "information content" of a distribution. The logarithm base is typically 2 (bits) or e (nats, natural units).

Historical context: Claude Shannon introduced entropy in his 1948 paper "A Mathematical Theory of Communication" [3], founding information theory. The key insight

is that entropy provides the *unique* (up to a constant) measure of uncertainty satisfying natural desiderata: continuity, monotonicity for equiprobable outcomes, and additivity for independent events.

1.6.1.1 Proof: $D_{KL}(p\|q) \geq 0$ via Jensen's Inequality

The non-negativity of KL divergence, one of the most important results in information theory, follows directly from Jensen's inequality.

Theorem 1.4 (Jensen's Inequality)**.** For a convex function ϕ and random variable X:

$$\phi(E[X]) \leq E[\phi(X)] \tag{1.53}$$

For a concave function, the inequality reverses. Equality holds if and only if X is deterministic (almost surely constant) or ϕ is affine on the support of X.

Non-negativity of KL Divergence. Since $-\log$ is a strictly convex function, by Jensen's inequality:

$$\begin{aligned}
-D_{KL}(p\|q) &= \sum_x p(x) \log \frac{q(x)}{p(x)} && (1.54)\\
&= E_p\left[\log \frac{q}{p}\right] && (1.55)\\
&\leq \log E_p\left[\frac{q}{p}\right] \quad \text{(Jensen's inequality, log is concave)} && (1.56)\\
&= \log\left(\sum_x p(x)\frac{q(x)}{p(x)}\right) && (1.57)\\
&= \log\left(\sum_x q(x)\right) = \log 1 = 0 && (1.58)
\end{aligned}$$

Therefore $D_{KL}(p\|q) \geq 0$, with equality if and only if $p = q$ almost everywhere. □

This proof is elegant but powerful: it tells us that the cross-entropy $H(p, q)$ is always at least as large as the true entropy $H(p)$, and the gap is exactly $D_{KL}(p\|q)$.

1.6.1.2 Mutual Information

Definition 1.23 (Mutual Information)**.** The mutual information between two random variables X and Y is:

$$I(X; Y) = D_{KL}(p(x, y)\|p(x)p(y)) = \sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)} \tag{1.59}$$

Mutual information satisfies:

$$\begin{aligned}
&I(X; Y) = H(X) - H(X \mid Y) = H(Y) - H(Y \mid X) && (1.60)\\
&I(X; Y) = H(X) + H(Y) - H(X, Y) && (1.61)\\
&I(X; Y) \geq 0 \quad \text{with equality iff } X \perp Y && (1.62)
\end{aligned}$$

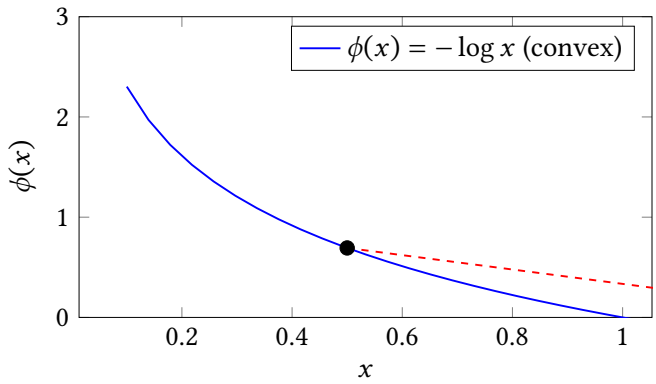


Figure 1.5 – Jensen's inequality for a convex function $\phi(x) = -\log x$: the function evaluated at the mean $E[X]$ lies below the expected value $E[\phi(X)]$.

Applications in deep learning:

- **InfoNCE Loss** (Chapter 3): Contrastive learning maximizes mutual information between augmented views of the same image.
- **Information Bottleneck Theory** [4]: Deep learning is viewed as successive compression of the input $\mathbf{X}$ into representations $\mathbf{T}$ that preserve maximal information about the target $\mathbf{Y}$:

$$\min_{p(\mathbf{t}|\mathbf{x})} I(\mathbf{X};\mathbf{T}) - \beta I(\mathbf{T};\mathbf{Y}) \tag{1.63}$$

- **VAE** [5]: The evidence lower bound (ELBO) can be written as:

$$\log p(\mathbf{x}) \geq \mathbb{E}_{q(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{x}|\mathbf{z})] - D_{KL}(q(\mathbf{z}|\mathbf{x})\|p(\mathbf{z})) \tag{1.64}$$

Property 1.5 (Properties of Entropy).

- $H(X) \geq 0$ for all distributions
- $H(X) = 0$ if and only if $p(x) = 1$ for some x (deterministic)
- $H(X)$ is maximized for the uniform distribution
- Binary entropy: $H_b(p) = -p\log p - (1-p)\log(1-p)$

1.6.2 Cross-Entropy and KL Divergence

Definition 1.24 (Cross-Entropy). For two probability distributions p (true) and q (predicted):

$$H(p,q) = -\sum_x p(x)\log q(x) = H(p) + D_{KL}(p\|q) \tag{1.65}$$

Definition 1.25 (KL Divergence).

$$D_{KL}(p\|q) = \sum_x p(x)\log\frac{p(x)}{q(x)} \tag{1.66}$$

KL divergence measures the "distance" from q to p (not a true metric, as it's asymmetric).

Property 1.6 (KL Divergence Properties).

- $D_{KL}(p\|q) \geq 0$ (non-negativity)

- $D_{KL}(p\|q) = 0$ if and only if $p = q$
- $D_{KL}(p\|q) \neq D_{KL}(q\|p)$ (asymmetric)

Example 1.13 (Cross-Entropy Loss)**.** For classification with N samples and K classes:

$$\mathcal{L}_{CE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik}\log \hat{y}_{ik} \tag{1.67}$$

where y_{ik} is the ground truth (one-hot) and $\hat{y}_{ik} = \text{softmax}(z_{ik})$ is the prediction.

Minimizing cross-entropy is equivalent to minimizing the KL divergence between the true and predicted distributions, since $H(p)$ is constant for one-hot labels.

Derivation of the softmax–cross-entropy gradient: This is one of the most important derivations in deep learning. For a single sample with true class c (one-hot: $y_c = 1$, $y_{k\neq c} = 0$):

$$\mathcal{L} = -\sum_{k=1}^{K} y_k \log \hat{y}_k = -\log \hat{y}_c = -\log \frac{e^{z_c}}{\sum_j e^{z_j}} = -z_c + \log \sum_j e^{z_j} \tag{1.68}$$

$$\frac{\partial \mathcal{L}}{\partial z_k} = -y_k + \frac{e^{z_k}}{\sum_j e^{z_j}} = \hat{y}_k - y_k \tag{1.69}$$

This remarkably clean gradient—prediction minus target—is the cornerstone of classification training. The softmax "cancels" with the cross-entropy logarithm, avoiding the numerical issues that plague computing them separately.

1.7 Chapter Summary

This chapter covered the essential mathematical foundations for neural networks:

1. **Linear Algebra:** Vectors, matrices, tensor operations, and norms are the building blocks for representing and manipulating data and parameters in neural networks.
2. **Calculus:** Derivatives, gradients, and the chain rule are fundamental for computing gradients during backpropagation.
3. **Probability Theory:** Distributions, expected values, and Bayes' rule provide the framework for understanding predictions and designing loss functions.
4. **Optimization:** Gradient descent and its variants are the workhorses of neural network training.
5. **Information Theory:** Entropy and KL divergence justify the use of cross-entropy loss.

These mathematical tools will be applied throughout the textbook as we explore activation functions, loss functions, network architectures, and training algorithms.

1.8 Further Reading

For deeper coverage of these topics:

- [6] - Chapter 2 covers mathematical foundations for deep learning
- [7] - Comprehensive treatment of linear algebra and probability
- [8] - Excellent introduction to linear algebra
- [9] - Classic text on information theory
- [4] - Information bottleneck theory for deep learning
- [2] - Loss surface analysis of neural networks
- [5] - Variational inference and the ELBO
- [1] - Low-rank adaptation via SVD

1.9 Exercises

1. **Linear Algebra:** Given a weight matrix $\mathbf{W} \in \mathbb{R}^{256\times512}$, how many parameters does this layer have if we include a bias term? Write the computational cost in FLOPs for a single forward pass.
2. **Calculus:** Compute the gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{W}}$ for a single-layer network with output $\mathbf{y} = \mathbf{W}\mathbf{x}$, where $\mathcal{L} = \|\mathbf{y} - \mathbf{t}\|^2$.
3. **Probability:** Show that the softmax function is invariant to additive constants: $\text{softmax}(\mathbf{z} + c) = \text{softmax}(\mathbf{z})$ for any constant c.
4. **Optimization:** Implement gradient descent for a simple quadratic function $f(x) = x^2$ and analyze the effect of different learning rates.
5. **Information Theory:** Prove that $D_{KL}(p\|q) \geq 0$ using Jensen's inequality.

Chapter 2

Activation Functions

2.1 Introduction

Activation functions are mathematical functions applied to the output of each neuron (or layer) in a neural network. They introduce **nonlinearity** into the network, enabling it to learn complex patterns that linear functions cannot capture. Without activation functions, no matter how many layers a network has, it would simply perform linear transformations, equivalent to a single linear layer.

The choice of activation function significantly impacts:

- The network's ability to learn complex patterns
- Training stability and convergence speed
- Gradient flow during backpropagation
- The output range of neurons
- Computational efficiency

In this chapter, we will examine the mathematical properties, advantages, disadvantages, and practical applications of various activation functions.

2.2 Why Nonlinearity? The Universal Approximation Theorem

Before surveying individual activation functions, it is worth asking a fundamental question: *Why do we need nonlinear activation functions at all?* The answer lies in one of the most celebrated theoretical results in neural network theory.

2.2.1 The Theorem

Theorem 2.1 (Universal Approximation Theorem [10])**.** Let σ be a non-constant, bounded, continuous activation function. Then for any continuous function $g : [0,1]^n \to \mathbb{R}$ and any $\epsilon > 0$, there exists an integer $N > 0$, real constants $\alpha_i, b_i \in \mathbb{R}$, and real vectors $\mathbf{w}_i \in \mathbb{R}^n$ such that:

$$F(\mathbf{x}) = \sum_{i=1}^{N} \alpha_i \, \sigma\left(\mathbf{w}_i^\top \mathbf{x} + b_i\right) \tag{2.1}$$

satisfies $|F(\mathbf{x}) - g(\mathbf{x})| < \epsilon$ for all $\mathbf{x} \in [0,1]^n$.

What this means: A single hidden layer with a non-polynomial activation function (including ReLU, sigmoid, tanh) can approximate any continuous function to arbitrary precision, given enough neurons.

2.2.2 Intuitive Explanation

Consider a function we wish to approximate on a 1D interval. Each neuron in the hidden layer computes a "bump" or "step" function (via $\sigma(\mathbf{w}^\top \mathbf{x} + b)$), and the output layer combines these with weights α_i to construct the target function. The more neurons we have, the finer the approximation:

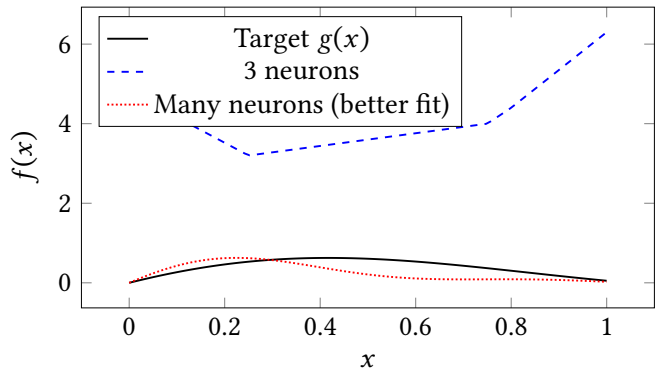


Figure 2.1 – Each neuron contributes a "bump" or "step." More neurons yield finer approximations of the target function.

2.2.3 Caveats and Practical Implications

The universal approximation theorem is an **existence** result, not a **constructive** one. It tells us that a suitable network *exists* but says nothing about:

1. **How many neurons are needed**—the bound is astronomically loose for high-dimensional functions.
2. **Whether gradient descent can find it**—the optimization landscape may have bad local minima.
3. **Whether deeper networks are more efficient**—Cybenko's and Hornik's results apply to single hidden layers, but depth provides *exponential* efficiency gains for compositional functions (see Montufar et al., 2014).

In practice, deep networks (many layers, moderate width) vastly outperform shallow, wide networks. The key insight is that depth enables **hierarchical feature composition**—each layer builds increasingly abstract representations.

2.3 Historical Development of Activation Functions

Understanding the history of activation functions illuminates why certain choices became dominant and how the field evolved:

The story of ReLU's rise is particularly instructive. Before 2012, sigmoid and tanh were the standard choices. However, in deep networks, these saturating functions caused **vanishing gradients**—the gradient through a product of L sigmoid derivatives is at most $(1/4)^L$, which becomes negligible for $L > 10$. The breakthrough of AlexNet [11] demonstrated that ReLU's non-saturating nature ($f'(x) = 1$ for $x > 0$) enables training of much deeper networks, fundamentally changing the field.

Table 2.1 – Historical Timeline of Activation Function Development

Year	Function	Context & Motivation	Key Reference
1943	Threshold (step)	McCulloch–Pitts neuron	McCulloch & Pitts
1957	Perceptron	Linear threshold unit	Rosenblatt
1960	Adaline	Linear activation + sign	Widrow & Hoff
1986	Sigmoid	Differentiable, backpropagation	Rumelhart et al.
1991	Tanh	Zero-centered alternative	
1998	ReLU	Biological inspiration	Hahnloser et al.
2010	Softmax	Probabilistic multi-class	
2011	ReLU (revival)	AlexNet: avoids vanishing gradients	[11]
2015	ELU, SELU	Smooth negative region	Clevert et al.
2015	PReLU	Learnable negative slope	He et al.
2016	Swish/SiLU	Self-gated, non-monotonic	Ramachandran et al.
2016	GELU	Probabilistic gating	Hendrycks & Gimpel
2019	Mish	Self-regularized	Misra
2022	StarReLU	Efficient for transformers	Maulik et al.

2.4 Neural Network Neurons

Before diving into activation functions, let us understand their role in a neuron.

A single artificial neuron computes:

$$y = f(\mathbf{w}^\top \mathbf{x} + b) = f\left(\sum_{i=1}^{n} w_i x_i + b\right) \tag{2.2}$$

where:

- $\mathbf{x} \in \mathbb{R}^n$ is the input vector
- $\mathbf{w} \in \mathbb{R}^n$ is the weight vector
- $b \in \mathbb{R}$ is the bias
- $f(\cdot)$ is the activation function
- $y \in \mathbb{R}$ is the output

2.5 Linear Activation Functions

While not technically "activation functions" in the practical sense, linear functions are sometimes used in output layers.

2.5.1 Identity Function

$$f(x) = x \tag{2.3}$$

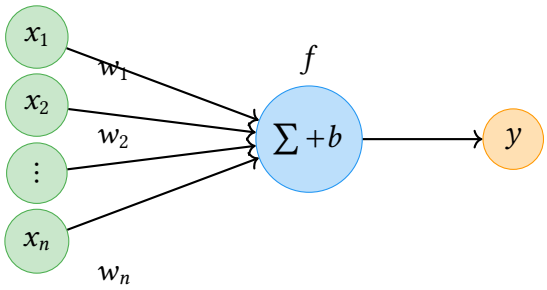


Figure 2.2 – A single neuron computes a weighted sum followed by an activation function

- **Output range:** $(-\infty, \infty)$
- **Derivative:** $f'(x) = 1$
- **Use case:** Regression output layer
- **Not recommended:** Hidden layers (makes the network linear)

2.6 Sigmoid Family

The sigmoid family of activation functions squash their input into the range $(0, 1)$, making them useful for binary classification output layers.

2.6.1 Sigmoid (Logistic)

$$\sigma(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{e^x + 1} \tag{2.4}$$

Properties 2.1 (Properties of Sigmoid).

- **Output range:** $(0, 1)$
- **Derivative:** $\sigma'(x) = \sigma(x)(1 - \sigma(x))$
- **Monotonically increasing**
- **Symmetric around 0.5**
- **Smooth and differentiable everywhere**

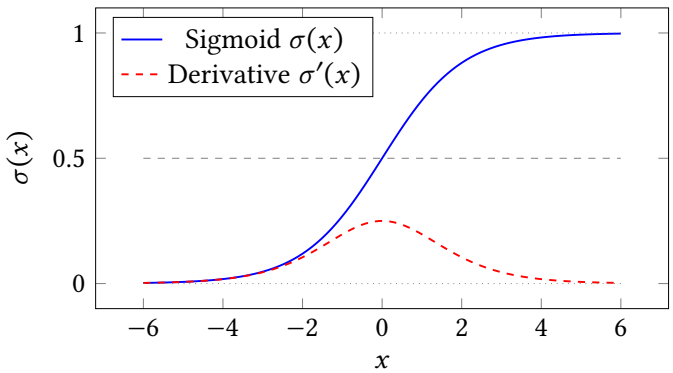


Figure 2.3 – The sigmoid function and its derivative

Advantages 2.1.

Smooth gradient

Outputs can be interpreted as probabilities

Historically widely used

Disadvantages 2.1.

Vanishing gradient: For large $|x|$, $\sigma'(x) \approx 0$

Not zero-centered: Outputs are always positive

Exponential computation: Slower than simpler functions

Saturation: Sigmoid saturates at 0 and 1

2.6.1.1 Mathematical Analysis: Why Sigmoid Causes Vanishing Gradients

Consider an L-layer network where each layer uses sigmoid activation. During backpropagation, the gradient flowing through layer l is multiplied by the derivative of each intermediate layer:

$$\frac{\partial \mathcal{L}}{\partial z_1} = \underbrace{\frac{\partial \mathcal{L}}{\partial z_L}}_{\text{output gradient}} \cdot \prod_{l=2}^{L} \underbrace{\frac{\partial a_l}{\partial z_l}}_{=\sigma'(z_l)} \cdot \underbrace{\frac{\partial z_l}{\partial a_{l-1}}}_{=\mathbf{W}_l} \tag{2.5}$$

Since $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq \frac{1}{4}$ (maximum at $z = 0$), the sigmoid derivative component alone provides a multiplicative factor of at most $(1/4)^{L-1}$:

$$\left\| \frac{\partial \mathcal{L}}{\partial z_1} \right\| \leq \left(\frac{1}{4} \right)^{L-1} \cdot \prod_{l=2}^{L} \|\mathbf{W}_l\| \cdot \left\| \frac{\partial \mathcal{L}}{\partial z_L} \right\| \tag{2.6}$$

For a 50-layer network: $(1/4)^{49} \approx 10^{-30}$. Even with moderate weight norms, the gradient effectively vanishes. This mathematical analysis explains why sigmoid networks deeper than a few layers were essentially untrainable before alternative activation functions were adopted.

2.6.1.2 Derivative Derivation

Starting from $\sigma(x) = \frac{1}{1+e^{-x}}$:

$$\sigma'(x) = \frac{d}{dx}\left((1 + e^{-x})^{-1}\right) = (1 + e^{-x})^{-2} \cdot e^{-x} \tag{2.7}$$

$$= \frac{1}{1 + e^{-x}} \cdot \frac{e^{-x}}{1 + e^{-x}} \tag{2.8}$$

$$= \sigma(x) \cdot (1 - \sigma(x)) \tag{2.9}$$

This elegant form means the gradient can be computed directly from the output, without evaluating the exponential again. The maximum value $\sigma'(0) = 0.25$ occurs at $x = 0$.

Example 2.1 (Sigmoid in Binary Classification)**.** For binary classification, the sigmoid output can be interpreted as:

$$P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) \tag{2.10}$$

The cross-entropy loss with sigmoid is:

$$\mathcal{L} = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] \tag{2.11}$$

where $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$.

2.6.2 Hyperbolic Tangent (Tanh)

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1 \tag{2.12}$$

Properties 2.2 (Properties of Tanh).

- **Output range:** $(-1, 1)$
- **Derivative:** $\tanh'(x) = 1 - \tanh^2(x)$
- **Zero-centered output**
- **Monotonically increasing**

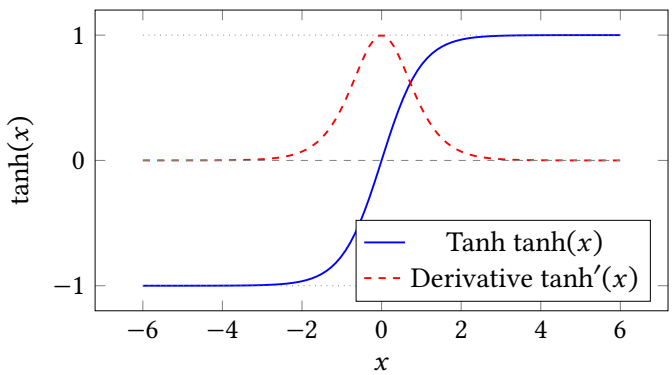


Figure 2.4 – The tanh function and its derivative

Advantages 2.2.

Zero-centered output (stronger gradients)

Stronger gradients than sigmoid

Outputs in $(-1, 1)$, often better for hidden layers

Disadvantages 2.2.

Vanishing gradient: Still present for large $|x|$

Exponential computation

Saturation at ± 1

2.7 Rectified Linear Unit (ReLU) Family

The ReLU family has become the dominant choice for hidden layers in modern deep learning due to their computational efficiency and effectiveness in training deep networks.

2.7.1 ReLU

$$\mathrm{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases} \tag{2.13}$$

Properties 2.3 (Properties of ReLU).

- **Output range:** $[0, \infty)$
- **Derivative:**

$$\mathrm{ReLU}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases} \tag{2.14}$$

- **Computationally efficient**
- **Sparse activation**

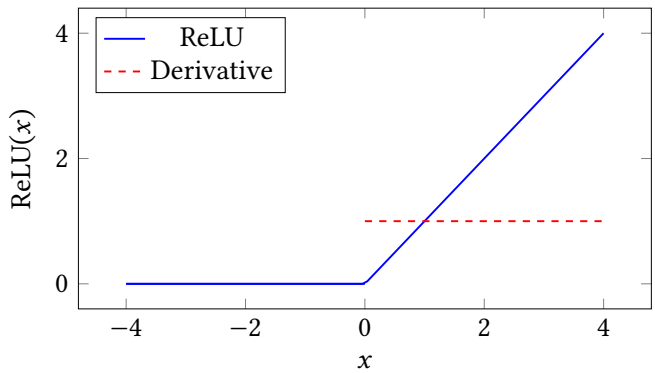


Figure 2.5 – The ReLU function and its derivative

Advantages 2.3.

Very fast computation

Reduces vanishing gradient problem for positive activations

Sparse representations: Not all neurons activate simultaneously

Excellent empirical performance in deep networks

Biological plausibility

2.7.1.1 Why ReLU Prevents Vanishing Gradients: Mathematical Analysis

The key advantage of ReLU over sigmoid is its **non-saturating** nature. For any neuron with a positive pre-activation $z_l > 0$:

$$\mathrm{ReLU}'(z_l) = 1 \quad \text{(no attenuation)} \tag{2.15}$$

In a deep network with ReLU activations, the gradient through the "active" path is:

$$\frac{\partial \mathcal{L}}{\partial z_1} = \frac{\partial \mathcal{L}}{\partial z_L} \cdot \prod_{l=2}^{L} \mathbf{1}_{\{z_l > 0\}} \cdot \mathbf{W}_l \tag{2.16}$$

The indicator $\mathbf{1}_{\{z_l>0\}}$ is either 0 or 1—there is no exponential decay. If roughly half the neurons are active at each layer, the gradient magnitude is:

$$\left\|\frac{\partial \mathcal{L}}{\partial z_1}\right\| \approx \left(\frac{1}{2}\right)^{L-1} \cdot \prod_{l=2}^{L} \|\mathbf{W}_l\| \cdot \left\|\frac{\partial \mathcal{L}}{\partial z_L}\right\| \tag{2.17}$$

While the factor $(1/2)^{L-1}$ still decays exponentially, in practice this is mitigated by:

- **He initialization** [12]: Setting weight variance to $2/n_{\text{in}}$ compensates for the expected fraction of inactive neurons.
- **Residual connections** [13]: Skip connections provide an alternative gradient path that bypasses multiplicative attenuation entirely (Chapter 6).
- **Batch normalization** [14]: Stabilizes the distribution of activations, keeping them in the non-saturated regime.

2.7.1.2 ReLU, Sparsity, and Biological Motivation

ReLU's $\max(0, x)$ operation creates sparse activations: for typical initialization, roughly 50% of neurons output zero. This sparsity has several benefits:

1. **Information disentanglement:** Sparse representations tend to be more interpretable and robust (similar to sparse coding in neuroscience).
2. **Efficiency:** Zero-valued neurons contribute nothing to downstream computation, enabling hardware optimization.
3. **Biological plausibility:** Biological neurons exhibit a threshold behavior—they fire only when input exceeds a threshold. ReLU captures this qualitatively, unlike the smooth sigmoid.

Disadvantages 2.3.

Dying ReLU problem: Neurons can "die" if they always output zero

Not zero-centered

Unbounded output

Note 2.1 (Dying ReLU Problem)**.** If a neuron's weights are updated such that $\mathbf{w}^\top\mathbf{x}+b \leq 0$ for all inputs, the neuron will output zero forever. This is called the "dying ReLU" or "dead neuron" problem. The neuron stops learning because the gradient is zero when the input is negative.

Solutions include:

- Leaky ReLU
- PReLU
- ELU
- Proper weight initialization
- Lower learning rates

2.7.2 Leaky ReLU

$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} = \max(0, x) + \alpha \min(0, x) \tag{2.18}$$

where α is a small positive constant (typically $\alpha = 0.01$).

Properties 2.4 (Properties of Leaky ReLU).

- **Output range:** $(-\infty, \infty)$
- **Derivative:**

$$\text{LeakyReLU}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ \alpha & \text{if } x \leq 0 \end{cases} \tag{2.19}$$

- **Small negative values allowed**

Advantages 2.4.

No dying ReLU problem

Allows small negative gradients

Approximately zero-centered

Disadvantages 2.4.

Hyperparameter α needs tuning

Performance may not be significantly better than ReLU

2.7.3 Parametric ReLU (PReLU)

$$\text{PReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} \tag{2.20}$$

where α is a learnable parameter (can be per channel).

Note 2.2. PReLU allows the network to learn the optimal slope for negative values, potentially improving performance.

2.7.4 Exponential Linear Unit (ELU)

$$\text{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{if } x \leq 0 \end{cases} \tag{2.21}$$

where $\alpha > 0$ is a hyperparameter.

Properties 2.5 (Properties of ELU).

- **Output range:** $(-\alpha, \infty)$
- **Smooth for** $x < 0$
- **Negative values bring mean closer to zero**

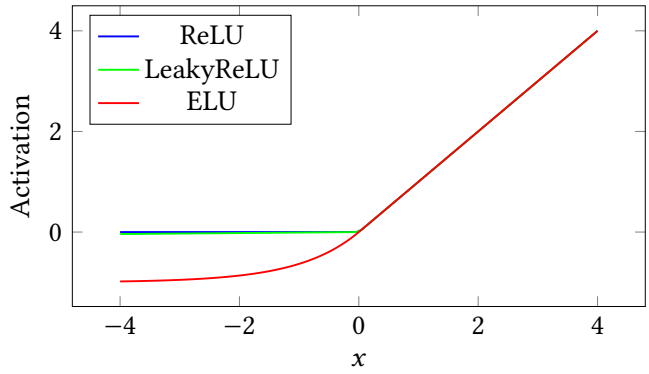


Figure 2.6 – Comparison of ReLU and its variants

Advantages 2.5.

No dying ReLU problem

Outputs closer to zero mean

Smoother gradients

Disadvantages 2.5.

Exponential computation for negative values

Slower forward pass

2.7.5 Swish / SiLU

$$\text{Swish}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \tag{2.22}$$

Proposed by Ramachandran et al. (2017), Swish often outperforms ReLU in deeper networks.

Properties 2.6 (Properties of Swish).

- **Output range:** Approximately $(-0.31, \infty)$
- **Self-gated**
- **Non-monotonic** (dip for small negative values)
- **Derivative:** $\text{Swish}'(x) = \sigma(x) + x \cdot \sigma(x)(1 - \sigma(x)) = \text{Swish}(x) + \sigma(x)(1 - \text{Swish}(x))$

2.7.6 Mish

Proposed by Misra (2019) [15], Mish is a self-regularized, non-monotonic activation function:

$$\text{Mish}(x) = x \cdot \tanh(\text{softplus}(x)) = x \cdot \tanh(\ln(1 + e^x)) \tag{2.23}$$

Properties 2.7 (Properties of Mish).

- **Output range:** Approximately $(-0.308, \infty)$ (bounded below, unbounded above)
- **Non-monotonic:** Similar to Swish, dips slightly for negative x
- **Smooth everywhere:** Infinitely differentiable (C^∞)

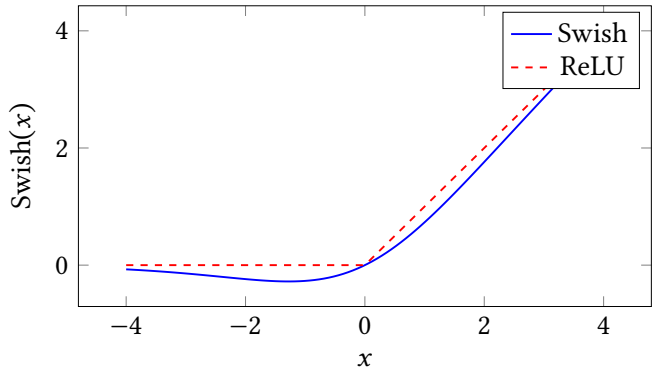


Figure 2.7 – Swish vs ReLU

- **Self-regularizing:** The non-monotonic shape acts as an implicit regularizer

The derivative is:

$$\text{Mish}'(x) = \frac{e^x \cdot (4(e^x + 2)) + 4x \cdot e^{2x} + 2 \cdot e^{3x} + x \cdot e^{4x} + 2}{(2e^x + 2)^2} \tag{2.24}$$

While the derivative is complex, the key insight is that Mish maintains a positive gradient throughout (like ReLU) but with a smoother, bounded gradient for negative inputs that provides gentle regularization. Empirically, Mish consistently matches or outperforms ReLU and Swish across image classification benchmarks (ImageNet, CIFAR-10/100) and object detection tasks.

Advantages 2.6.

Stronger performance than ReLU/Swish on vision tasks

Smooth, non-monotonic shape provides implicit regularization

No additional hyperparameters

Disadvantages 2.6.

More expensive computation (tanh + softplus)

Complex derivative formula

2.7.7 StarReLU

Introduced by Maulik et al. (2024) [16], StarReLU is a self-gated activation function specifically designed for transformers:

$$\text{StarReLU}(x) = s \cdot \text{ReLU}(x)^2 + \beta \tag{2.25}$$

where s and β are learnable parameters (initialized to $s = 1, \beta = 0$).

Motivation: In transformers, the computational bottleneck is the activation function applied to every element of the $N \times d$ hidden state tensor. StarReLU's quadratic form $\text{ReLU}(x)^2$ is simpler than GELU/Swish (no tanh or σ) while providing:

- **Self-gating:** The squaring operation naturally gates negative inputs to zero.

- **Learnable scale and shift:** s and β adapt the activation range.
- **Computational efficiency:** 60% fewer FLOPs than GELU in transformer blocks, with no accuracy loss.

StarReLU represents the trend of designing activation functions that are **hardware-aware**—exploiting the fact that simple operations (multiply, add) are far cheaper than transcendental functions (exp, tanh, erf) on modern hardware.

2.7.8 GELU (Gaussian Error Linear Unit)

$$\mathrm{GELU}(x) = x \cdot \Phi(x) \tag{2.26}$$

where $\Phi(x)$ is the CDF of the standard normal distribution:

$$\Phi(x) = \frac{1}{2}\left[1 + \mathrm{erf}\left(\frac{x}{\sqrt{2}}\right)\right] \tag{2.27}$$

A computationally efficient approximation:

$$\mathrm{GELU}(x) \approx \frac{x}{2} \cdot \left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}\left(x + 0.044715x^3\right)\right)\right) \tag{2.28}$$

GELU is used in BERT, GPT, and many transformer architectures.

2.7.8.1 Probabilistic Interpretation: GELU, Dropout, and the Gaussian CDF

The GELU function has a deep probabilistic interpretation that connects it to dropout [17]. Consider a stochastic neuron where the input x is multiplied by a random Bernoulli mask $m \sim \mathrm{Bernoulli}(\Phi(x))$:

$$y = x \cdot m, \quad P(m = 1 \mid x) = \Phi(x) \tag{2.29}$$

The expected output of this stochastic neuron is:

$$\mathbb{E}[y \mid x] = x \cdot \Phi(x) = \mathrm{GELU}(x) \tag{2.30}$$

Key insight: GELU is the *deterministic expectation* of a stochastically gated neuron where the gate probability equals the Gaussian CDF. This provides a unified view:

- **Dropout** randomly zeroes activations with probability p (input-independent).
- **GELU** adaptively gates activations based on their magnitude (input-dependent).
- Both can be viewed as multiplicative noise, but GELU's noise is *adaptive*—larger inputs are more likely to pass through.

This interpretation explains why GELU works so well in transformers: it provides a smooth, "soft" version of dropout that depends on the input magnitude, offering implicit regularization without the training instability of random dropout in attention mechanisms.

2.7.8.2 Derivative of GELU

From $\text{GELU}(x) = x\Phi(x)$, by the product rule:

$$\text{GELU}'(x) = \Phi(x) + x\phi(x) \tag{2.31}$$

where $\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$ is the standard normal PDF. Note that $\text{GELU}'(x) > 0$ for all x (the function is strictly increasing) but $\text{GELU}'(x) < 1$ (unlike ReLU), providing natural gradient scaling.

Properties 2.8 (Properties of GELU).

- **Nonlinear in all regions**
- **Approximately equal to:** $0.5x(1 + \tanh(\sqrt{2/\pi}(x + 0.044715x^3)))$
- **Zero-centered output**
- **Stochastic regularizer** when used with dropout

Advantages 2.7.

State-of-the-art performance in transformers

Combines benefits of ReLU and dropout

Smooth and non-monotonic

2.8 Special Activation Functions

2.8.1 Softmax

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{K} e^{x_j}} \quad \text{for } i = 1, \dots, K \tag{2.32}$$

Softmax is used for multi-class classification output layers.

Properties 2.9 (Properties of Softmax).

- **Output range:** Each output in $(0, 1)$
- **Sum constraint:** $\sum_i \text{softmax}(x_i) = 1$
- **Probabilistic interpretation**
- **Shift invariant:** $\text{softmax}(x + c) = \text{softmax}(x)$ for any constant c

Advantages 2.8.

Outputs are valid probability distributions

Enables cross-entropy loss optimization

Handles multi-class classification naturally

Disadvantages 2.7.

Only used in output layers (not hidden layers)

Can saturate when logits are very large

Note 2.3 (Softmax in Practice)**.** For numerical stability, implement softmax as:

$$\text{softmax}(x_i) = \frac{e^{x_i - \max(x)}}{\sum_j e^{x_j - \max(x)}} \tag{2.33}$$

This subtracts the maximum value to prevent overflow in the exponential.

2.9 Choosing an Activation Function

The choice of activation function depends on the specific task and network architecture. Here are general guidelines:

Table 2.2 – Activation Function Selection Guide

Layer Type	Recommended Functions	Notes
Hidden (General)	ReLU, GELU	ReLU is the default choice
Hidden (Deep)	ReLU, GELU, ELU	Avoid sigmoid/tanh in deep networks
Hidden (CNN)	ReLU, GELU	LeakyReLU for medical imaging
Hidden (RNN)	Tanh	Often default for RNNs
Hidden (Transformer)	GELU	Used in BERT, GPT, etc.
Output (Binary)	Sigmoid	For probability output
Output (Multi-class)	Softmax	For probability distribution
Output (Regression)	Identity / None	For unbounded output
Output (Bounded)	Sigmoid, Tanh	For $[0, 1]$ or $[-1, 1]$ output

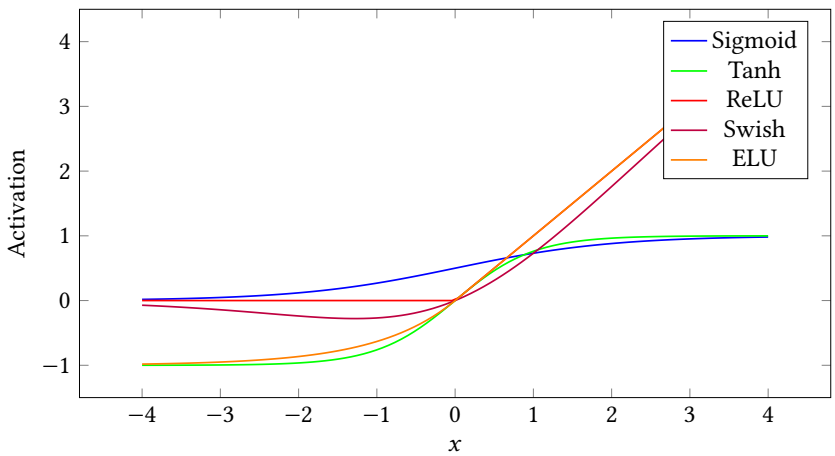


Figure 2.8 – Comparison of various activation functions

2.10 Practical Considerations

2.10.1 Weight Initialization

Different activation functions work best with appropriate weight initialization:

- **Sigmoid/Tanh:** Xavier/Glorot initialization ($\mathcal{N}(0, \sqrt{2/(n_{in} + n_{out})})$)
- **ReLU (and variants):** He initialization ($\mathcal{N}(0, \sqrt{2/n_{in}})$)
- **GELU:** He initialization works well

See Chapter 7 on Optimization for more details.

2.10.2 Batch Normalization with Activations

When using batch normalization, the activation function typically comes **after** batch norm:

$$\mathbf{y} = f(\mathrm{BN}(\mathbf{W}\mathbf{x} + \mathbf{b})) \tag{2.34}$$

This ordering was popularized by the original BatchNorm paper and is now standard practice.

2.11 Implementation in PyTorch

Listing 2.1 – Activation Functions in PyTorch

```python
import torch
import torch.nn as nn

# Common activations
relu = nn.ReLU()
leaky_relu = nn.LeakyReLU(0.01)  # slope for negative values
prelu = nn.PReLU()  # learnable slope
elu = nn.ELU(alpha=1.0)
selu = nn.SELU()  # Self-Normalizing Neural Networks
gelu = nn.GELU()  # Used in Transformers
silu = nn.SiLU()  # Same as Swish
sigmoid = nn.Sigmoid()
tanh = nn.Tanh()
softmax = nn.Softmax(dim=1)

# Example usage
x = torch.randn(32, 128)  # batch of 32, 128 features
y = relu(x)  # Apply ReLU

# Softmax with dimension specification
logits = torch.randn(32, 10)  # 10 classes
probs = softmax(dim=1)(logits)

# GELU in transformers
transformer_layer = nn.TransformerEncoderLayer(
    d_model=512,
    nhead=8,
```

```
    activation='gelu'  # or use nn.GELU() directly
)
```

2.12 Chapter Summary

1. **Activation functions introduce nonlinearity** into neural networks, enabling them to learn complex patterns.
2. **Sigmoid and Tanh** squash inputs to bounded ranges but suffer from vanishing gradients.
3. **ReLU** is computationally efficient and reduces vanishing gradients but can suffer from dying neurons.
4. **Variants like Leaky ReLU, PReLU, and ELU** address the dying ReLU problem.
5. **GELU** has become popular in transformer architectures.
6. **Softmax** is used for multi-class classification output layers.
7. **Choice depends on** task, architecture, and training stability.

Table 2.3 – Activation Function Quick Reference

Function	Formula	Range	Common Use
Sigmoid	$1/(1+e^{-x})$	$(0,1)$	Binary output
Tanh	$(e^x - e^{-x})/(e^x + e^{-x})$	$(-1,1)$	RNN, rarely CNN
ReLU	$\max(0,x)$	$[0,\infty)$	Default hidden
Leaky ReLU	$\max(\alpha x, x)$	$(-\infty,\infty)$	Prevent dying
ELU	x if $x>0$, else $\alpha(e^x-1)$	$(-\alpha,\infty)$	Better gradients
GELU	$x \cdot \Phi(x)$	varies	Transformers
Softmax	$e^{x_i}/\sum e^{x_j}$	$(0,1)^K$	Multi-class output

2.13 Further Reading

- [10] - Universal Approximation Theorem: theoretical foundations
- Ramachandran et al., "Searching for Activation Functions" (2017) - Systematic comparison of activation functions
- Hendrycks and Gimpel, "Gaussian Error Linear Units (GELU)" (2016) - Introduction of GELU
- [17] - Unified view of GELU activation and dropout
- Clevert et al., "Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs)" (2015)
- [15] - Mish: A self-regularized non-monotonic neural activation function

- [16] - StarReLU: Efficient self-gated activation for transformers
- Goodfellow et al., "Deep Learning" Chapter 6
- [12] - Delving deep into rectifiers: He initialization

2.14 Exercises

1. **Gradient Analysis:** Compute the gradient of the ELU function for both positive and negative inputs.
2. **Comparison:** Plot the activation functions covered in this chapter and their derivatives using Python/Matplotlib.
3. **Dying ReLU:** Implement a simple experiment to demonstrate the dying ReLU problem and show how LeakyReLU addresses it.
4. **Numerical Stability:** Show why subtracting $\max(x)$ from logits before computing softmax improves numerical stability.
5. **Custom Activation:** Design and implement a custom activation function that combines properties of ReLU and GELU. Test it on a simple image classification task.
6. **Self-Normalization:** Research SELU (Scaled Exponential Linear Unit) and explain how it enables self-normalizing neural networks.

Chapter 3

Loss Functions

3.1 Introduction

Loss functions (also called objective functions, cost functions, or error functions) are the mathematical measures that quantify how well a neural network's predictions match the true targets. The training process aims to minimize this loss by adjusting the network's parameters through gradient-based optimization.

The choice of loss function depends on:

- The type of task (classification, regression, etc.)
- The distribution of the target variable
- The desired properties of the solution (sparsity, robustness, etc.)
- Computational considerations

3.2 The Role of Loss Functions

Given a dataset $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$ and a model $f(\mathbf{x}; \theta)$, the empirical risk is:

$$\mathcal{R}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell(f(\mathbf{x}_i; \theta), \mathbf{y}_i) \tag{3.1}$$

where $\ell(\cdot, \cdot)$ is the loss function.

Training seeks to find:

$$\theta^* = \arg\min_{\theta} \mathcal{R}(\theta) \tag{3.2}$$

The loss function should:

1. Be differentiable (for gradient-based optimization)
2. Penalize larger errors appropriately
3. Encourage correct behavior for the specific task
4. Have useful statistical interpretations

3.3 Regression Loss Functions

Regression tasks predict continuous values. The choice of loss function encodes assumptions about the noise distribution and determines how the model handles outliers.

3.3.1 Mean Squared Error (MSE)

The Mean Squared Error (L2 loss) is the most common regression loss:

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2 = \frac{1}{N} \sum_{i=1}^{N} e_i^2 \tag{3.3}$$

where $e_i = y_i - \hat{y}_i$ is the residual.

3.3.1.1 Derivation: MSE as Maximum Likelihood under Gaussian Noise

The statistical justification for MSE comes from maximum likelihood estimation. Suppose the data is generated by:

$$y_i = f(\mathbf{x}_i; \theta) + \epsilon_i, \quad \epsilon_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2) \tag{3.4}$$

The likelihood of a single observation is:

$$p(y_i \mid \mathbf{x}_i; \theta, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \hat{y}_i)^2}{2\sigma^2}\right) \tag{3.5}$$

The negative log-likelihood (NLL) for the full dataset:

$$\mathcal{L}_{\text{NLL}} = -\sum_{i=1}^{N} \log p(y_i \mid \mathbf{x}_i; \theta, \sigma^2) \tag{3.6}$$

$$= -\sum_{i=1}^{N} \left[-\frac{1}{2}\log(2\pi\sigma^2) - \frac{(y_i - \hat{y}_i)^2}{2\sigma^2}\right] \tag{3.7}$$

$$= \frac{N}{2}\log(2\pi\sigma^2) + \frac{1}{2\sigma^2}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 \tag{3.8}$$

Since $\frac{N}{2}\log(2\pi\sigma^2)$ is a constant that does not depend on θ:

$$\boxed{\arg\min_{\theta} \mathcal{L}_{\text{NLL}} = \arg\min_{\theta} \sum_{i=1}^{N}(y_i - \hat{y}_i)^2 = \arg\min_{\theta} \mathcal{L}_{MSE}} \tag{3.9}$$

Takeaway: Minimizing MSE is equivalent to fitting a model under the assumption of additive, i.i.d. Gaussian noise. This is why MSE works well when errors are roughly Gaussian but fails when the noise distribution has heavy tails (e.g., Laplace noise).

3.3.1.2 Gradient Analysis

The gradient of MSE with respect to the prediction:

$$\frac{\partial \mathcal{L}_{MSE}}{\partial \hat{y}_i} = -\frac{2}{N}(y_i - \hat{y}_i) = \frac{2}{N}e_i \tag{3.10}$$

The gradient is **linear in the error**—a double-edged sword:

- **Near the optimum** ($e_i \approx 0$): Gradient is small, enabling precise convergence.
- **Far from the optimum** or **outliers** ($|e_i| \gg 0$): Gradient is very large, causing overshooting and excessive sensitivity to outliers.

3.3.1.3 Connection to Variance and R^2

For a model $\hat{y}_i = f(\mathbf{x}_i)$, the total variance decomposition holds:

$$\underbrace{\sum_i (y_i - \bar{y})^2}_{\text{Total SS}} = \underbrace{\sum_i (\hat{y}_i - \bar{y})^2}_{\text{Explained SS}} + \underbrace{\sum_i (y_i - \hat{y}_i)^2}_{\text{Residual SS (MSE} \times N)} \tag{3.11}$$

This gives the R^2 score: $R^2 = 1 - \frac{\text{MSE}}{\text{Var}(y)}$.

3.3.2 Mean Absolute Error (MAE)

The Mean Absolute Error (L1 loss):

$$\mathcal{L}_{MAE} = \frac{1}{N} \sum_{i=1}^{N} |y_i - \hat{y}_i| \tag{3.12}$$

3.3.2.1 Derivation: MAE as MLE under Laplace Noise

If the noise follows a Laplace (double exponential) distribution:

$$\epsilon_i \sim \text{Laplace}(0, b), \quad p(\epsilon) = \frac{1}{2b} \exp\left(-\frac{|\epsilon|}{b}\right) \tag{3.13}$$

Then:

$$-\log p(y_i \mid \mathbf{x}_i; \theta) = \log(2b) + \frac{|y_i - \hat{y}_i|}{b} \tag{3.14}$$

$$\propto |y_i - \hat{y}_i| \tag{3.15}$$

Thus MAE is the MLE loss under Laplace noise. The Laplace distribution has heavier tails than the Gaussian, making MAE inherently more robust to outliers.

3.3.2.2 Comparison: MSE vs MAE

Table 3.1 – MSE vs MAE: Comprehensive Comparison

Property	**MSE (L_2)**	**MAE (L_1)**
Noise assumption	Gaussian $\mathcal{N}(0, \sigma^2)$	Laplace Laplace$(0, b)$
Outlier sensitivity	High (quadratic penalty)	Low (linear penalty)
Gradient w.r.t. $\hat{y}$	$-2(y - \hat{y})/N$	$-\text{sign}(y - \hat{y})/N$
At optimum $\hat{y}^*$	Mean $\bar{y}$	Median $\tilde{y}$
Smoothness	Smooth everywhere	Non-differentiable at 0
Convergence speed	Faster (quadratic near min)	Slower (constant gradient)
Statistical efficiency	Higher (for Gaussian data)	Lower

Key insight: The difference between MSE and MAE ultimately boils down to *what you believe about the noise distribution.* If you expect occasional large outliers, MAE is more appropriate. If the errors are truly Gaussian, MSE is statistically optimal.

3.3.3 Huber Loss: The Best of Both Worlds

Huber loss [18] smoothly interpolates between MSE (for small errors) and MAE (for large errors):

$$\mathcal{L}_{\text{Huber}}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{if } |y - \hat{y}| \leq \delta \\ \delta \cdot |y - \hat{y}| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases} \tag{3.16}$$

The hyperparameter δ (typically $\delta = 1$) controls the transition point:

- For $|e| < \delta$: Quadratic penalty (like MSE)—efficient optimization near the optimum.
- For $|e| \geq \delta$: Linear penalty (like MAE)—robust to outliers.

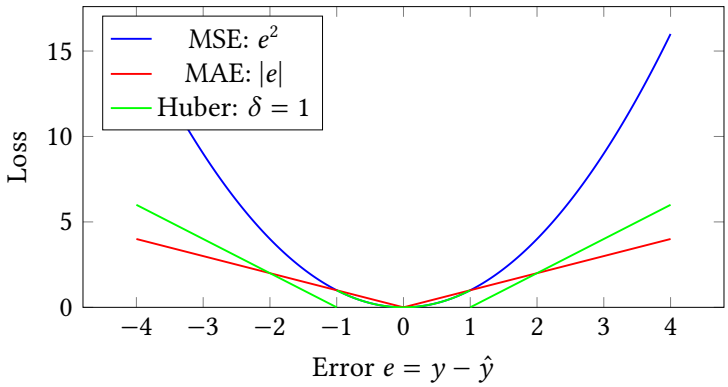


Figure 3.1 – MSE, MAE, and Huber loss. Huber loss is quadratic near zero (fast convergence) and linear for large errors (outlier robustness).

Example 3.1 (Huber Loss Gradient)**.** The gradient of Huber loss is continuous:

$$\frac{\partial \mathcal{L}_{\text{Huber}}}{\partial \hat{y}} = \begin{cases} \hat{y} - y & \text{if } |y - \hat{y}| \leq \delta \\ \delta \cdot \text{sign}(\hat{y} - y) & \text{otherwise} \end{cases} \tag{3.17}$$

The gradient is clamped to $\pm\delta$ for large errors, preventing gradient explosion from outliers while maintaining smooth quadratic behavior near the optimum.

3.4 Classification Loss Functions

Classification tasks predict discrete class labels. The loss function must encourage the model to assign high probability to the correct class.

3.4.1 From Likelihood to Cross-Entropy: A Complete Derivation Chain

To truly understand why cross-entropy is the standard classification loss, we must trace the complete derivation from first principles.

3.4.1.1 Step 1: The Generative Model

For a K-class classification problem, we model the output as a categorical distribution parameterized by the network:

$$p(y = k \mid \mathbf{x}; \theta) = \hat{y}_k, \quad k = 1, \dots, K \tag{3.18}$$

where $\hat{y}_k = \text{softmax}(\mathbf{z})_k$ and $\mathbf{z} = f(\mathbf{x}; \theta)$ are the logits.

3.4.1.2 Step 2: Maximum Likelihood

Given N i.i.d. samples, the likelihood is:

$$\mathcal{L}(\theta) = \prod_{i=1}^{N} p(y_i \mid \mathbf{x}_i; \theta) = \prod_{i=1}^{N} \prod_{k=1}^{K} \hat{y}_{ik}^{y_{ik}} \tag{3.19}$$

where $y_{ik} \in \{0, 1\}$ is the one-hot encoding.

3.4.1.3 Step 3: Negative Log-Likelihood = Cross-Entropy

Taking the negative log:

$$-\log \mathcal{L}(\theta) = -\sum_{i=1}^{N} \sum_{k=1}^{K} y_{ik} \log \hat{y}_{ik} \tag{3.20}$$

$$= -\sum_{i=1}^{N} \log \hat{y}_{i,c_i} \quad (\text{since } y_{ik} = \delta_{k,c_i}) \tag{3.21}$$

where c_i is the true class for sample i. This is the **categorical cross-entropy loss**.

3.4.1.4 Step 4: Connection to Information Theory

Recall from Chapter 1 that the cross-entropy between the true distribution p (one-hot: $p_{c_i} = 1$, $p_k = 0$ for $k \neq c_i$) and predicted distribution $q = \hat{\mathbf{y}}$ is:

$$H(p, q) = -\sum_{k=1}^{K} p_k \log q_k = -\log q_{c_i} \tag{3.22}$$

$$H(p) = 0 \quad (\text{true distribution is deterministic}) \tag{3.23}$$

Therefore:

$$D_{KL}(p \| q) = H(p, q) - H(p) = -\log \hat{y}_{c_i} = \mathcal{L}_{\text{CE}} \tag{3.24}$$

Minimizing cross-entropy ≡ minimizing KL divergence ≡ maximizing likelihood.

3.4.1.5 Step 5: The Beautiful Gradient

The softmax–cross-entropy gradient for a single sample:

$$\frac{\partial \mathcal{L}}{\partial z_j} = \frac{\partial}{\partial z_j}\left(-\log \frac{e^{z_c}}{\sum_k e^{z_k}}\right) \tag{3.25}$$

$$= -\frac{\partial}{\partial z_j}\left(z_c - \log \sum_k e^{z_k}\right) \tag{3.26}$$

$$= \begin{cases} \hat{y}_j - 1 = \hat{y}_j - y_j & \text{if } j = c \\ \hat{y}_j - 0 = \hat{y}_j - y_j & \text{if } j \neq c \end{cases} \tag{3.27}$$

$$= \boxed{\hat{y}_j - y_j} \tag{3.28}$$

The gradient is simply **prediction minus truth**. This elegantly simple result means:

- If $\hat{y}_c$ (predicted probability of correct class) is too low, the gradient pushes it up.
- If $\hat{y}_j$ (predicted probability of wrong class) is too high, the gradient pushes it down.
- The gradient automatically scales with confidence—a very wrong prediction produces a larger gradient.

3.4.2 Binary Cross-Entropy

For binary classification ($y \in \{0, 1\}$) with sigmoid output $\hat{y} = \sigma(z)$:

$$\mathcal{L}_{BCE} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)\right] \tag{3.29}$$

This is a special case of categorical cross-entropy with $K = 2$ (and numerically more stable implementation). The gradient w.r.t. the logit z is also elegantly simple:

$$\frac{\partial \mathcal{L}_{BCE}}{\partial z} = \hat{y} - y = \sigma(z) - y \tag{3.30}$$

3.4.3 Categorical Cross-Entropy

For multi-class classification (K classes):

$$\mathcal{L}_{CE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik} \log \hat{y}_{ik} \tag{3.31}$$

where $\hat{y}_{ik} = \text{softmax}(z_{ik})$.

Numerical stability: In practice, we compute the log-softmax directly:

$$\log \hat{y}_k = z_k - \log \sum_{j=1}^{K} e^{z_j} = z_k - \text{logsumexp}(\mathbf{z}) \tag{3.32}$$

The logsumexp function is implemented using the log-sum-exp trick:

$$\text{logsumexp}(\mathbf{z}) = m + \log \sum_j e^{z_j - m}, \quad m = \max(\mathbf{z}) \tag{3.33}$$

This prevents overflow/underflow in the exponential.

3.5 Advanced Loss Functions

3.5.1 Focal Loss

Focal Loss, introduced by Lin et al. (2017) [19] for dense object detection (RetinaNet), addresses the fundamental problem of **class imbalance** in single-stage detectors. In object detection, the vast majority of proposed anchor boxes are background (easy negatives), so the standard cross-entropy loss is dominated by easily classified examples that provide little learning signal.

3.5.1.1 Motivation: The Class Imbalance Problem

Consider a one-stage object detector with N anchors per image and typically only $O(1)$ positive (object) anchors. The cross-entropy loss:

$$\mathcal{L}_{CE} = -\log p_t \tag{3.34}$$

where $p_t = p$ if $y = 1$ (positive) and $p_t = 1 - p$ if $y = 0$ (negative). The problem:

- **Easy negatives** ($p_t \geq 0.9$): Even though there are millions, each contributes $-\log(0.9) \approx 0.105$ to the loss—small individually but enormous cumulatively.
- **Hard positives** ($p_t \leq 0.1$): Rare but critical, each contributing $-\log(0.1) \approx 2.3$—about 22× more than an easy negative.
- The combined effect: the gradient signal from hard examples is drowned out by the sheer number of easy examples.

3.5.1.2 The Modulating Factor

Focal Loss adds a modulating factor $(1 - p_t)^\gamma$ that down-weights the loss for well-classified examples:

$$\mathcal{L}_{\text{FL}} = -(1 - p_t)^\gamma \log p_t \tag{3.35}$$

3.5.1.3 Analysis of the Modulating Factor

For different values of p_t and γ:

Key observations:

- When $\gamma = 0$, focal loss reduces to standard cross-entropy.
- For $\gamma = 2$: a $p_t = 0.9$ example contributes 100× less than a $p_t = 0.1$ example.
- For $\gamma = 5$: easy examples contribute essentially zero loss.

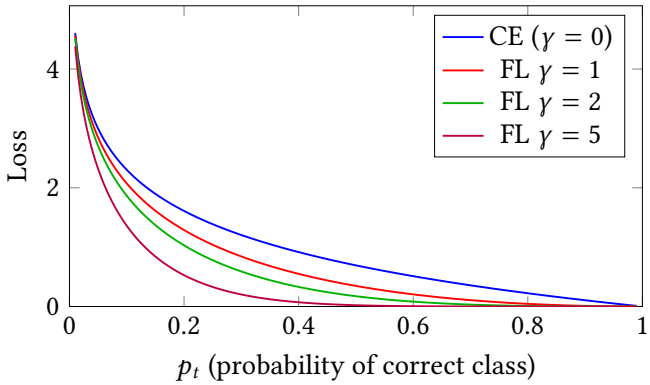


Figure 3.2 – Focal Loss with different γ values. As γ increases, the loss for easy examples ($p_t \to 1$) is increasingly suppressed.

Table 3.2 – Focal Loss Modulating Factor $(1 - p_t)^\gamma$

p_t	$\gamma = 0$	$\gamma = 1$	$\gamma = 2$	$\gamma = 5$
0.1 (hard)	1.000	0.900	0.810	0.590
0.3 (medium)	1.000	0.700	0.490	0.168
0.5 (ambiguous)	1.000	0.500	0.250	0.031
0.9 (easy)	1.000	0.100	0.010	~0
0.99 (very easy)	1.000	0.010	0.0001	~0

3.5.1.4 Alpha-Balanced Focal Loss

In practice, focal loss also incorporates a class-balancing weight $\alpha_t \in [0, 1]$:

$$\mathcal{L}_{\text{FL}} = -\alpha_t (1 - p_t)^\gamma \log p_t \tag{3.36}$$

The interplay between α and γ is subtle. In Lin et al.'s experiments, $\alpha = 0.25$ and $\gamma = 2$ achieved the best performance. The authors found that α matters less as γ increases—because the modulating factor already effectively down-weights the majority class.

3.5.1.5 Focal Loss Gradient

For binary classification with $y \in \{0, 1\}$ and $p_t = p$ if $y = 1$, $p_t = 1 - p$ if $y = 0$:

$$\frac{\partial \mathcal{L}_{\text{FL}}}{\partial p} = \alpha_t \left[\gamma p_t^\gamma \log p_t - (1 - p_t)^{\gamma - 1} (1 - \gamma) \right] \cdot \frac{\partial p_t}{\partial p} \tag{3.37}$$

The modulating factor $(1 - p_t)^\gamma$ naturally appears in the gradient, further emphasizing that the gradient magnitude for easy examples is suppressed.

3.5.2 Label Smoothing

Label smoothing [20], introduced by Szegedy et al. (2016) as part of the Inception-v2 architecture, replaces hard one-hot labels with soft labels. Despite its simplicity, it has become a standard technique in modern deep learning, especially for large language

models (GPT, BERT).

3.5.2.1 The Mechanism

Given a hard label $\mathbf{y} = [0, \dots, 1, \dots, 0]$ (one-hot at position c), label smoothing constructs:

$$y_k^{\text{LS}} = \begin{cases} 1 - \epsilon + \frac{\epsilon}{K} & \text{if } k = c \\ \frac{\epsilon}{K} & \text{if } k \neq c \end{cases} \tag{3.38}$$

where $\epsilon \in [0, 1)$ is the smoothing parameter (typically $\epsilon = 0.1$). For example, with $K = 10$ classes and $\epsilon = 0.1$:

$$\mathbf{y}^{\text{LS}} = [0.01, 0.01, \dots, 0.91, \dots, 0.01] \tag{3.39}$$

3.5.2.2 Why Does Label Smoothing Work? Three Perspectives

Perspective 1: Preventing Overconfidence.

Hard labels encourage the model to push logits to extreme values ($z_c \to +\infty$, $z_{k \neq c} \to -\infty$). This leads to:

- **Large logit magnitudes**, increasing the softmax output gap.
- **Poor calibration**: The model's confidence does not match its accuracy.
- **Reduced generalization**: Muller et al. (2019) showed that a neural network with cross-entropy on hard labels has a minimum where $\|\mathbf{w}_c\| \gg \|\mathbf{w}_{k \neq c}\|$, i.e., the weight vector for the correct class grows much larger than others.

Label smoothing prevents this by assigning non-zero probability to all classes, bounding the maximum achievable confidence at $1 - \epsilon + \epsilon/K$.

Perspective 2: Regularization via KL Divergence.

The cross-entropy with smoothed labels decomposes as:

$$\mathcal{L}_{\text{LS}} = -\sum_k y_k^{\text{LS}} \log \hat{y}_k \tag{3.40}$$

$$= -(1 - \epsilon) \underbrace{\log \hat{y}_c}_{\text{CE with hard label}} - \frac{\epsilon}{K} \underbrace{\sum_{k=1}^{K} \log \hat{y}_k}_{\text{negative entropy}} \tag{3.41}$$

$$= (1 - \epsilon)\mathcal{L}_{CE} - \epsilon H(\hat{\mathbf{y}}) \tag{3.42}$$

The second term $H(\hat{\mathbf{y}})$ is the **entropy of the predicted distribution**. Maximizing entropy encourages the predictions to be more uniform—a form of entropy regularization. This prevents the model from collapsing to very peaked distributions.

Perspective 3: Bayesian Interpretation.

Label smoothing can be viewed as incorporating a Dirichlet prior over the class probabilities. If $p \sim \text{Dir}(\alpha \mathbf{1})$ (symmetric Dirichlet), then:

$$\mathbb{E}[\log p_k] = \psi(\alpha) - \psi(K\alpha) \tag{3.43}$$

where ψ is the digamma function. For small α, this approximates $\log \frac{1}{K}$, which corresponds to uniform label smoothing.

3.5.2.3 Practical Impact

Label smoothing has been shown to:

1. Improve calibration (predicted probabilities better match true frequencies)
2. Improve generalization on ImageNet (+0.2–0.5% top-1 accuracy)
3. Enable better knowledge distillation (the softer logits serve as better teacher signals)
4. Reduce the vulnerability to adversarial examples

3.6 Contrastive and Self-Supervised Loss Functions

Recent advances in self-supervised learning (2020–2024) have introduced a new family of loss functions that learn representations by contrasting positive pairs against negative pairs, without requiring manual labels.

3.6.1 Contrastive Loss

The simplest form of contrastive loss was introduced for metric learning (e.g., face verification, Siamese networks):

$$\mathcal{L}_{\text{contrastive}} = \frac{1}{2}\left[y \cdot d^2 + (1-y) \cdot \max(0, m-d)^2\right] \tag{3.44}$$

where:

- $d = \|f(\mathbf{x}_1) - f(\mathbf{x}_2)\|_2$ is the distance between two embeddings
- $y \in \{0, 1\}$: $y = 1$ for a positive pair (same class), $y = 0$ for negative pair
- $m > 0$ is a margin hyperparameter

Intuition: Positive pairs are pushed close ($d \to 0$), while negative pairs are pushed apart until they are at least m apart (beyond which no further penalty applies).

3.6.2 Triplet Loss

Triplet loss extends contrastive learning by comparing **anchor**, **positive**, and **negative** samples simultaneously:

$$\mathcal{L}_{\text{Triplet}} = \max\left(0, \underbrace{\|f_a - f_p\|_2^2}_{\text{positive distance}} - \underbrace{\|f_a - f_n\|_2^2}_{\text{negative distance}} + \alpha\right) \tag{3.45}$$

Geometric interpretation: The loss is zero when the positive is at least α closer to the anchor than the negative. The margin α controls how much separation is enforced.

3.6.3 InfoNCE Loss

InfoNCE [21], the foundation of modern contrastive learning (SimCLR [22], MoCo, CLIP), frames representation learning as a **multiple-choice classification** problem:

Definition 3.1 (InfoNCE Loss)**.** Given a query $\mathbf{q}$, one positive key $\mathbf{k}_+$, and $N-1$ negative keys $\{\mathbf{k}_-\}$:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(\mathbf{q}^\top \mathbf{k}_+/\tau)}{\sum_{i=1}^{N} \exp(\mathbf{q}^\top \mathbf{k}_i/\tau)} \tag{3.46}$$

where $\tau > 0$ is a temperature hyperparameter.

3.6.3.1 Connection to Mutual Information

The remarkable theoretical result by Oord et al. (2018) [21] shows that InfoNCE provides a lower bound on mutual information:

$$I(\mathbf{x}; \mathbf{y}) \geq \log N - \mathcal{L}_{\text{InfoNCE}} \tag{3.47}$$

This means minimizing InfoNCE maximizes the mutual information between the query and positive key, encouraging the encoder to capture all shared information between augmented views of the same image.

3.6.3.2 The Role of Temperature τ

The temperature parameter τ controls the "sharpness" of the distribution:

- **Small** τ (e.g., 0.07 in CLIP): Makes the distribution sharp, increasing the penalty for negative samples that are too close. Encourages the model to be more "discriminative."
- **Large** τ (e.g., 0.5–1.0): Softer distribution, more uniform gradients across negatives.

For a similarity score $s = \mathbf{q}^\top \mathbf{k}/\tau$ and N negatives drawn from a uniform distribution over a hypersphere of dimension d:

$$\mathbb{E}[\mathcal{L}_{\text{InfoNCE}}] \approx -\log \sigma\left(\frac{s_+ - \mathbb{E}[s_-]}{\sqrt{\text{Var}(s)}}\right) + \text{const} \tag{3.48}$$

This shows that the loss effectively measures how separable the positive similarity is from the expected negative similarity.

3.6.4 Dice Loss

For segmentation tasks where class imbalance is severe (e.g., medical image segmentation), Dice loss [23] directly optimizes the Dice coefficient:

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2\sum_i \hat{y}_i \cdot y_i + \epsilon}{\sum_i \hat{y}_i + \sum_i y_i + \epsilon} \tag{3.49}$$

3.6.4.1 Derivation: Dice Coefficient and Jaccard Index

The **Sørensen–Dice coefficient** measures the overlap between predicted and ground truth segmentation masks:

$$\text{Dice} = \frac{2|A \cap B|}{|A| + |B|} \tag{3.50}$$

The **Jaccard index** (Intersection over Union, IoU) is related:

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} = \frac{\text{Dice}}{2 - \text{Dice}}, \quad \text{Dice} = \frac{2 \cdot \text{IoU}}{1 + \text{IoU}} \tag{3.51}$$

Why Dice Loss works for class imbalance: Unlike cross-entropy, which treats each pixel independently, Dice loss directly optimizes the overlap metric that is used for evaluation. This means:

- A single false positive pixel reduces the numerator by 1 and increases the denominator by 1—a double penalty.
- The loss is inherently normalized by the size of both predicted and ground truth regions, making it scale-invariant.
- Small objects contribute proportionally more to the loss than they would under cross-entropy.

Practical tip: Dice loss is often combined with cross-entropy as $\mathcal{L} = \mathcal{L}_{CE} + \lambda \mathcal{L}_{\text{Dice}}$ with $\lambda \in [0.5, 1.0]$. Cross-entropy provides stable pixel-wise gradients, while Dice loss ensures region-level overlap.

3.6.5 KL Divergence Loss

The KL divergence loss is used when the target is itself a probability distribution (as opposed to a one-hot label):

$$\mathcal{L}_{KL}(p \| q) = \sum_k p_k \log \frac{p_k}{q_k} \tag{3.52}$$

Applications:

- **Knowledge distillation**: The target p is the teacher's soft output distribution (after temperature scaling), and q is the student's prediction.
- **Variational inference** (VAEs): Matching the approximate posterior $q(\mathbf{z}|\mathbf{x})$ to the prior $p(\mathbf{z})$.
- **Domain adaptation**: Aligning the feature distributions of source and target domains.

Important: KL divergence is asymmetric. $D_{KL}(p \| q) \neq D_{KL}(q \| p)$. In knowledge distillation:

- **Forward KL** $D_{KL}(p_{\text{teacher}} \| q_{\text{student}})$: "Mean-seeking"—the student covers all modes of the teacher. Used for distillation.

- **Reverse KL** $D_{KL}(q_{\text{student}} \| p_{\text{teacher}})$: "Mode-seeking"—the student fits a few modes well but may miss others. Used in variational inference.

3.7 Choosing a Loss Function

Table 3.3 – Loss Function Selection Guide

Task	Loss	Notes
Binary Classification	BCE	Standard choice
Multi-class	Cross-Entropy	With label smoothing
Regression	MSE	Sensitive to outliers
Robust Regression	MAE/Huber	Outlier resistant
Imbalanced Classification	Focal Loss	Focus on hard examples
Segmentation	Dice + CE	Handle class imbalance

3.8 Chapter Summary

Loss functions are the bridge between the mathematical model and the learning objective:

1. **MSE = MLE under Gaussian noise**: Minimizing MSE is equivalent to fitting a model with additive Gaussian noise assumptions. The quadratic penalty makes it sensitive to outliers but efficient near the optimum.
2. **MAE = MLE under Laplace noise**: The linear penalty provides robustness to outliers but yields slower convergence. Huber loss combines the advantages of both.
3. **Cross-Entropy = MLE for categorical distributions**: The softmax–cross-entropy combination produces the elegant gradient $\hat{y}_k - y_k$, enabling efficient classification training.
4. **Focal Loss**: Addresses class imbalance by dynamically down-weighting easy examples via a modulating factor $(1 - p_t)^\gamma$. Essential for dense prediction tasks.
5. **Label Smoothing**: Prevents overconfidence by distributing probability mass across classes, with connections to Bayesian priors and entropy regularization.
6. **Contrastive Losses (InfoNCE)**: The foundation of self-supervised learning, bounding mutual information between views of the same data point.
7. **Dice Loss**: Directly optimizes the evaluation metric for segmentation, naturally handling class imbalance.

Table 3.4 – Complete Loss Function Decision Framework

Scenario	Loss	Why	Key Ref
Clean regression	MSE	MLE, Gaussian noise	
Regression + outliers	Huber ($\delta = 1$)	Robust + efficient	[18]
Binary classification	BCE	MLE, Bernoulli	
Multi-class	Cross-Entropy	MLE, categorical	
Imbalanced detection	Focal Loss ($\gamma = 2$)	Down-weights easy negs	[19]
Medical segmentation	Dice + CE	Overlap optimization	[23]
Self-supervised	InfoNCE ($\tau = 0.07$)	MI maximization	[22]
Knowledge distillation	KL Divergence	Distribution matching	
NLP / large models	CE + Label Smooth	Calibration	[20]

3.9 Further Reading

- Goodfellow et al., "Deep Learning" [6] — Chapter 5 (Machine Learning Basics) and Chapter 6 (Deep Feedforward Networks)
- Bishop, "Pattern Recognition and Machine Learning" [7] — Chapters 1–2 for probabilistic foundations
-
- [19] — Focal Loss for Dense Object Detection (ICCV 2017)
- [20] — Rethinking the Inception Architecture (Label Smoothing)
- [18] — Robust Estimation of a Location Parameter (Huber Loss)
- [23] — V-Net: Dice Loss for segmentation
- [22] — SimCLR: Simple Framework for Contrastive Learning
- [21] — Contrastive Predictive Coding (InfoNCE)
- [24] — Masked Autoencoders (MAE) for self-supervised vision

Chapter 4

Neural Network Architectures

4.1 Introduction

Neural network architectures define how neurons are organized and connected to process information. Different architectures are suited for different types of data and tasks. This chapter covers the three fundamental neural network architectures: Deep Neural Networks (DNN), Convolutional Neural Networks (CNN), and Recurrent Neural Networks (RNN).

4.2 Deep Neural Networks (DNN)

A Deep Neural Network consists of multiple layers of neurons, where information flows from input to output without loops.

4.2.1 Architecture

A fully-connected DNN performs:

$$\mathbf{h}^{(l)} = f^{(l)}(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) \tag{4.1}$$

where $l = 1, \dots, L$ indexes the layers.

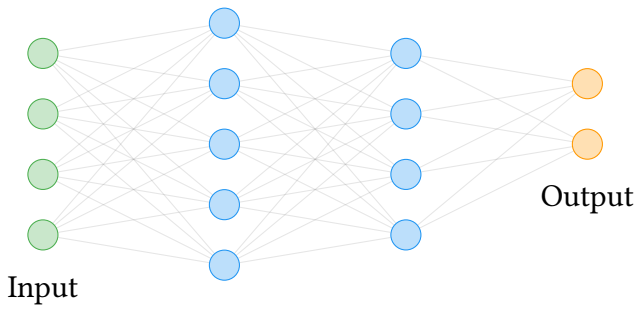


Figure 4.1 – Deep Neural Network architecture

Properties 4.1 (DNN Characteristics).

- **Parameter count:** $\sum_{l=1}^{L}(n^{(l-1)} \cdot n^{(l)} + n^{(l)})$
- **Universal approximation:** Can approximate any continuous function with sufficient width/depth
- **Suitable for:** Tabular data, feature-based classification

4.3 Convolutional Neural Networks (CNN)

CNNs exploit spatial structure in data through local connections and parameter sharing.

4.3.1 Convolutional Layer

The convolution operation:

$$y[i, j] = \sum_m \sum_n w[m, n] \cdot x[i + m, j + n] \tag{4.2}$$

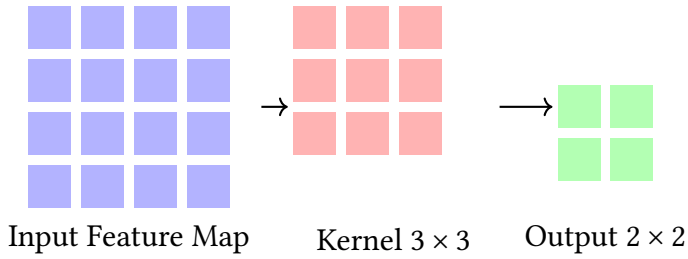


Figure 4.2 – Convolution operation

Key parameters:

- **Kernel size:** $K \times K$ (typically 3×3, 5×5, 7×7)
- **Stride:** s (step size, typically 1 or 2)
- **Padding:** p (zero-padding to control output size)

Output size:

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - K}{s} + 1 \right\rfloor \tag{4.3}$$

Properties 4.2 (CNN Advantages).

- **Local receptive field:** Each neuron sees only a local patch
- **Parameter sharing:** Same weights across spatial locations
- **Translation equivariance:** Shift in input → Shift in output
- **Suitable for:** Images, video, spatial data

4.3.2 Receptive Field Analysis

The **receptive field** (RF) of a neuron is the region in the input image that affects its output. Understanding RF growth is crucial for architecture design.

4.3.2.1 Single-Layer Receptive Field

A single convolutional layer with kernel K, stride s, and padding p produces an output whose RF is simply K. The output size is:

$$o_i = \left\lfloor \frac{i_{in} + 2p - K}{s} + 1 \right\rfloor \tag{4.4}$$

4.3.2.2 Multi-Layer Receptive Field

For a stack of L convolutional layers, the effective receptive field (ERF) of the output grows as:

$$\mathrm{RF} = 1 + \sum_{l=1}^{L} \left((K_l - 1) \cdot \prod_{l'=1}^{l-1} s_{l'} \right) \tag{4.5}$$

Example 4.1 (Two 3×3 Convolutions)**.** Two stacked 3×3 convolutions with stride 1:

$$\mathrm{RF} = 1 + (3-1) \times 1 + (3-1) \times 1 = 5 \times 5 \tag{4.6}$$

This matches a single 5×5 convolution's receptive field, but with fewer parameters ($2 \times 9C^2$ vs $25C^2$).

4.3.2.3 Effective Receptive Field

While the **theoretical** receptive field covers the entire region above, the **effective** receptive field (the region that actually influences the output) has a Gaussian-like distribution centered on the receptive field center (Luo et al., 2016). Pixels near the boundary contribute significantly less to the output than those at the center.

This has important design implications:

- Simply increasing the theoretical RF does not proportionally increase the effective RF
- Dilated/atrous convolutions can expand the effective RF without increasing parameters
- Deeper networks with small kernels are preferred over shallow networks with large kernels

4.3.3 1×1 Convolution: Cross-Channel Mixing

The 1×1 convolution, also called a **pointwise convolution**, operates on each pixel independently but mixes information across channels. Despite its simplicity, it is one of the most versatile operations in modern architectures.

Mathematical interpretation. A 1×1 convolution with C_{in} input channels and C_{out} output channels is equivalent to applying a fully-connected layer at each spatial position:

$$y_{c_{out}}[i, j] = \sum_{c_{in}=1}^{C_{in}} w_{c_{out}, c_{in}} \cdot x_{c_{in}}[i, j] + b_{c_{out}} \tag{4.7}$$

This can be written as a matrix multiplication: $\mathbf{Y}[i, j] = \mathbf{W} \cdot \mathbf{X}[i, j]$, where $\mathbf{W} \in \mathbb{R}^{C_{out} \times C_{in}}$.

Properties 4.3 (Key Uses of 1×1 Convolution)**.**

- **Dimensionality reduction (bottleneck):** Reduce $C_{in} \to C_{in}/4$ before an expensive 3×3 convolution, then project back. Used in Inception, ResNet bottleneck, and MobileNet. Parameters: $C_{in} \times C_{out}$ (vs $K^2 \times C_{in} \times C_{out}$ for $K \times K$ conv).
- **Channel mixing:** Enable cross-channel information flow without spatial aggregation. Essential when features from different kernels need to interact.

- **Nonlinearity injection:** After each 1×1 conv, an activation function (ReLU/GELU) adds representational power. In ResNeXt and MLP-Mixer, these "channel MLPs" are stacked.
- **Depth increase without spatial cost:** Increase network depth without affecting spatial resolution or adding significant parameters.

4.3.4 Transposed Convolution (Deconvolution)

For upsampling tasks (segmentation, generation), the **transposed convolution** (sometimes called deconvolution) reverses the spatial reduction of a standard convolution.

Given a standard convolution defined by stride s, padding p, and kernel K, the transposed convolution maps an input of size H_{in} to:

$$H_{out} = (H_{in} - 1) \times s - 2p + K \tag{4.8}$$

Note 4.1 (Transposed Conv ≠ Inverse Convolution)**.** A transposed convolution is **not** the mathematical inverse of a convolution. It is the transpose of the convolution operator (hence the name). Given $\mathbf{Y} = \mathbf{C}\mathbf{X}$ where $\mathbf{C}$ is the convolution as a matrix, the transposed convolution computes $\mathbf{C}^\top\mathbf{Y}$. This has the same parameters but does not recover $\mathbf{X}$ exactly.

4.4 Recurrent Neural Networks (RNN)

RNNs process sequential data by maintaining hidden state.

4.4.1 Architecture

For input sequence $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(T)}$:

$$\mathbf{h}^{(t)} = f(\mathbf{W}_{xh}\mathbf{x}^{(t)} + \mathbf{W}_{hh}\mathbf{h}^{(t-1)} + \mathbf{b}_h) \tag{4.9}$$
$$\mathbf{y}^{(t)} = g(\mathbf{W}_{hy}\mathbf{h}^{(t)} + \mathbf{b}_y) \tag{4.10}$$

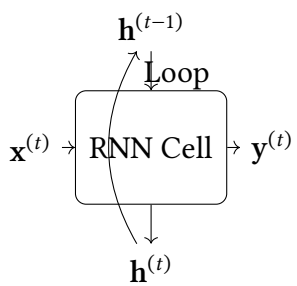


Figure 4.3 – RNN architecture

Properties 4.4 (RNN Characteristics)**.**

- **Sequential processing:** Can handle variable-length sequences
- **Shared parameters:** Same weights across time steps
- **Memory:** Hidden state captures past information

- **Suitable for:** Text, time series, audio, video

Note 4.2 (Vanishing Gradient Problem)**.** Standard RNNs suffer from vanishing gradients during backpropagation through time (BPTT), making it difficult to learn long-range dependencies. This led to the development of LSTM and GRU architectures.

4.4.2 Vanishing Gradient in RNNs: A Mathematical Proof

Understanding *why* vanilla RNNs fail on long sequences requires examining the gradient flow through time. We derive the exact condition for vanishing gradients.

4.4.2.1 Backpropagation Through Time (BPTT)

Consider the total loss over a sequence of length T:

$$\mathcal{L} = \sum_{t=1}^{T} \ell(\mathbf{y}^{(t)}, \hat{\mathbf{y}}^{(t)}) \tag{4.11}$$

The gradient of $\mathcal{L}$ with respect to $\mathbf{h}^{(k)}$ at an earlier time step k involves a chain of Jacobians:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(k)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(T)}} \prod_{t=k+1}^{T} \frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(t-1)}} \tag{4.12}$$

The Jacobian of the hidden state transition (with $\mathbf{h}^{(t)} = \tanh(\mathbf{W}_{hh}\mathbf{h}^{(t-1)} + \mathbf{W}_{xh}\mathbf{x}^{(t)} + \mathbf{b})$) is:

$$\frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(t-1)}} = \operatorname{diag}\left(1 - (\mathbf{h}^{(t)})^2\right) \cdot \mathbf{W}_{hh} \tag{4.13}$$

4.4.2.2 Spectral Analysis

The product of $T - k$ Jacobians can be analyzed through their spectral properties:

$$\left\| \prod_{t=k+1}^{T} \frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(t-1)}} \right\| \leq \prod_{t=k+1}^{T} \left\| \frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(t-1)}} \right\| \leq \prod_{t=k+1}^{T} \gamma_t \tag{4.14}$$

where $\gamma_t = \lambda_{\max}\left(\mathbf{W}_{hh}\right) \cdot \sup_i |1 - (h_i^{(t)})^2| \leq \lambda_{\max}\left(\mathbf{W}_{hh}\right)$ since $|1 - \tanh^2(x)| \leq 1$.

Theorem 4.1 (Vanishing/Exploding Gradient Condition)**.** The gradient through time satisfies:

$$\left\| \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(k)}} \right\| \approx \left\| \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(T)}} \right\| \cdot \gamma^{T-k} \tag{4.15}$$

where $\gamma = \lambda_{\max}(\mathbf{W}_{hh})$. Therefore:

- If $\gamma < 1$: gradient **vanishes** exponentially as $\gamma^{T-k} \to 0$
- If $\gamma > 1$: gradient **explodes** exponentially
- If $\gamma = 1$ (on the boundary): gradient is preserved, but this is a measure-zero condition

Properties 4.5 (Why This Matters in Practice)**.**

- Typical random initialization yields $\gamma \ll 1$ with high probability (Bengio et al., 1994)

- For a sequence of length $T = 100$ and $\gamma = 0.9$: gradient magnitude $\approx 0.9^{100} \approx 2.7 \times 10^{-5}$ — effectively zero
- The eigenvalues of $\mathbf{W}_{hh}$ are clipped to $[-1, 1]$ by the tanh derivative, pushing toward vanishing
- This is fundamentally a **dynamical systems** problem: the RNN is a discrete-time dynamical system whose fixed points are either attractive ($\gamma < 1$) or repulsive ($\gamma > 1$)

4.5 Long Short-Term Memory (LSTM)

LSTM addresses the vanishing gradient problem with gating mechanisms.

4.5.1 LSTM Gates

$$\mathbf{f}^{(t)} = \sigma(\mathbf{W}_f[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}] + \mathbf{b}_f) \quad \text{(forget gate)} \tag{4.16}$$
$$\mathbf{i}^{(t)} = \sigma(\mathbf{W}_i[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}] + \mathbf{b}_i) \quad \text{(input gate)} \tag{4.17}$$
$$\mathbf{o}^{(t)} = \sigma(\mathbf{W}_o[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}] + \mathbf{b}_o) \quad \text{(output gate)} \tag{4.18}$$
$$\tilde{\mathbf{c}}^{(t)} = \tanh(\mathbf{W}_c[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}] + \mathbf{b}_c) \quad \text{(cell candidate)} \tag{4.19}$$
$$\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \odot \mathbf{c}^{(t-1)} + \mathbf{i}^{(t)} \odot \tilde{\mathbf{c}}^{(t)} \quad \text{(cell state)} \tag{4.20}$$
$$\mathbf{h}^{(t)} = \mathbf{o}^{(t)} \odot \tanh(\mathbf{c}^{(t)}) \quad \text{(hidden state)} \tag{4.21}$$

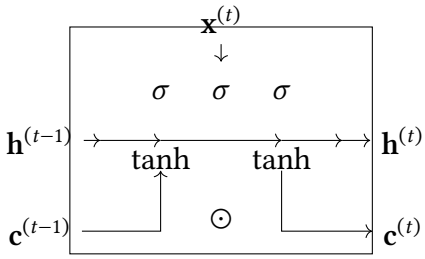


Figure 4.4 – LSTM cell structure

Properties 4.6 (LSTM Advantages).

- **Selective memory:** Gates control what to remember/forget
- **Long-range dependencies:** Cell state enables gradient flow
- **Widely used:** NLP, speech, time series

4.5.2 Why LSTM Solves the Vanishing Gradient

The key insight is that the cell state $\mathbf{c}^{(t)}$ provides an **additive** (rather than multiplicative) path for gradient flow.

4.5.2.1 Gradient Flow Through the Cell State

Unlike the vanilla RNN where $\mathbf{h}^{(t)} = f(\mathbf{W}_{hh}\mathbf{h}^{(t-1)} + \cdots)$, the LSTM cell state updates via addition:

$$\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \odot \mathbf{c}^{(t-1)} + \mathbf{i}^{(t)} \odot \tilde{\mathbf{c}}^{(t)} \tag{4.22}$$

The gradient of the loss with respect to the cell state at time k is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}^{(k)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{c}^{(T)}} \prod_{t=k+1}^{T} \left(\mathbf{f}^{(t)} \odot \frac{\partial \mathbf{c}^{(t)}}{\partial \mathbf{c}^{(t-1)}} + \cdots \right) \tag{4.23}$$

The critical Jacobian is:

$$\frac{\partial \mathbf{c}^{(t)}}{\partial \mathbf{c}^{(t-1)}} = \operatorname{diag}\left(\mathbf{f}^{(t)}\right) \tag{4.24}$$

When the forget gate $\mathbf{f}^{(t)} \approx \mathbf{1}$ (which the network can learn to set), the gradient is **passed through with near-identity mapping**:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}^{(k)}} \approx \frac{\partial \mathcal{L}}{\partial \mathbf{c}^{(T)}} \tag{4.25}$$

Properties 4.7 (Intuition: What Each Gate Does).

- **Forget gate $\mathbf{f}^{(t)}$:** "Should I erase this information?" Values near 0 discard old content; values near 1 preserve it. This is the key to solving vanishing gradients — the network learns when to keep information flowing.
- **Input gate $\mathbf{i}^{(t)}$:** "Is this new input worth writing?" Acts as a gate on the cell candidate. Prevents irrelevant inputs from overwriting the cell state.
- **Output gate $\mathbf{o}^{(t)}$:** "Should I expose this information now?" Controls when the cell state contributes to the hidden state. Allows the cell to store information without immediately using it.
- **Cell state $\mathbf{c}^{(t)}$:** The "conveyor belt." Information flows along this path largely unimpeded when forget gates are near 1. This is the **information highway** that bypasses the vanishing gradient problem.

4.5.2.2 Comparison: RNN vs LSTM Gradient

Table 4.1 – Gradient Flow: Vanilla RNN vs LSTM

Aspect	Vanilla RNN	LSTM
State update	$\mathbf{h}^{(t)} = \tanh(\mathbf{W}\mathbf{h}^{(t-1)} + \cdots)$	$\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \odot \mathbf{c}^{(t-1)} + \cdots$
Gradient path	Multiplicative	Additive + gated
Jacobian	$\operatorname{diag}(1 - \mathbf{h}^2)\mathbf{W}_{hh}$	$\operatorname{diag}(\mathbf{f}^{(t)})$
Learning-based	No control over gradient flow	Network learns $\mathbf{f}^{(t)}$
Long-range ($T = 100$)	$\gamma^{100} \approx 0$	$1.0^{100} = 1.0$

4.6 Gated Recurrent Unit (GRU)

GRU simplifies LSTM with fewer gates while maintaining performance.

4.6.1 GRU Equations

$$\mathbf{z}^{(t)} = \sigma(\mathbf{W}_z[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}]) \quad \text{(update gate)} \tag{4.26}$$

$$\mathbf{r}^{(t)} = \sigma(\mathbf{W}_r[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}]) \quad \text{(reset gate)} \tag{4.27}$$

$$\tilde{\mathbf{h}}^{(t)} = \tanh(\mathbf{W}[\mathbf{r}^{(t)} \odot \mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}]) \tag{4.28}$$

$$\mathbf{h}^{(t)} = (1 - \mathbf{z}^{(t)}) \odot \mathbf{h}^{(t-1)} + \mathbf{z}^{(t)} \odot \tilde{\mathbf{h}}^{(t)} \tag{4.29}$$

Comparison with LSTM:

- Fewer parameters (2 gates vs 3)
- No separate cell state
- Often performs comparably to LSTM

4.7 Choosing Architecture

Table 4.2 – Architecture Selection Guide

Architecture	Best For	Examples
DNN	Tabular data	Classification, regression
CNN	Images, spatial data	Object detection, segmentation
RNN/LSTM	Sequences	NLP, time series
Transformer	Long sequences	Translation, generation

4.8 Chapter Summary

1. DNNs are fully-connected networks for general-purpose learning
2. CNNs exploit spatial structure through local connections and parameter sharing
3. Receptive field grows with network depth; effective RF has Gaussian-like distribution
4. 1×1 convolutions enable efficient cross-channel mixing and dimensionality reduction
5. RNNs process sequential data with recurrent connections
6. Vanilla RNNs suffer from exponentially vanishing/exploding gradients (γ^{T-k})
7. LSTM solves this with an additive cell state pathway and learnable forget gates

8. GRU simplifies LSTM with fewer parameters while maintaining competitive performance
9. Architecture choice depends on data type, sequence length, and task requirements

4.9 Further Reading

- Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2), 157–166.
- Hochreiter, S. & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- Cho, K. et al. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *EMNLP*.
- Luo, W., Li, Y., Urtasun, R., & Zemel, R. (2016). Understanding the effective receptive field in deep convolutional neural networks. *NeurIPS*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *CVPR*.

Chapter 5

Core Operations in Neural Networks

5.1 Introduction

This chapter covers the fundamental building blocks of modern neural networks: convolution, pooling, and normalization operations. Understanding these operations is essential for designing and implementing effective neural network architectures.

5.2 Convolution Operation

Convolution is the cornerstone of convolutional neural networks (CNNs).

5.2.1 2D Convolution

For input $\mathbf{X} \in \mathbb{R}^{H \times W}$ and kernel $\mathbf{K} \in \mathbb{R}^{K \times K}$:

$$y[i, j] = \sum_{m=0}^{K-1} \sum_{n=0}^{K-1} k[m, n] \cdot x[i + m, j + n] \tag{5.1}$$

In practice, we often add bias and use multiple kernels for multiple output channels.

Properties 5.1 (Convolution Parameters).

- **Kernel size:** $K \times K$ (common: 3×3, 5×5, 7×7)
- **Stride (s):** Step size for sliding window
- **Padding (p):** Zero padding around input
- **Dilation (d):** Spacing between kernel elements

Output size with padding and stride:

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - d(K - 1) - 1}{s} + 1 \right\rfloor \tag{5.2}$$

5.2.2 Grouped and Depthwise Convolution

- **Grouped convolution:** Split input channels into groups, apply separate conv per group
- **Depthwise convolution:** One kernel per input channel (grouped with $G = C_{in}$)
- **Pointwise convolution:** 1×1 conv (used to mix channels)

MobileNet uses: Depthwise Separable Convolution = Depthwise + Pointwise

Properties 5.2 (Depthwise Separable Convolution).

- **Standard conv FLOPs:** $H \cdot W \cdot K^2 \cdot C_{in} \cdot C_{out}$
- **Separable conv FLOPs:** $H \cdot W \cdot K^2 \cdot C_{in} + H \cdot W \cdot C_{in} \cdot C_{out}$
- **Speedup:** $\frac{1}{K^2} + \frac{1}{C_{out}}$ approximately

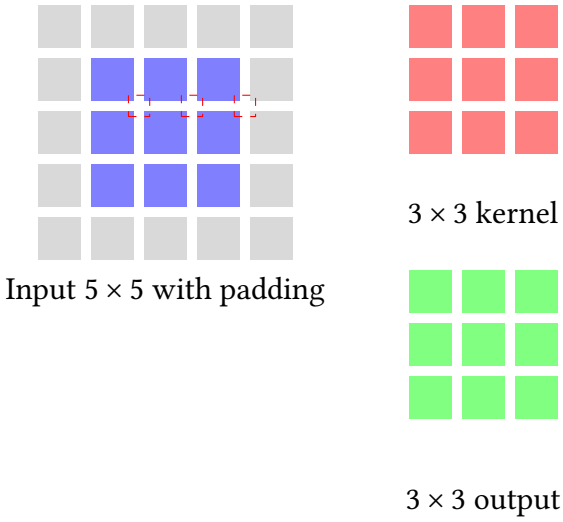


Figure 5.1 – Convolution with padding

5.2.3 Dilated (Atrous) Convolution

Dilated convolution inserts zeros between kernel elements, expanding the receptive field without increasing the parameter count or computation:

$$y[i,j] = \sum_{m=0}^{K-1} \sum_{n=0}^{K-1} k[m,n] \cdot x[i + m \cdot d, j + n \cdot d] \tag{5.3}$$

where d is the dilation rate.

Properties 5.3 (Dilated Convolution Properties).

- **Receptive field with dilation** d: Effective kernel size $= d(K-1)+1$
- **FLOPs:** Same as standard convolution (only the sampling pattern changes)
- **Parameters:** Same as standard convolution
- **Key application:** Semantic segmentation (DeepLab series), where large receptive fields are needed without pooling or striding

Example 5.1 (Stacked Dilated Convolutions). Three 3×3 convolutions with dilation rates $d = 1, 2, 4$ produce a total receptive field of:

$$\text{RF} = 1 + (3-1) \times 1 + (3-1) \times 2 + (3-1) \times 4 = 15 \tag{5.4}$$

This matches a 15×15 standard convolution but with only $3 \times 9C^2$ parameters instead of $225C^2$. This is the core idea behind **DeepLab** (Chen et al., 2017).

5.3 Pooling Operations

Pooling reduces spatial dimensions and provides translation invariance.

5.3.1 Max Pooling

$$y[i,j] = \max_{m,n \in \mathcal{N}(i,j)} x[m,n] \tag{5.5}$$

where $\mathcal{N}(i, j)$ is the neighborhood.

- **Retains strongest activation**
- **Provides local translation invariance**
- **Common:** 2×2 with stride 2

5.3.2 Average Pooling

$$y[i, j] = \frac{1}{K \cdot K} \sum_{m,n \in \mathcal{N}(i,j)} x[m, n] \tag{5.6}$$

- **Averages over neighborhood**
- **Less aggressive than max pooling**
- **Used in global pooling**

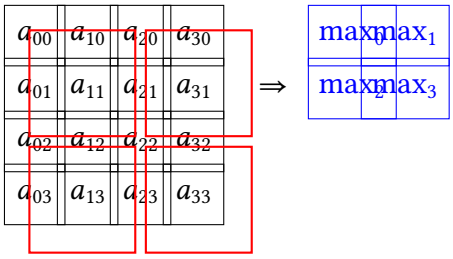


Figure 5.2 – Max pooling 2×2, stride 2

5.3.3 Global Pooling

Applied over entire feature map to produce single value per channel:

- **Global Average Pooling (GAP):** $\frac{1}{HW} \sum_{i,j} x_{ij}$
- **Global Max Pooling:** $\max_{i,j} x_{ij}$
- **Replaces FC layers** in modern architectures

5.4 Normalization Techniques

Normalization stabilizes training by controlling layer input distributions.

5.4.1 Batch Normalization

Normalizes activations across batch dimension:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta \tag{5.7}$$

where:

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i \tag{5.8}$$

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2 \tag{5.9}$$

Properties 5.4 (BatchNorm Properties).

- **Reduces internal covariate shift**
- **Allows higher learning rates**
- **Provides slight regularization**
- **Dependent on batch size and can be problematic in small batches**

Note 5.1 (BatchNorm vs. Inference)**.** During training, BatchNorm uses batch statistics. During inference, use running statistics:

$$\hat{x} = \frac{x - \tilde{\mu}}{\sqrt{\tilde{\sigma}^2 + \epsilon}} \tag{5.10}$$

where $\tilde{\mu}$ and $\tilde{\sigma}^2$ are running averages.

5.4.2 Layer Normalization

Normalizes across features within a single sample:

$$\mu = \frac{1}{D}\sum_{i=1}^{D} x_i, \quad \sigma = \frac{1}{D}\sum_{i=1}^{D}(x_i - \mu)^2 \tag{5.11}$$

- **Independent of batch size**
- **Used in RNNs and Transformers**
- **Same for training and inference**

Table 5.1 – Normalization Comparison

Method	Normalize over	Batch dep.	Use case
BatchNorm	(H, W, C)	Yes	CNNs
LayerNorm	(H, W, N)	No	Transformers
InstanceNorm	(H, W)	No	Style transfer
GroupNorm	(H, W, G)	No	Small batch CNNs

5.4.3 Instance Normalization

Normalizes each sample and channel independently:

$$y_{nchw} = \frac{x_{nchw} - \mu_{nc}}{\sqrt{\sigma_{nc}^2 + \epsilon}} \tag{5.12}$$

Used in style transfer where instance-specific statistics matter.

5.4.4 Group Normalization

Group Normalization (GN) divides channels into G groups and normalizes within each group independently (Wu & He, 2018). This provides a unified framework that subsumes LayerNorm and InstanceNorm as special cases.

For a tensor of shape (N, C, H, W) with C/G channels per group:

$$\mu_g = \frac{1}{(C/G) \cdot H \cdot W} \sum_{c \in \mathcal{G}_g} \sum_{h,w} x_{nchw} \tag{5.13}$$

$$\sigma_g^2 = \frac{1}{(C/G) \cdot H \cdot W} \sum_{c \in \mathcal{G}_g} \sum_{h,w} (x_{nchw} - \mu_g)^2 \tag{5.14}$$

$$\hat{x}_{nchw} = \frac{x_{nchw} - \mu_g}{\sqrt{\sigma_g^2 + \epsilon}} \tag{5.15}$$

Properties 5.5 (GN as a Unified Framework).

- $G = 1$: GroupNorm becomes **LayerNorm** (normalize across all channels)
- $G = C$: GroupNorm becomes **InstanceNorm** (normalize per-channel)
- $G = 32$ is the default in most implementations (a good tradeoff)
- **Independent of batch size**: works with batch size 1 (e.g., in medical imaging)
- Computationally equivalent to BatchNorm; only the grouping changes

Table 5.2 – Normalization Methods: Unified View

Method	Compute stats over	Batch dep.	Primary use
BatchNorm	(N, H, W) per C	Yes	CNNs (large batch)
LayerNorm	(H, W, C) per N	No	Transformers, RNNs
InstanceNorm	(H, W) per N, C	No	Style transfer
GroupNorm	$(H, W, C/G)$ per N	No	Detection, small batch

5.5 Regularization Operations

Beyond normalization, several operations prevent overfitting by introducing stochasticity or constraints during training.

5.5.1 Dropout

Dropout (Srivastava et al., 2014) randomly sets activations to zero during training with probability p:

$$\tilde{x}_i = \frac{m_i \cdot x_i}{1 - p}, \quad m_i \sim \text{Bernoulli}(1 - p) \tag{5.16}$$

The scaling factor $1/(1-p)$ (inverted dropout) ensures that $\mathbb{E}[\tilde{x}_i] = x_i$, so no adjustment is needed at inference.

5.5.1.1 Why Dropout Works: Bayesian Interpretation

Dropout can be viewed as approximate Bayesian inference (Gal & Ghahramani, 2016). Each dropout mask defines a different sub-network, and training with dropout is equivalent to:

$$p(\mathbf{y} \mid \mathbf{x}, \mathcal{D}) \approx \frac{1}{T} \sum_{t=1}^{T} p(\mathbf{y} \mid \mathbf{x}, \mathbf{W} \odot \mathbf{m}^{(t)}) \tag{5.17}$$

This is **model averaging** over an exponential number of thinned networks (2^N for N units), which reduces variance and improves generalization.

5.5.1.2 Variants of Dropout

Table 5.3 – Dropout Variants

Variant	Operation	Use Case
Standard Dropout	Zero activations with prob. p	FC layers
DropConnect	Zero *weights* with prob. p	FC layers
Spatial Dropout	Drop entire feature maps	CNNs
DropPath	Skip entire path/blocks	Deep networks
DropBlock	Drop contiguous regions	CNNs (regularization)

DropPath (Stochastic Depth, Huang et al., 2016) randomly skips entire residual blocks during training. For a block with input $\mathbf{x}$ and output $\mathcal{F}(\mathbf{x})$:

$$\tilde{\mathbf{y}} = \begin{cases} \mathbf{x} & \text{with probability } p_L \\ \mathcal{F}(\mathbf{x}) + \mathbf{x} & \text{with probability } 1 - p_L \end{cases} \tag{5.18}$$

The drop rate p_L typically increases linearly with depth, creating shorter effective networks during training that act as an ensemble.

5.6 Combining Operations

Modern networks combine these operations in standardized blocks.

5.6.1 Basic Conv Block

A typical convolutional block:

1. Convolution
2. Batch Normalization
3. Activation (ReLU/GELU)

Common patterns:

- Conv-BN-ReLU (VGG style)
- BN-ReLU-Conv (ResNet style, pre-activation)

Listing 5.1 – Conv Block in PyTorch

```python
import torch.nn as nn

class ConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch, kernel_size=3, stride=1):
        super().__init__()
        padding = kernel_size // 2
        self.block = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size, stride, padding, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True)
        )

    def forward(self, x):
        return self.block(x)

# Pre-activation block (ResNet style)
class PreActBlock(nn.Module):
    def __init__(self, in_ch, out_ch, stride=1):
        super().__init__()
        self.block = nn.Sequential(
            nn.BatchNorm2d(in_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(in_ch, out_ch, 3, stride, 1, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, 3, 1, 1, bias=False)
        )
        self.shortcut = nn.Identity() if stride==1 and in_ch==out_ch else \
                        nn.Conv2d(in_ch, out_ch, 1, stride, bias=False)

    def forward(self, x):
        return self.block(x) + self.shortcut(x)
```

5.7 Chapter Summary

1. Convolution extracts local features with shared weights
2. Depthwise separable convolution reduces computation for mobile deployment
3. Dilated convolution expands receptive field without extra parameters
4. Pooling provides spatial reduction and invariance
5. Normalization stabilizes training dynamics (BN for CNNs, LN for Transformers, GN for small batches)
6. Dropout and its variants provide regularization through stochastic depth
7. Modern architectures combine these in standardized blocks
8. Choice of operations depends on task and data characteristics

5.8 Further Reading

- Ioffe, S. & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *ICML*.
- Ba, J., Kiros, J., & Hinton, G. (2016). Layer normalization. *arXiv:1607.06450*.
- Wu, Y. & He, K. (2018). Group normalization. *ECCV*.
- Srivastava, N. et al. (2014). Dropout: A simple way to prevent neural networks from overfitting. *JMLR*, 15, 1929–1958.
- Gal, Y. & Ghahramani, Z. (2016). Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. *ICML*.
- Huang, G. et al. (2016). Deep networks with stochastic depth. *ECCV*.
- Chen, L.-C. et al. (2017). Rethinking atrous convolution for semantic image segmentation. *arXiv:1706.05587*.
- Howard, A. et al. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv:1704.04861*.

Chapter 6

Skip Connections and Attention Mechanisms

6.1 Introduction

Deep networks face two fundamental challenges: (1) gradients vanish or explode as they propagate through many layers, and (2) models need mechanisms to selectively focus on the most relevant parts of their inputs. Skip connections address the first challenge by providing gradient highways, while attention mechanisms address the second by enabling dynamic, input-dependent weighting. Together, these techniques form the backbone of modern architectures from ResNet to the Transformer.

6.2 Skip Connections

Skip connections (shortcuts) bypass layers to enable gradient flow in deep networks.

6.2.1 Residual Connections (ResNet)

The residual learning framework [13]:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x} \tag{6.1}$$

where $\mathcal{F}$ is the learned residual mapping.

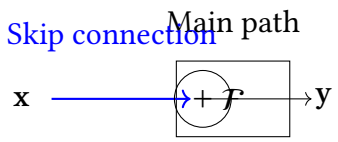


Figure 6.1 – Residual connection

If dimensions match: $\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}$

If dimensions differ: $\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x}$

6.2.2 Why Residual Connections Work: Theoretical Analysis

6.2.2.1 Gradient Flow Perspective

Consider a network with L residual blocks. The gradient of the loss $\mathcal{L}$ with respect to an intermediate activation $\mathbf{x}_l$ is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L}\left(1 + \frac{\partial}{\partial \mathbf{x}_l}\sum_{i=l}^{L-1}\mathcal{F}(\mathbf{x}_i, W_i)\right) \tag{6.2}$$

The key insight: the gradient contains an **identity term** (the "1") that is added to the residual gradient. Even if the residual gradient $\frac{\partial \mathcal{F}}{\partial \mathbf{x}}$ vanishes, the identity ensures that

information can propagate directly:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_l} \geq \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L} \cdot 1 \tag{6.3}$$

This is in stark contrast to a plain network, where the gradient involves a product of $L - l$ Jacobians:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_l}^{\text{plain}} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L} \prod_{i=l}^{L-1} \frac{\partial \mathbf{x}_{i+1}}{\partial \mathbf{x}_i} \tag{6.4}$$

Each factor less than 1 in magnitude causes exponential decay.

6.2.2.2 Identity Mapping Hypothesis

If the optimal function for a layer is close to identity (which is often the case for deep networks), learning the residual $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$ is easier than learning $\mathcal{H}(\mathbf{x})$ from scratch. The network only needs to push the residual to zero, rather than learning a near-identity transformation through nonlinear layers.

Properties 6.1 (ResNet Benefits).

- Enables training of very deep networks (100+ layers)
- Alleviates vanishing/exploding gradients via identity shortcuts
- Reduces degradation problem (deeper ≠ higher training error)
- Easy to implement and computationally minimal
- Theoretical guarantees: any function learnable by a plain network is also learnable by its residual counterpart

6.2.3 Pre-activation ResNet

He et al. (2016) proposed moving BatchNorm and ReLU before the convolutional layers (pre-activation) instead of after (post-activation):

$$\text{Post-activation:} \quad \mathbf{y} = \text{ReLU}(\text{BN}(\mathcal{F}(\mathbf{x}) + \mathbf{x})) \tag{6.5}$$

$$\text{Pre-activation:} \quad \mathbf{y} = \mathcal{F}(\text{ReLU}(\text{BN}(\mathbf{x}))) + \mathbf{x} \tag{6.6}$$

Pre-activation provides a cleaner signal propagation path: the identity shortcut is entirely "clean" (no nonlinearity or BN in the path), which improves gradient flow.

6.2.4 Bottleneck Architecture

Used in deeper ResNets to reduce complexity:

- 1×1 conv to reduce dimensions
- 3×3 conv (bottleneck)
- 1×1 conv to restore dimensions

For input C, bottleneck outputs C:

1. Conv $1 \times 1, C \to C//4$
2. Conv $3 \times 3, C//4 \to C//4$
3. Conv $1 \times 1, C//4 \to C$

6.2.5 ResNeXt: Grouped Convolutions in Residual Blocks

ResNeXt [25] introduces grouped convolutions into the residual block. Instead of a single wide convolution, it uses G parallel "cardinal" paths with smaller width d:

$$\mathcal{F}(\mathbf{x}) = \sum_{i=1}^{G} \mathcal{T}_i(\mathbf{x}) \tag{6.7}$$

where each $\mathcal{T}_i$ is a transformation with C/G channels.

Properties 6.2 (ResNeXt vs ResNet).

- **Same FLOPs and parameters** as a wider ResNet, but better accuracy
- **Increasing G improves accuracy** even when total width is fixed
- The key is **cardinality** (number of groups) rather than just depth or width
- $G = 32$ with width $d = 4$ is a common configuration

6.2.6 Stochastic Depth (DropPath)

Stochastic Depth [26] randomly drops entire residual blocks during training. For a linearly increasing drop rate $p_l = 1 - \frac{l}{L}(1 - p_L)$:

$$\mathbf{y}_l = \begin{cases} \mathbf{x}_l & \text{with probability } p_l \text{ (skip block)} \\ \mathcal{F}_l(\mathbf{x}_l) + \mathbf{x}_l & \text{with probability } 1 - p_l \text{ (keep block)} \end{cases} \tag{6.8}$$

This trains an ensemble of networks of varying depth, providing regularization without extra parameters. It is now a standard component in modern architectures like EfficientNet and Vision Transformers.

6.2.7 Dense Connections (DenseNet)

Each layer receives feature maps from all preceding layers [27]:

$$\mathbf{x}_l = H_l([\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{l-1}]) \tag{6.9}$$

Properties 6.3 (DenseNet Properties).

- **Feature reuse:** Each layer accesses all preceding features
- **Strong gradient flow:** Direct connections from any layer to the loss
- **Parameter efficiency:** Fewer parameters than ResNet at comparable accuracy
- **Memory cost:** Higher due to concatenation (mitigated by dense connectivity of small growth rate k)
- **Total parameters in a DenseNet block:** $\mathcal{O}(k \times k \times (l-1) \times K^2)$ where k is the growth rate

6.3 Attention Mechanisms

Attention allows models to focus on relevant parts of the input.

6.3.1 Self-Attention (Scaled Dot-Product)

The scaled dot-product attention function [28]:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{6.10}$$

where:

- $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$ (Query), $\mathbf{Q} \in \mathbb{R}^{n \times d_k}$
- $\mathbf{K} = \mathbf{X}\mathbf{W}_K$ (Key), $\mathbf{K} \in \mathbb{R}^{n \times d_k}$
- $\mathbf{V} = \mathbf{X}\mathbf{W}_V$ (Value), $\mathbf{V} \in \mathbb{R}^{n \times d_v}$
- $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{model}}}$ is the input sequence

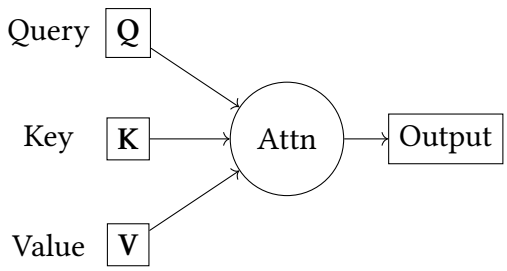


Figure 6.2 – Self-attention mechanism

6.3.1.1 Why Scale by $\sqrt{d_k}$?

Without scaling, when d_k is large, the dot products $\mathbf{q}_i \cdot \mathbf{k}_j = \sum_{m=1}^{d_k} q_{im} k_{jm}$ grow in magnitude (variance $= d_k$ for unit-variance random vectors). This pushes the softmax into regions with extremely small gradients:

$$\text{If } d_k = 64: \quad \text{softmax}\begin{pmatrix}8.0\\8.1\end{pmatrix} \approx \begin{pmatrix}0.475\\0.525\end{pmatrix}, \quad \text{softmax}\begin{pmatrix}800\\810\end{pmatrix} \approx \begin{pmatrix}0.000\\1.000\end{pmatrix} \tag{6.11}$$

Scaling by $\sqrt{d_k}$ ensures the dot products have unit variance, keeping the softmax in its "responsive" regime.

6.3.1.2 Computational Complexity

For a sequence of length n with model dimension d:

$$\text{Q, K, V projections:} \quad \mathcal{O}(n \cdot d^2) \tag{6.12}$$

$$\text{Attention scores } \mathbf{Q}\mathbf{K}^\top: \quad \mathcal{O}(n^2 \cdot d) \tag{6.13}$$

$$\text{Output softmax}(\cdot)\mathbf{V}: \quad \mathcal{O}(n^2 \cdot d) \tag{6.14}$$

$$\textbf{Total:} \quad \mathcal{O}(n^2 \cdot d + n \cdot d^2) \tag{6.15}$$

The dominant term is $\mathcal{O}(n^2 \cdot d)$ for $n > d$, making self-attention quadratic in sequence length — a key bottleneck for long sequences.

Properties 6.4 (Self-Attention Properties).

- **Global receptive field:** Every token attends to every other token in one layer
- **Computationally $O(n^2 d)$:** Quadratic in sequence length
- **Interpretable:** Attention weights show pairwise relevance
- **Permutation equivariant:** Self-attention is order-invariant by default (position encoding adds order information)

6.3.2 Multi-Head Attention

Instead of a single attention function, Multi-Head Attention (MHA) runs h attention functions in parallel with different learned projections [28]:

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V) \tag{6.16}$$

$$\text{MHA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)\mathbf{W}^O \tag{6.17}$$

where each $\mathbf{W}_i^Q, \mathbf{W}_i^K \in \mathbb{R}^{d \times d_k}$, $\mathbf{W}_i^V \in \mathbb{R}^{d \times d_v}$, $\mathbf{W}^O \in \mathbb{R}^{h d_v \times d}$. Typically $d_k = d_v = d/h$.

Properties 6.5 (Why Multiple Heads?).

- **Diverse representations:** Each head can learn different attention patterns (syntactic, semantic, positional)
- **Same total cost:** $h \times \mathcal{O}(n^2 \cdot d_k) = \mathcal{O}(n^2 \cdot d)$ — same as single-head
- **Linear projection:** The output projection $\mathbf{W}^O$ mixes information from all heads
- **Empirical:** $h = 8$ or $h = 16$ heads is standard; diminishing returns beyond ~32

6.3.3 Cross-Attention

Cross-attention allows the model to attend to a **different** sequence from the one being generated. The queries come from one sequence, while keys and values come from another:

$$\text{Cross-Attention}(\mathbf{Q}_{\text{target}}, \mathbf{K}_{\text{source}}, \mathbf{V}_{\text{source}}) \tag{6.18}$$

This is the core mechanism in Encoder-Decoder architectures (e.g., machine translation):

- **Encoder:** Processes source sequence via self-attention
- **Decoder:** Uses cross-attention to query the encoder's representations
- The decoder "looks up" relevant source tokens for each generated target token

6.3.4 Positional Encoding

Since self-attention is permutation-equivariant, we must inject positional information explicitly.

6.3.4.1 Sinusoidal Positional Encoding

The original Transformer uses fixed sinusoidal encodings [28]:

$$\mathrm{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \tag{6.19}$$

$$\mathrm{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \tag{6.20}$$

Properties 6.6 (Properties of Sinusoidal PE).

- Each dimension corresponds to a sinusoid of different wavelength (from 2π to $10000 \cdot 2\pi$)
- Allows the model to learn relative positions: PE_{pos+k} can be expressed as a linear function of PE_{pos}
- No learned parameters — works across sequence lengths
- Limited in extrapolating to longer sequences than seen during training

6.3.4.2 Modern Alternatives

Table 6.1 – Positional Encoding Methods

Method	Mechanism	Key Property
Learned PE	Trainable embeddings	Flexible, limited to max length
RoPE	Rotate Q, K by angle	Relative position via dot product
ALiBi	Add linear bias to scores	Extrapolates to unseen lengths
T5 Bias	Add bias to attention scores	Simple, effective

Rotary Position Embedding (RoPE) [29] encodes position by rotating the query and key vectors:

$$\langle \mathbf{q}_m, \mathbf{k}_n \rangle_{\mathrm{RoPE}} = \langle R_m \mathbf{q}, R_n \mathbf{k} \rangle = f(\mathbf{q}, \mathbf{k}, n - m) \tag{6.21}$$

This naturally encodes **relative position** $n - m$ into the attention score. RoPE is the default in LLaMA and many modern LLMs.

ALiBi [30] adds a fixed linear bias to attention scores based on relative distance:

$$\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{m} \cdot [-n + m]\right)\mathbf{V} \tag{6.22}$$

where $\mathbf{m}$ is a non-learned slope vector. ALiBi's key advantage is **length extrapolation**: models trained on short sequences can process much longer ones at inference.

6.3.5 Squeeze-and-Excitation (SE) Attention

Channel attention mechanism [31]:

1. **Squeeze:** Global average pooling $\mathbf{z} = \frac{1}{H \times W} \sum_{i,j} \mathbf{X}_{:,i,j}$

2. **Excitation:** FC-ReLU-FC-Sigmoid $\mathbf{s} = \sigma(W_2\delta(W_1\mathbf{z}))$
3. **Scale:** $\tilde{\mathbf{X}} = \mathbf{s} \cdot \mathbf{X}$

Properties 6.7 (SE Block).

- **Modeling channel dependencies**
- **Lightweight:** Reduction ratio $r = 16$ means only $2C^2/r$ extra parameters
- **Plug-and-play:** Can be added to any existing architecture
- **Achieves >1% top-1 improvement** on ImageNet with negligible overhead

6.3.6 CBAM: Convolutional Block Attention Module

CBAM [32] combines **channel attention** and **spatial attention** in sequence:

Channel Attention Module (CAM):

$$\mathbf{M}_c(\mathbf{X}) = \sigma\big(\mathrm{MLP}(\mathrm{AvgPool}(\mathbf{X})) + \mathrm{MLP}(\mathrm{MaxPool}(\mathbf{X}))\big) \tag{6.23}$$

Spatial Attention Module (SAM):

$$\mathbf{M}_s(\mathbf{X}') = \sigma\big(f^{7\times7}([\mathrm{AvgPool}(\mathbf{X}'); \mathrm{MaxPool}(\mathbf{X}')])\big) \tag{6.24}$$

The full process: $\mathbf{X}'' = \mathbf{M}_s(\mathbf{M}_c(\mathbf{X}) \odot \mathbf{X}) \odot \mathbf{X}$.

By attending to both *what* (channels) and *where* (spatial locations), CBAM captures richer attention patterns than channel-only SE.

6.3.7 Window Attention (Swin Transformer)

To address the $O(n^2)$ complexity of global self-attention, the Swin Transformer [33] restricts attention to **local windows**:

$$\mathrm{WindowAttention}(\mathbf{x}) = \mathrm{softmax}\left(\frac{\mathbf{Q}_w\mathbf{K}_w^\top}{\sqrt{d}} + \mathbf{B}\right)\mathbf{V}_w \tag{6.25}$$

where $\mathbf{B}$ is a **relative position bias** (learned), and the subscript w denotes computation within a window of size $M \times M$.

Properties 6.8 (Window Attention vs Global Attention).

- **Complexity:** $O(n \cdot M^2 \cdot d)$ vs $O(n^2 \cdot d)$ — linear in image size when M is fixed
- **Cross-window connection:** Shifted window partitioning in alternating layers enables information flow between windows
- **Local first:** Low-level features are inherently local; window attention is a natural inductive bias for vision

6.3.8 Efficient Attention Variants

Listing 6.1 – Multi-Head Self-Attention in PyTorch

```
import torch
import torch.nn as nn
import math
```

Table 6.2 – Attention Complexity Comparison

Method	Complexity	Global?	Reference
Standard MHA	$O(n^2 d)$	Yes	Vaswani 2017
Linformer	$O(nkd)$	Approx	Wang et al. 2020
Linear Attn	$O(nd^2)$	Approx	Katharopoulos 2020
Performer	$O(nd)$	Approx	Choromanski 2021
Swin (window)	$O(nM^2 d)$	Shifted	Liu et al. 2021
Flash Attention	$O(n^2 d)$	Yes	Dao et al. 2022

```python

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads, dropout=0.1):
        super().__init__()
        self.d_model = d_model
        self.n_heads = n_heads
        self.d_k = d_model // n_heads

        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        B, N, D = x.shape

        # Project and reshape: (B, N, D) -> (B, H, N, d_k)
        Q = self.W_q(x).view(B, N, self.n_heads, self.d_k).transpose(1, 2)
        K = self.W_k(x).view(B, N, self.n_heads, self.d_k).transpose(1, 2)
        V = self.W_v(x).view(B, N, self.n_heads, self.d_k).transpose(1, 2)

        # Scaled dot-product attention
        scores = Q @ K.transpose(-2, -1) / math.sqrt(self.d_k)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, float('-inf'))
        attn = torch.softmax(scores, dim=-1)
        attn = self.dropout(attn)

        # Concatenate heads
        out = (attn @ V).transpose(1, 2).contiguous().view(B, N, D)
        return self.W_o(out)
```

6.4 Chapter Summary

1. Skip connections enable deep networks by providing gradient highways
2. Residual connections add identity shortcuts; the gradient always has a path of magnitude ≥ 1
3. Pre-activation ResNet keeps the shortcut path "clean" for better gradient flow
4. ResNeXt shows that cardinality (grouped paths) is more effective than width

5. Stochastic Depth provides regularization by training an ensemble of sub-networks
6. Self-attention computes pairwise interactions in $O(n^2 d)$ time
7. Multi-head attention learns diverse attention patterns at the same computational cost
8. Cross-attention connects Encoder and Decoder in sequence-to-sequence models
9. Positional encoding (sinusoidal, RoPE, ALiBi) injects order information
10. Channel attention (SE, CBAM) and window attention (Swin) adapt attention to specific domains
11. Efficient attention variants aim to reduce the quadratic complexity

6.5 Further Reading

- He, K. et al. (2016). Deep residual learning for image recognition. *CVPR*.
- He, K. et al. (2016). Identity mappings in deep residual networks. *ECCV*.
- Xie, S., Girshick, R., Dollar, P., Tu, Z., & He, K. (2017). Aggregated residual transformations for deep neural networks. *CVPR*.
- Huang, G. et al. (2016). Deep networks with stochastic depth. *ECCV*.
- Huang, G. et al. (2017). Densely connected convolutional networks. *CVPR*.
- Vaswani, A. et al. (2017). Attention is all you need. *NeurIPS*.
- Su, J. et al. (2021). RoFormer: Enhanced transformer with rotary position embedding. *arXiv:2104.09864*.
- Press, O. et al. (2022). Train short, test long: Attention with linear biases enables input length extrapolation. *ICLR*.
- Hu, J., Shen, L., & Sun, G. (2018). Squeeze-and-excitation networks. *CVPR*.
- Woo, S. et al. (2018). CBAM: Convolutional block attention module. *ECCV*.
- Liu, Z. et al. (2021). Swin transformer: Hierarchical vision transformer using shifted windows. *ICCV*.

Chapter 7

Optimizers and Learning Rate Schedulers

7.1 Gradient Descent Optimization

Training neural networks minimizes a loss function by updating parameters in the direction that reduces loss.

7.1.1 Batch Gradient Descent

Updates parameters using the gradient of the entire dataset:

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta \mathcal{L}(\theta_t) \tag{7.1}$$

- **Advantages:** Stable convergence, exact gradient
- **Disadvantages:** Slow for large datasets, memory intensive

7.1.2 Stochastic Gradient Descent (SGD)

Updates parameters using gradient of a single sample:

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta \mathcal{L}(\theta_t; \mathbf{x}^{(i_t)}, \mathbf{y}^{(i_t)}) \tag{7.2}$$

- **Advantages:** Fast, escapes local minima, online learning
- **Disadvantages:** Noisy gradients, requires smaller learning rate

7.1.3 Mini-Batch Gradient Descent

Updates using a batch of B samples:

$$\theta_{t+1} = \theta_t - \alpha \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \mathcal{L}(\theta_t; \mathbf{x}^{(i)}, \mathbf{y}^{(i)}) \tag{7.3}$$

This is the standard approach in deep learning.

7.2 Gradient Descent with Momentum

Momentum accelerates convergence by accumulating velocity:

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + (1 - \beta) \nabla_\theta \mathcal{L}(\theta_t) \tag{7.4}$$

$$\theta_{t+1} = \theta_t - \alpha \mathbf{v}_{t+1} \tag{7.5}$$

- $\beta \in [0, 1)$ is the momentum coefficient (typically 0.9)
- **Reduces oscillations** in steep directions
- **Accelerates convergence** in shallow directions

Note 7.1 (Nesterov Accelerated Gradient)**.** Nesterov momentum looks ahead:

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t - \alpha \nabla_\theta \mathcal{L}(\theta_t + \beta \mathbf{v}_t) \tag{7.6}$$
$$\theta_{t+1} = \theta_t + \mathbf{v}_{t+1} \tag{7.7}$$

This provides better convergence guarantees and often faster training.

7.3 Adaptive Learning Rate Methods

These methods adjust learning rates per-parameter based on gradient history.

7.3.1 AdaGrad

Adapts learning rate inversely proportional to accumulated squared gradients:

$$\mathbf{g}_t = \nabla_\theta \mathcal{L}(\theta_t) \tag{7.8}$$
$$\mathbf{r}_t = \mathbf{r}_{t-1} + \mathbf{g}_t \odot \mathbf{g}_t \tag{7.9}$$
$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\mathbf{r}_t + \epsilon}} \odot \mathbf{g}_t \tag{7.10}$$

- **Good for sparse features**
- **Learning rate decreases** over time
- **May stop too early** in deep learning

7.3.2 RMSProp

Exponentially weighted moving average of squared gradients:

$$\mathbf{g}_t = \nabla_\theta \mathcal{L}(\theta_t) \tag{7.11}$$
$$\mathbf{r}_t = \beta \mathbf{r}_{t-1} + (1-\beta)\mathbf{g}_t \odot \mathbf{g}_t \tag{7.12}$$
$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\mathbf{r}_t + \epsilon}} \odot \mathbf{g}_t \tag{7.13}$$

- **Addresses AdaGrad's diminishing learning rate**
- **Common default for RNNs**
- $\beta = 0.9$ is typical

7.3.3 Adam (Adaptive Moment Estimation)

Combines momentum and RMSProp:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t \quad \text{(first moment)} \tag{7.14}$$
$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2 \quad \text{(second moment)} \tag{7.15}$$
$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t} \quad \text{(bias correction)} \tag{7.16}$$
$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t} \tag{7.17}$$
$$\theta_{t+1} = \theta_t - \frac{\alpha \hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \tag{7.18}$$

Properties 7.1 (Adam Properties).

- **Default optimizer** for most deep learning tasks
- **Robust to hyperparameters**
- **Default parameters:** $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$
- **Bias correction** for early time steps

Listing 7.1 – Optimizers in PyTorch

```
import torch.optim as optim

# SGD with momentum
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)

# SGD with Nesterov momentum
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9, nesterov=True)

# Adam
optimizer = optim.Adam(model.parameters(), lr=0.001, betas=(0.9, 0.999))

# AdamW (decoupled weight decay)
optimizer = optim.AdamW(model.parameters(), lr=0.001, weight_decay=0.01)

# RMSProp
optimizer = optim.RMSprop(model.parameters(), lr=0.01, alpha=0.99)
```

7.3.4 AdamW (Adam with Weight Decay)

Decouples weight decay from gradient update:

$$\theta_{t+1} = \theta_t - \alpha \frac{\mathbf{m}_t}{\sqrt{\mathbf{v}_t} + \epsilon} - \alpha\lambda\theta_t \tag{7.19}$$

- **Proper L2 regularization** unlike original Adam
- **Better generalization** in many tasks
- **Standard in transformers**

7.3.5 AdaDelta

AdaDelta (Zeiler, 2012) addresses AdaGrad's monotonically decreasing learning rate by restricting the window of accumulated past gradients to a fixed size w using an exponentially decaying average:

$$\mathbf{r}_t = \rho\mathbf{r}_{t-1} + (1-\rho)\mathbf{g}_t^2 \tag{7.20}$$

$$\Delta\boldsymbol{\theta}_t = -\frac{\sqrt{\mathbf{s}_{t-1} + \epsilon}}{\sqrt{\mathbf{r}_t + \epsilon}}\mathbf{g}_t \tag{7.21}$$

$$\mathbf{s}_t = \rho\mathbf{s}_{t-1} + (1-\rho)(\Delta\boldsymbol{\theta}_t)^2 \tag{7.22}$$

where ρ is a decay constant (typically 0.95).

Properties 7.2 (AdaDelta Properties).

- **No manual learning rate setting**: The numerator $\sqrt{\mathbf{s}_{t-1} + \epsilon}$ replaces α, making the method largely hyperparameter-free

- **Robust to different gradient scales**: Adapts per-parameter using accumulated updates
- **Rarely used today**: Superseded by Adam and its variants

7.3.6 Unified View of Adaptive Methods

Table 7.1 – Adaptive Optimizer Family

Method	1st moment	2nd moment	LR scaling
AdaGrad	No	$\mathbf{r}_t = \mathbf{r}_{t-1} + \mathbf{g}_t^2$	$\alpha/\sqrt{\mathbf{r}_t}$
RMSProp	No	$\mathbf{r}_t = \rho\mathbf{r}_{t-1} + (1-\rho)\mathbf{g}_t^2$	$\alpha/\sqrt{\mathbf{r}_t}$
AdaDelta	No	Same as RMSProp	$\sqrt{\mathbf{s}}/\sqrt{\mathbf{r}_t}$
Adam	$\mathbf{m}_t$ (bias corr.)	$\mathbf{v}_t$ (bias corr.)	$\alpha\hat{\mathbf{m}}_t/\sqrt{\hat{\mathbf{v}}_t}$
AdamW	Same as Adam	Same as Adam	+ decoupled $\lambda\theta_t$

7.3.7 Convergence Rate Analysis

Theorem 7.1 (SGD Convergence Rate)**.** For a smooth (L-Lipschitz gradient) and μ-strongly convex function f, gradient descent with learning rate $\alpha = 1/L$ satisfies:

$$f(\theta_t) - f(\theta^*) \leq \left(1 - \frac{\mu}{L}\right)^t \left(f(\theta_0) - f(\theta^*)\right) \tag{7.23}$$

This is a **linear convergence rate** with rate $1 - \mu/L$.

Properties 7.3 (Convergence Rates for Different Optimizers)**.**

- **Convex + smooth:** SGD achieves $O(1/T)$ rate; accelerated methods achieve $O(1/T^2)$
- **Non-convex:** All methods converge at $O(1/\sqrt{T})$ to a stationary point
- **Adam** is not guaranteed to converge in the non-convex setting without modification (Reddi et al., 2018); **AdamW** and **AMSGrad** fix this issue
- In practice, Adam and AdamW converge faster initially but SGD+Momentum often achieves better final generalization

7.4 Gradient Clipping

Prevents exploding gradients. Two main strategies:

7.4.1 Value Clipping

Clip each gradient component individually:

$$\tilde{g}_i = \text{clip}(g_i, -c, c) = \max(\min(g_i, c), -c) \tag{7.24}$$

where c is the clip value (typically 0.5–5.0).

7.4.2 Norm Clipping

Scale the entire gradient vector if its norm exceeds a threshold:

$$\tilde{\mathbf{g}} = \begin{cases} \mathbf{g} & \text{if } \|\mathbf{g}\| \leq c \\ \frac{c}{\|\mathbf{g}\|}\mathbf{g} & \text{if } \|\mathbf{g}\| > c \end{cases} \tag{7.25}$$

This preserves the gradient **direction** while capping its magnitude.

Properties 7.4 (When to Use Gradient Clipping).

- **RNNs and Transformers:** Essential to prevent exploding gradients through long sequences
- **Fine-tuning pretrained models:** Prevents destabilization from large gradient updates
- **Norm clipping** is generally preferred over value clipping (preserves gradient direction)
- Typical max norm: 1.0 (transformers) to 5.0 (RNNs)
- In PyTorch: `torch.nn.utils.clip_grad_norm_`(model.parameters(), max_norm=1.0)

7.5 Learning Rate Schedulers

Learning rate schedules adjust the learning rate during training.

7.5.1 Step Decay

$$\alpha_t = \alpha_0 \cdot \gamma^{\lfloor t/T \rfloor} \tag{7.26}$$

where γ is the decay factor (e.g., 0.5) and T is the period.

7.5.2 Exponential Decay

$$\alpha_t = \alpha_0 \cdot e^{-\lambda t} \tag{7.27}$$

7.5.3 Cosine Annealing

$$\alpha_t = \alpha_{min} + \frac{1}{2}(\alpha_{max} - \alpha_{min})\left(1 + \cos\left(\frac{t\pi}{T}\right)\right) \tag{7.28}$$

7.5.4 Warmup

Gradually increase learning rate at the start:

$$\alpha_t = \alpha_{max} \cdot \frac{t}{T_{warmup}} \quad \text{for } t < T_{warmup} \tag{7.29}$$

- **Prevents large gradients** at the start
- **Important for transformers**
- **Common: 500-10000 warmup steps**

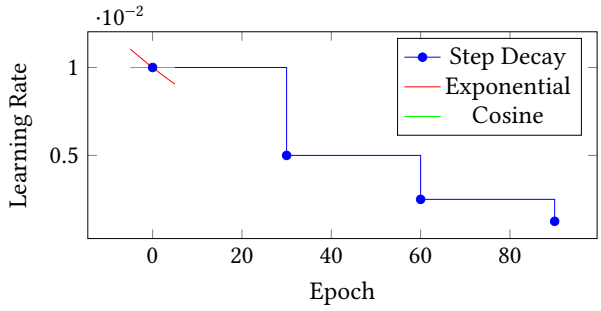


Figure 7.1 – Learning rate schedules

7.5.5 Warmup + Cosine Decay

The standard schedule for modern transformer training combines warmup with cosine annealing:

$$\alpha_t = \begin{cases} \alpha_{max} \cdot \frac{t}{T_{warmup}} & \text{if } t < T_{warmup} \\ \alpha_{min} + \frac{1}{2}(\alpha_{max} - \alpha_{min})\left(1 + \cos\left(\frac{(t - T_{warmup})\pi}{T_{total} - T_{warmup}}\right)\right) & \text{otherwise} \end{cases} \tag{7.30}$$

This is the default in most transformer training recipes (BERT, GPT, ViT).

7.5.6 ReduceLROnPlateau

Monitors a metric (typically validation loss) and reduces the learning rate when improvement stalls:

$$\alpha_t = \alpha_{t-1} \cdot \gamma \quad \text{if no improvement for } p \text{ epochs} \tag{7.31}$$

where γ is the decay factor (typically 0.1) and p is the patience (typically 5–10 epochs).

- **Adaptive**: No need to predefine the total number of epochs
- **Useful when training time is unknown** (e.g., early stopping)
- Common in training CNNs for image classification

7.5.7 OneCycleLR

Proposed by Smith (2018), OneCycleLR uses a single cycle with three phases:

1. **Increase** LR from α_{min} to α_{max} (first ~30% of steps)
2. **Decrease** LR from α_{max} to α_{min} (next ~60% of steps)
3. **Fine-tune**: Decrease LR further to $\alpha_{min}/100$ (final ~10% of steps)

Optionally, the momentum follows an inverse pattern (high when LR is low).

Properties 7.5 (OneCycleLR Benefits).

- **Faster convergence**: Often achieves the same accuracy in fewer epochs
- **Regularization effect**: The high-to-low LR cycle acts as implicit regularization
- **No warmup needed**: Built into the schedule
- Popular in fastai and for transfer learning tasks

7.6 Optimizer Selection

Table 7.2 – Optimizer Selection Guide

Optimizer	Good for	Notes
SGD + Momentum	Image classification	Best generalization
Adam	General purpose	Default choice
AdamW	Transformers, NLP	Best for fine-tuning
RMSProp	RNNs	Good for recurrent models

7.7 Chapter Summary

1. Gradient descent is the foundation of neural network optimization
2. Momentum accelerates convergence by accumulating velocity
3. Adaptive methods (AdaGrad, RMSProp, AdaDelta, Adam) adjust per-parameter learning rates
4. Adam combines momentum and adaptive learning rates; AdamW adds proper weight decay
5. Gradient clipping prevents exploding gradients (norm clipping preferred)
6. Learning rate schedulers (step, cosine, plateau, OneCycleLR) improve convergence
7. Warmup is important for transformer training
8. SGD+Momentum often generalizes better; Adam converges faster

7.8 Further Reading

- Kingma, D. & Ba, J. (2015). Adam: A method for stochastic optimization. *ICLR*.
- Loshchilov, I. & Hutter, F. (2019). Decoupled weight decay regularization. *ICLR*.
- Zeiler, M. (2012). ADADELTA: An adaptive learning rate method. *arXiv:1212.5701*.
- Reddi, S. et al. (2018). On the convergence of Adam and beyond. *ICLR*.
- Smith, L. (2018). A disciplined approach to neural network hyper-parameters. *NeurIPS Workshop*.
- Loshchilov, I. & Hutter, F. (2017). SGDR: Stochastic gradient descent with warm restarts. *ICLR*.
- Paszke, A. et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *NeurIPS*.

Chapter 8

Classic Network Architectures

8.1 Evolution of Neural Networks

The history of deep learning shows a progression from simple networks to increasingly complex architectures, each building upon innovations of the previous era.

8.2 Perceptron and Early Neural Networks

8.2.1 The Perceptron (1958)

The perceptron, introduced by Frank Rosenblatt, was the first attempt at artificial neural networks:

$$y = \begin{cases} 1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b > 0 \\ 0 & \text{otherwise} \end{cases} \tag{8.1}$$

Limitations: Can only solve linearly separable problems (XOR problem showed this).

8.3 LeNet (1998)

Yann LeCun's LeNet was the first successful CNN for digit recognition.

Architecture 8.1 (LeNet-5 Architecture)**.**

1. Input: 32×32 grayscale image
2. C1: 6 filters of 5×5, stride 1 $\rightarrow 28 \times 28 \times 6$
3. S2: 2×2 avg pool, stride 2 $\rightarrow 14 \times 14 \times 6$
4. C3: 16 filters of $5 \times 5 \rightarrow 10 \times 10 \times 16$
5. S4: 2×2 avg pool $\rightarrow 5 \times 5 \times 16$
6. C5: 120 filters of $5 \times 5 \rightarrow 120$
7. F6: Fully connected, 84 units
8. Output: 10 classes (digits)

Total parameters: ~60,000

Key innovations:

- Weight sharing through convolutions
- Local receptive fields
- Spatial subsampling (pooling)

8.4 AlexNet (2012)

AlexNet won ImageNet LSVRC-2012 with a significant margin, revolutionizing computer vision.

Architecture 8.2 (AlexNet Architecture).

1. Input: $224 \times 224 \times 3$ RGB image
2. Conv1: 96 filters of 11×11, stride 4 $\rightarrow 55 \times 55 \times 96$
3. Pool1: 3×3 max pool, stride 2 $\rightarrow 27 \times 27 \times 96$
4. Norm1: Response normalization
5. Conv2: 256 filters of $5 \times 5 \rightarrow 27 \times 27 \times 256$
6. Pool2: 3×3 max pool $\rightarrow 13 \times 13 \times 256$
7. Conv3: 384 filters of 3×3
8. Conv4: 384 filters of 3×3
9. Conv5: 256 filters of 3×3
10. Pool5: 3×3 max pool $\rightarrow 6 \times 6 \times 256$
11. FC6: 4096 neurons
12. FC7: 4096 neurons
13. FC8: 1000 classes (ImageNet)

Key innovations:

- ReLU activation (faster training)
- GPU training (5-6 days vs weeks)
- Data augmentation
- Dropout regularization
- Response normalization

Parameters: ~60 million

8.5 VGGNet (2014)

VGGNet emphasized depth and simplicity, using only 3×3 convolutions.

Architecture 8.3 (VGG-16 Architecture).

1. Input: $224 \times 224 \times 3$
2. Conv blocks:
   - Block 1: 2× conv 64, pool
   - Block 2: 2× conv 128, pool
   - Block 3: 3× conv 256, pool
   - Block 4: 3× conv 512, pool
   - Block 5: 3× conv 512, pool
3. FC: 4096, 4096, 1000

Key insight: Two 3×3 convolutions have receptive field of 5×5 but fewer parameters:

- 5×5 conv: $25C^2$ parameters

- Two 3×3 convs: $18C^2$ parameters (28% reduction)
- More nonlinearities with two ReLUs

Parameters: ~138 million

8.6 GoogLeNet / Inception (2014)

Introduced the Inception module with parallel multi-scale convolutions.

Architecture 8.4 (Inception Module)**.** Parallel operations at different scales:

- 1×1 conv (bottleneck)
- 3×3 conv (after 1×1)
- 5×5 conv (after 1×1)
- 3×3 max pool (after 1×1)

Concatenate all outputs along channel dimension.

Key innovations:

- Multi-scale feature extraction
- Dimensionality reduction with 1×1 conv
- Global Average Pooling
- Auxiliary classifiers for training

Parameters: ~4 million (very efficient!)

8.7 ResNet (2015)

ResNet introduced residual connections, enabling training of very deep networks.

Architecture 8.5 (ResNet-50 (Bottleneck))**.**

1. Conv1: 7×7, 64, stride 2
2. Pool: 3×3 max, stride 2
3. Stage 1: 3 blocks, 64 channels
4. Stage 2: 4 blocks, 128 channels
5. Stage 3: 6 blocks, 256 channels
6. Stage 4: 3 blocks, 512 channels
7. Pool: Global Average
8. FC: 1000 classes

Key innovations:

- Skip connections for gradient flow
- Identity shortcuts
- Bottleneck design (1×1, 3×3, 1×1)
- Enables 1000+ layers

Parameters: ~25 million

8.8 DenseNet (2017)

Dense connections where each layer receives inputs from all preceding layers.

Architecture 8.6 (DenseNet Block)**.** For a block with L layers, each receives $k_0 + k \times (l-1)$ inputs:

$$\mathbf{x}_1 = H_1([\mathbf{x}_0]) \tag{8.2}$$
$$\mathbf{x}_2 = H_2([\mathbf{x}_0, \mathbf{x}_1]) \tag{8.3}$$
$$\mathbf{x}_3 = H_3([\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2]) \tag{8.4}$$
$$\vdots \tag{8.5}$$
$$\mathbf{x}_L = H_L([\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{L-1}]) \tag{8.6}$$

Key innovations:

- Feature reuse reduces parameters
- Strong gradient flow
- Better parameter efficiency than ResNet

Parameters: ~27 million (similar to ResNet but better)

8.9 Transformer (2017)

The Transformer architecture replaced recurrence with self-attention.

Architecture 8.7 (Transformer Architecture)**.**

1. **Encoder:** N identical layers
   - Multi-head self-attention
   - Feed-forward network
   - Layer normalization
   - Residual connections
2. **Decoder:** N identical layers
   - Masked multi-head self-attention
   - Encoder-decoder attention
   - Feed-forward network
3. **Positional encoding** (sinusoidal or learned)

Key innovations:

- Self-attention replaces recurrence
- Parallelizable (unlike RNNs)
- Captures long-range dependencies
- Scalable with computation

Parameters: 65M (base) to 175B (GPT-3) and beyond

8.10 MobileNet (2017)

MobileNet introduced depthwise separable convolutions for efficient mobile inference (Howard et al., 2017).

Architecture 8.8 (MobileNet Building Block).

1. Depthwise convolution: $K \times K$ conv, 1 filter per input channel
2. Pointwise convolution: 1×1 conv, C_{out} filters

FLOPs reduction vs standard convolution:

$$\frac{\text{Standard}}{\text{Separable}} = \frac{K^2 \cdot C_{in} \cdot C_{out}}{K^2 \cdot C_{in} + C_{in} \cdot C_{out}} \approx \frac{1}{\frac{1}{C_{out}} + \frac{1}{K^2}} \tag{8.7}$$

For $K = 3$, $C_{out} = 256$: speedup $\approx$ 8–9×.

Properties 8.1 (MobileNet Design Choices).

- **Width multiplier** α: Reduces channels uniformly ($\alpha \in \{0.25, 0.5, 0.75, 1.0\}$)
- **Resolution multiplier** ρ: Reduces input resolution (224, 192, 160, 128)
- MobileNetV1: 4.2M params, 0.575 GMACs for $\alpha = 1.0$
- MobileNetV2: Adds inverted residuals and linear bottlenecks (3.4M params)
- MobileNetV3: NAS-optimized, hardware-aware design (2.5M–5.4M params)

8.11 SENet (2018)

Squeeze-and-Excitation Networks (Hu et al., 2018) introduce channel attention to adaptively recalibrate feature maps.

Architecture 8.9 (SE Block).

1. **Squeeze**: Global average pooling $\rightarrow 1 \times 1 \times C$ descriptor
2. **Excitation**: Two FC layers with bottleneck (reduction ratio $r = 16$)

$$\mathbf{z} = \mathbf{W}_2 \cdot \delta(\mathbf{W}_1 \cdot \mathbf{s}) \tag{8.8}$$

$$\text{where} \quad \mathbf{s} = \frac{1}{H \cdot W} \sum_{i,j} \mathbf{X}_{i,j} \quad \text{(squeeze)} \tag{8.9}$$

3. **Scale**: Channel-wise multiplication $\tilde{\mathbf{X}}_c = \mathbf{X}_c \cdot \sigma(\mathbf{z})_c$

Key insight: Different channels have different importance for different inputs. The SE block learns which channels to emphasize or suppress. Added overhead is minimal ($<$ 1% of total FLOPs) but consistently improves accuracy by 1–2%.

8.12 EfficientNet (2019)

EfficientNet (Tan & Le, 2019) scales all three dimensions (depth, width, resolution) simultaneously using a compound scaling coefficient ϕ:

$$\text{depth:} \quad d = \alpha^\phi \tag{8.10}$$

$$\text{width:} \quad w = \beta^\phi \tag{8.11}$$

$$\text{resolution:} \quad r = \gamma^\phi \tag{8.12}$$

$$\text{subject to:} \quad \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2 \quad (\text{target FLOPs} \approx 2^\phi) \tag{8.13}$$

Table 8.1 – EfficientNet Family

Variant	Params	FLOPs	Resolution	Top-1 Acc.
B0	5.3M	0.39B	224^2	77.1%
B1	7.8M	0.70B	240^2	79.1%
B2	9.2M	1.0B	260^2	80.1%
B3	12M	1.8B	300^2	81.6%
B4	19M	4.2B	380^2	82.9%
B7	66M	37B	600^2	84.3%

EfficientNet achieves state-of-the-art accuracy with significantly fewer parameters and FLOPs than previous architectures. The compound scaling principle has become a standard design guideline.

8.13 Vision Transformer (ViT) (2020)

Vision Transformer (Dosovitskiy et al., 2020) applies the transformer architecture directly to images, treating patches as tokens.

Architecture 8.10 (ViT Architecture).

1. **Patch embedding**: Split 224×224 image into 16×16 patches (196 tokens), linearly project each
2. **[CLS] token**: Prepend a learnable class token
3. **Positional encoding**: Add learned position embeddings
4. **Transformer encoder**: L layers of multi-head self-attention + MLP
5. **Classification head**: Use [CLS] token output for classification

Properties 8.2 (ViT Key Insights).

- **No inductive bias**: Unlike CNNs, ViT has no built-in translation equivariance or locality — it must learn these from data
- **Data hungry**: ViT requires large-scale pretraining (ImageNet-21k or JFT-300M) to outperform CNNs
- **Scales well**: Performance improves consistently with more data and compute
- **ViT-B/16**: 86M params, 17.6B FLOPs, 79.9% Top-1 (ImageNet, from 21k pretrain)
- **ViT-L/16**: 307M params, 61.6B FLOPs, 82.3% Top-1

- **DeiT**: Distillation enables ViT training from scratch on ImageNet-1k alone

Note 8.1 (CNNs vs ViTs: The Current Landscape)**.** As of 2024, the field has converged toward hybrid approaches:

- **ConvNeXt**: Modern CNNs redesigned to match ViT scalability (Liu et al., 2022)
- **Swin Transformer**: Hierarchical ViT with shifted windows for dense prediction
- **Both paradigms achieve similar performance**; the choice often depends on downstream task and infrastructure

8.14 U-Net (2015)

U-Net (Ronneberger et al., 2015) is an encoder-decoder architecture designed for biomedical image segmentation. Its U-shaped structure gives it its name.

Architecture 8.11 (U-Net Architecture)**.**

1. **Contracting path (encoder)**: 4 blocks of 3×3 conv + ReLU + 2×2 max pool
2. **Bottleneck**: 3×3 conv at the lowest resolution
3. **Expansive path (decoder)**: Upsampling + concatenation with skip connections + 3×3 conv
4. Skip connections connect each encoder level to the corresponding decoder level

Properties 8.3 (U-Net Design Principles)**.**

- **Skip connections**: Preserve fine-grained spatial detail lost during downsampling
- **Symmetric design**: Encoder and decoder have matching resolutions
- **Works with limited data**: Original paper achieved excellent results with only 30 training images
- Widely extended: Attention U-Net, U-Net++, nnU-Net (automated configuration)
- Foundation of modern segmentation: used in medical imaging, satellite analysis, and autonomous driving

8.15 ShuffleNet (2018)

ShuffleNet (Zhang et al., 2018) introduces channel shuffling to enable efficient point-wise group convolutions.

The key operation, **channel shuffle**, rearranges channels across groups before the next group convolution:

$$\text{Shuffle}(\mathbf{X})_{g,c,h,w} = \mathbf{X}_{\lfloor c/(C/G) \rfloor, \left(g + c \cdot G\right) \bmod C, h, w} \tag{8.14}$$

Without shuffling, stacked group convolutions would only communicate within their group (blocking cross-group information flow). Channel shuffle provides an efficient alternative to the 1×1 convolution used in MobileNet.

Properties 8.4 (ShuffleNet vs MobileNet)**.**

- ShuffleNetV1: 1.5× faster than MobileNet on ARM devices

- ShuffleNetV2: Introduces channel split and four guidelines for efficient design (equal channel width, balanced ops, minimize memory access, element-wise ops cheap)
- ShuffleNetV2 (1.5×): 3.5M params, 299M MACs, 73.3% Top-1

Table 8.2 – Architecture Comparison

Network	Year	Parameters	Top-1 Acc.	Key Innovation
AlexNet	2012	60M	62.5%	GPU training, ReLU
VGG-16	2014	138M	71.5%	3×3 conv stack
GoogLeNet	2014	4M	72.0%	Inception module
ResNet-50	2015	25M	76.1%	Skip connections
DenseNet-121	2017	8M	74.4%	Dense connections
MobileNetV2	2018	3.4M	72.0%	Depthwise separable
EfficientNet-B7	2019	66M	84.3%	Compound scaling
ViT-B/16	2021	86M	79.9%	Patch-based attention
ConvNeXt-L	2022	350M	85.1%	Modernized CNN

8.16 Chapter Summary

1. LeNet pioneered CNNs for digit recognition
2. AlexNet popularized deep learning with GPU training and ReLU
3. VGG showed the power of depth through 3×3 convolution stacks
4. Inception introduced multi-scale processing and dimensionality reduction
5. ResNet enabled ultra-deep networks with skip connections
6. DenseNet improved parameter efficiency through feature reuse
7. MobileNet and ShuffleNet brought efficient inference to mobile devices
8. SENet introduced channel attention, later extended to CBAM
9. EfficientNet showed that compound scaling outperforms single-dimension scaling
10. ViT demonstrated that pure attention can match or exceed CNNs at scale
11. U-Net defined the encoder-decoder paradigm for segmentation
12. The field continues to converge: modern CNNs and ViTs achieve similar performance

8.17 Further Reading

- Krizhevsky, A., Sutskever, I., & Hinton, G. (2012). ImageNet classification with deep convolutional neural networks. *NeurIPS*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *CVPR*.
- Hu, J., Shen, L., & Sun, G. (2018). Squeeze-and-excitation networks. *CVPR*.
- Howard, A. et al. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv:1704.04861*.
- Tan, M. & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *ICML*.
- Dosovitskiy, A. et al. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. *ICLR*.
- Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. *MICCAI*.
- Liu, Z. et al. (2022). A convnet for the 2020s. *CVPR*.

Chapter 9

Parameter and Computation Analysis

9.1 Importance of Model Efficiency

Understanding model efficiency is crucial for:

- Deployment on resource-constrained devices
- Training cost estimation
- Model selection and architecture design
- Benchmarking and comparison

Key metrics:

- **Parameters:** Model size and memory footprint
- **FLOPs:** Computational complexity
- **Latency:** Inference time
- **Throughput:** Samples per second

9.2 Parameter Counting

Parameters are the learnable weights and biases in the network.

9.2.1 Fully Connected Layer

For input dimension d_{in} and output dimension d_{out}:

$$\text{Weights} = d_{in} \times d_{out} \tag{9.1}$$

$$\text{Biases} = d_{out} \tag{9.2}$$

$$\text{Total} = d_{in} \times d_{out} + d_{out} \tag{9.3}$$

Example: FC layer from 784 to 256:

$$784 \times 256 + 256 = 200,704 + 256 = 200,960 \text{ parameters} \tag{9.4}$$

9.2.2 Convolutional Layer

For input channels C_{in}, output channels C_{out}, kernel size K:

$$\text{Weights} = K \times K \times C_{in} \times C_{out} \tag{9.5}$$

$$\text{Biases} = C_{out} \quad \text{(typically)} \tag{9.6}$$

$$\text{Total} = K^2 \times C_{in} \times C_{out} + C_{out} \tag{9.7}$$

Example: Conv layer 3×3, 64 in, 128 out:

$$3 \times 3 \times 64 \times 128 + 128 = 73,728 + 128 = 73,856 \text{ parameters} \tag{9.8}$$

9.2.3 Batch Normalization

For C channels:

$$\gamma \text{ (scale)} = C \tag{9.9}$$
$$\beta \text{ (shift)} = C \tag{9.10}$$
$$\mu \text{ (running mean)} = C \quad \text{(inference only)} \tag{9.11}$$
$$\sigma \text{ (running var)} = C \quad \text{(inference only)} \tag{9.12}$$
$$\text{Total trainable} = 2C \tag{9.13}$$

Note: During training, BN has 2 trainable and 2 non-trainable parameters per channel.

9.3 FLOPs (Floating Point Operations)

FLOPs measure the computational cost of a forward pass.

9.3.1 FLOPs for FC Layer

For matrix multiplication $\mathbf{Y} = \mathbf{WX} + \mathbf{b}$:

$$\text{Multiplications} = d_{out} \times d_{in} \quad \text{(per sample)} \tag{9.14}$$
$$\text{Additions} = d_{out} \times (d_{in} - 1) \quad \text{(approximately } d_{out} \times d_{in}\text{)} \tag{9.15}$$
$$\text{FLOPs} \approx 2 \times d_{out} \times d_{in} \quad \text{(counting mult+add as 2 FLOPs)} \tag{9.16}$$

For batch of N: FLOPs $= 2 \times N \times d_{out} \times d_{in}$

9.3.2 FLOPs for Conv Layer

For output feature map $H_{out} \times W_{out}$:

$$\text{FLOPs per output pixel} = K^2 \times C_{in} \times C_{out} \quad \text{(mult-add pairs)} \times 2 \tag{9.17}$$
$$\text{Total FLOPs} = H_{out} \times W_{out} \times K^2 \times C_{in} \times C_{out} \times 2 \tag{9.18}$$

Example: 224×224 input, 3×3 conv, 64 in, 128 out:

$$H_{out} \times W_{out} = 224 \times 224 = 50,176 \tag{9.19}$$
$$\text{FLOPs} = 50,176 \times 3 \times 3 \times 64 \times 128 \times 2 \approx 1.17 \text{ GFLOPs} \tag{9.20}$$

Note 9.1 (MACs vs FLOPs)**.** Sometimes FLOPs are reported as Multiply-Accumulate (MAC) operations:

$$\text{MAC} = \sum_i a_i \cdot b_i \tag{9.21}$$

Each MAC typically counts as 1 operation (or 2 FLOPs if counting mult and add sepa-

rately).

1 MAC ≈ 2 FLOPs for basic arithmetic.

9.4 Practical Examples

9.4.1 LeNet-5 Parameter Count

Layer	Input	Weights	FLOPs
Conv1	$1 \times 32 \times 32$	150	$2.7M$
Conv2	$6 \times 14 \times 14$	$2,400$	$4.2M$
FC1	400	$240K$	$480K$
FC2	120	$48K$	$48K$
Total	-	~60K	~7M

9.4.2 AlexNet Parameter Count

Layer	Output	Params	FLOPs
Conv1	$96 \times 55 \times 55$	35K	105M
Conv2	$256 \times 27 \times 27$	614K	448M
Conv3	$384 \times 13 \times 13$	885K	149M
Conv4	$384 \times 13 \times 13$	1.3M	224M
Conv5	$256 \times 13 \times 13$	884K	149M
FC6	4096	37.7M	37.7M
FC7	4096	16.8M	16.8M
FC8	1000	4.1M	4.1M
Total	-	~60M	~1.1B

9.5 Transfer Learning and Efficiency

Transfer learning can dramatically reduce required data and training.

Properties 9.1 (Transfer Learning Benefits).

- Lower training cost (no training from scratch)
- Can use smaller datasets
- Faster convergence
- Often better final performance

Common strategies:

1. **Freeze:** Keep pretrained weights fixed
2. **Fine-tune:** Adapt all/some layers with small learning rate
3. **Extract features:** Use pretrained as feature extractor

9.6 Detailed Model Breakdown

9.6.1 ResNet-50 Parameter Analysis

ResNet-50 consists of 4 stages with bottleneck blocks (1×1, 3×3, 1×1 convolutions).

Table 9.1 – ResNet-50 Layer-by-Layer Parameter Count

Layer	Output Shape	Parameters	FLOPs
Conv1 (7×7, 64, s2)	$112^2 \times 64$	9,408	118M
MaxPool 3×3, s2	$56^2 \times 64$	0	–
Stage 1 (3 blocks, 64)	$56^2 \times 256$	215,808	560M
Stage 2 (4 blocks, 128)	$28^2 \times 512$	1,167,616	1,180M
Stage 3 (6 blocks, 256)	$14^2 \times 1024$	4,715,776	2,360M
Stage 4 (3 blocks, 512)	$7^2 \times 2048$	7,095,552	2,360M
Global AvgPool	2048	0	–
FC (1000 classes)	1000	2,049,000	4M
Total		**25,557,032**	**≈4.1B**

Key observations:

- Stage 3 and 4 contain 46% of all parameters but 55% of FLOPs (high channel counts at lower resolutions)
- The final FC layer alone is 2M parameters (8% of total) — often replaced with GAP in modern variants
- ResNet-50-D (with avgpool in downsampling) improves accuracy with zero extra parameters

9.6.2 Transformer Model Scaling

For a transformer model with L layers, d_{model} hidden dimension, h attention heads, and vocabulary size V:

$$\text{Embedding params} = V \times d_{model} \tag{9.22}$$

$$\text{Per-layer params} \approx 12 \cdot d_{model}^2 \quad (\text{attention: } 4d_{model}^2, \text{ FFN: } 8d_{model}^2) \tag{9.23}$$

$$\text{Total params} \approx V \cdot d_{model} + 12L \cdot d_{model}^2 \tag{9.24}$$

Note 9.2 (The Scaling Laws)**.** Kaplan et al. (2020) showed that transformer performance

Table 9.2 – Transformer Model Sizes

Model	Layers	d_{model}	Params	FLOPs/token
BERT-Base	12	768	110M	22.5B
BERT-Large	24	1024	340M	84B
GPT-2 Small	12	768	117M	–
GPT-2 Medium	24	1024	345M	–
GPT-2 XL	24	1600	774M	–
GPT-3	96	12288	175B	–
LLaMA-7B	32	4096	6.7B	–

depends predictably on three factors:

$$\mathcal{L}(N, D, C) \approx \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D} \tag{9.25}$$

where N = parameters, D = data size, C = compute, and $\alpha_N \approx 0.076$, $\alpha_D \approx 0.095$. This implies that to maintain efficiency, parameters, data, and compute should scale proportionally ($\approx \times 10$ each for each "order of magnitude" improvement).

9.7 Activation Memory Analysis

During training, memory consumption is dominated by **activations** (intermediate feature maps stored for backpropagation), not parameters.

9.7.1 Activation Memory Formula

For a convolutional layer output of shape (N, C, H, W) in FP32:

$$\text{Activation memory} = N \times C \times H \times W \times 4 \text{ bytes (FP32)} \tag{9.26}$$

Example 9.1 (ResNet-50 Activation Memory)**.** For batch size $N = 32$, FP32:

- Stage 1 output ($56^2 \times 256$): $32 \times 256 \times 56^2 \times 4 = 103$ MB
- Stage 2 output ($28^2 \times 512$): $32 \times 512 \times 28^2 \times 4 = 51$ MB
- Stage 3 output ($14^2 \times 1024$): $32 \times 1024 \times 14^2 \times 4 = 26$ MB
- Total activations for all layers (including intermediates): $\approx$ 3–4 GB

With mixed precision (FP16), activation memory is halved ($\approx$ 2 GB).

Properties 9.2 (Reducing Activation Memory)**.**

- **Gradient checkpointing**: Recompute activations during backward pass. Trades compute (~33% overhead) for memory (~60% reduction).
- **Mixed precision (AMP)**: Store activations in FP16, master weights in FP32. Halves activation memory with minimal accuracy loss.
- **Gradient accumulation**: Simulate large batch sizes without proportional memory increase.

- **DeepSpeed ZeRO**: Partition optimizer states, gradients, and parameters across GPUs.

9.8 Model Compression and Quantization

9.8.1 Quantization

Quantization reduces the numerical precision of weights and activations:

Table 9.3 – Quantization Levels

Precision	Bytes/param	Range	Size (100M)	Speedup
FP32	4	$\pm 3.4 \times 10^{38}$	400 MB	1×
FP16	2	± 65504	200 MB	2×
BF16	2	$\pm 3.4 \times 10^{38}$	200 MB	2×
INT8	1	$[-128, 127]$	100 MB	2–4×
INT4	0.5	$[-8, 7]$	50 MB	4–8×

Post-training quantization (PTQ) quantizes a trained model without retraining. **Quantization-aware training (QAT)** simulates quantization during training for higher accuracy.

9.8.2 Other Compression Techniques

- **Pruning**: Remove unimportant weights (magnitude-based or structured). Can reduce FLOPs by 2–3× with $< 1\%$ accuracy loss.
- **Knowledge distillation**: Train a smaller "student" model to mimic a larger "teacher". The student often exceeds a same-size model trained directly.
- **Low-rank factorization**: Decompose weight matrices $\mathbf{W} = \mathbf{U}\mathbf{V}^\top$ where $\text{rank}(\mathbf{U}) = \text{rank}(\mathbf{V}) \ll \text{rank}(\mathbf{W})$.

9.9 Chapter Summary

1. Parameter count determines model size and memory requirements
2. FLOPs measure computational complexity (1 MAC $\approx$ 2 FLOPs)
3. ResNet-50: 25.6M params, 4.1B FLOPs; transformer params scale as $12Ld^2$
4. Activation memory often exceeds parameter memory during training
5. Gradient checkpointing and mixed precision reduce memory with compute trade-offs
6. Quantization (INT8, INT4) enables efficient deployment on edge devices
7. Transfer learning leverages pretrained models efficiently
8. Scaling laws guide the design of large foundation models

9.10 Further Reading

- He, K. & Sun, J. (2015). Convolutional neural networks at constrained time cost. *CVPR*.
- Kaplan, J. et al. (2020). Scaling laws for neural language models. *arXiv:2001.08361*.
- Han, S. et al. (2016). Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. *ICLR*.
- Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network. *NeurIPS Workshop*.
- Micikevicius, P. et al. (2018). Mixed precision training. *ICLR*.
- Howard, A. et al. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv:1704.04861*.
- Chen, Y. et al. (2016). Training deep nets with sublinear memory cost. *arXiv:1604.06174*.

Chapter 10

Training Process and Frameworks

10.1 Introduction

Training a neural network is the process of finding a set of parameters that minimize a given loss function over a dataset. While the mathematical foundations of optimization (gradient descent, backpropagation) were covered in previous chapters, this chapter focuses on the *practical and theoretical aspects* of the training process itself. We examine why certain techniques work, how to diagnose training failures, and what modern frameworks provide to make large-scale training tractable.

The training process encompasses several interrelated components: the optimization algorithm, the regularization strategy, the hyperparameter configuration, and the infrastructure for efficient computation. A deep understanding of how these components interact is essential for successfully training modern neural networks.

10.2 The Training Loop

Neural network training consists of iterative optimization over data using minibatch stochastic gradient descent (SGD) or its variants.

10.2.1 Formal Description

Given a dataset $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$, a model $f(\mathbf{x}; \theta)$ with parameters θ, and a loss function ℓ, the training objective is:

$$\theta^* = \arg\min_\theta \frac{1}{N} \sum_{i=1}^N \ell\big(f(\mathbf{x}_i; \theta), \mathbf{y}_i\big) \tag{10.1}$$

In practice, we compute the gradient over minibatches $\mathcal{B}_t \subset \mathcal{D}$ of size B:

$$\mathbf{g}_t = \nabla_\theta \frac{1}{B} \sum_{i \in \mathcal{B}_t} \ell\big(f(\mathbf{x}_i; \theta_t), \mathbf{y}_i\big) \tag{10.2}$$

Algorithm 2 Training Loop

1. Initialize model weights θ_0
2. For each epoch $e = 1, 2, \ldots, E$:
   (a) Shuffle training data
   (b) For each minibatch $\mathcal{B}_t$:
      i. Forward pass: compute predictions $\hat{\mathbf{y}} = f(\mathbf{x}; \theta_t)$
      ii. Compute loss $\mathcal{L}_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \ell(\hat{\mathbf{y}}_i, \mathbf{y}_i)$
      iii. Backward pass: compute gradients $\mathbf{g}_t = \nabla_\theta \mathcal{L}_t$
      iv. Update weights: $\theta_{t+1} = \theta_t - \alpha \cdot \text{optimizer_step}(\mathbf{g}_t)$
   (c) Evaluate on validation set
   (d) Save checkpoint if validation metric improved
3. Return best model

10.2.2 Why Minibatch SGD Works

A natural question is: why use small batches instead of computing the exact gradient over the full dataset? The answer lies in a beneficial interplay between three factors:

1. **Noise as implicit regularization.** The gradient noise from small batches prevents the optimizer from settling into sharp minima that generalize poorly. Keskar et al. [34] showed that large-batch training tends to converge to "sharp" minima with worse generalization, while small-batch SGD finds "flat" minima.
2. **Computational efficiency.** Modern hardware (GPUs/TPUs) is optimized for parallel matrix operations. A batch size of 32–256 fully utilizes GPU parallelism while still providing meaningful gradient estimates.
3. **Regular progress monitoring.** Each minibatch step provides a signal, allowing early detection of training issues (divergence, oscillation) before wasting hours of compute.

Theorem 10.1 (Convergence of SGD with Decreasing Learning Rate)**.** Under convexity and smoothness assumptions on $\mathcal{L}$, SGD with learning rate schedule $\alpha_t = \mathcal{O}(1/\sqrt{t})$ satisfies:

$$\mathbb{E}[\mathcal{L}(\theta_T)] - \mathcal{L}(\theta^*) \leq \mathcal{O}\left(\frac{1}{\sqrt{T}}\right) \tag{10.3}$$

compared to $\mathcal{O}(1/T)$ for full-batch gradient descent. The slower rate is the price paid for cheaper per-step computation.

10.3 Hyperparameters

Critical hyperparameters that affect training dynamics and final model quality.

10.3.1 Learning Rate

The learning rate α is arguably the single most important hyperparameter. It directly controls the step size in parameter space and determines whether training converges, oscillates, or diverges.

10.3.1.1 Effect of Learning Rate

- **Too high:** The loss diverges or oscillates wildly. The optimizer "overshoots" minima.
- **Too low:** Training converges extremely slowly and may get trapped in poor local minima or saddle points.
- **Just right:** The loss decreases steadily, with slight oscillation that provides regularization.
- **Typical range:** 10^{-5} to 0.1
- **Default for Adam:** 10^{-3}
- **Default for SGD:** 10^{-2}

10.3.1.2 Learning Rate Range Test

Smith [35] proposed a simple but effective method for finding a good learning rate. The idea is to start with a very small learning rate and increase it exponentially during a single training run, plotting the loss against the learning rate:

1. Start with $\alpha_{\text{min}} = 10^{-6}$ (or smaller)
2. Increase α by a factor of e (or a constant factor) each batch
3. Stop when the loss starts to increase
4. Choose α at the point of steepest descent (typically 10^{-1} to 10^{-1} of the maximum value)

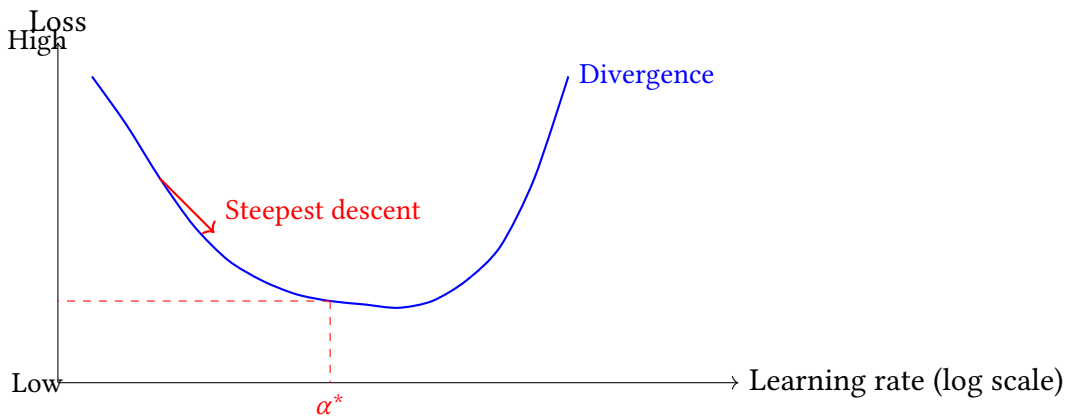


Figure 10.1 – Learning rate range test. The steepest descent point α^* provides a good starting learning rate.

10.3.1.3 Learning Rate Warmup

For large models trained with Adam, it is standard practice to use a warmup phase where the learning rate is gradually increased from zero to the target value over the first few hundred or thousand steps. This prevents instability in the early stages when the Adam momentum estimates are unreliable (being initialized to zero).

The warmup schedule is typically:

$$\alpha_t = \alpha_{\max} \cdot \min\left(1, \frac{t}{T_{\text{warmup}}}\right) \tag{10.4}$$

where T_{warmup} is the number of warmup steps (e.g., 1000 for Transformer models [28]).

10.3.2 Batch Size

The batch size B controls how many samples are processed before each parameter update. It has profound effects on both training speed and model quality.

10.3.2.1 Batch Size Effects

- **Small batch (16–64):** Noisier gradients provide implicit regularization, often better generalization. But underutilizes GPU parallelism.
- **Medium batch (64–256):** Good trade-off between generalization and hardware utilization. Most practitioners' default.
- **Large batch (256–8192+):** Better hardware utilization, faster wall-clock training. But typically requires learning rate scaling and may hurt generalization.

10.3.2.2 Linear Scaling Rule

Goyal et al. **goyal2017accurate** proposed the linear scaling rule for large-batch training: when the batch size is multiplied by k, the learning rate should also be multiplied by k:

$$\alpha_{\text{large}} = \alpha_{\text{small}} \times \frac{B_{\text{large}}}{B_{\text{small}}} \tag{10.5}$$

Note 10.1 (Batch Size vs. Generalization)**.** Research by Keskar et al. [34] and Hoffer et al. [36] shows that smaller batches often achieve better generalization because gradient noise acts as implicit regularization. However, the effect depends on other factors like learning rate, network architecture, and training duration.

10.4 Regularization

Regularization encompasses all techniques that reduce overfitting by constraining the model's capacity or adding noise to the training process. We provide theoretical foundations for the most important methods.

10.4.1 Dropout

Dropout [37] is a regularization technique that randomly sets neuron activations to zero during training with probability p (the dropout rate).

Table 10.1 – Hyperparameter Ranges and Default Values

Hyperparameter	Typical Range	Default	Notes
Learning Rate	10^{-5} to 0.1	Adam: 10^{-3}; SGD: 10^{-2}	Most important
Batch Size	16–8192	32–256	Depends on GPU memory
Epochs	10–1000	Task-dependent	Early stopping recommended
Weight Decay	10^{-6} to 10^{-2}	10^{-2} (AdamW)	See Section 10.4.2
Dropout Rate	0.1–0.5	0.1–0.3	See Section 10.4.1
Momentum	0.9–0.999	0.9 (SGD)	0.9/0.999 for Adam
Warmup Steps	0–10K	1000 (Transformers)	Large models benefit more

10.4.1.1 Forward Pass with Dropout

During training, for a layer with input $\mathbf{x}$ and weight matrix $\mathbf{W}$:

$$\mathbf{y} = \mathbf{W} \cdot (\mathbf{x} \odot \mathbf{m}), \quad \mathbf{m} \sim \text{Bernoulli}(1 - p) \tag{10.6}$$

where $\mathbf{m}$ is a binary mask and $\odot$ denotes element-wise multiplication.

10.4.1.2 Inverted Dropout

The standard implementation uses *inverted dropout*, which scales activations during training so that no scaling is needed at inference time:

$$\mathbf{y} = \frac{\mathbf{W} \cdot (\mathbf{x} \odot \mathbf{m})}{1 - p} \tag{10.7}$$

This ensures that $\mathbb{E}[\mathbf{y}_{\text{train}}] = \mathbf{W}\mathbf{x} = \mathbf{y}_{\text{inference}}$, meaning the expected output is the same with and without dropout.

Proof: For any component x_j of the input:

$$\mathbb{E}\left[\frac{x_j \cdot m_j}{1 - p}\right] = \frac{1}{1 - p} \cdot \mathbb{E}[x_j \cdot m_j] \tag{10.8}$$

$$= \frac{1}{1 - p} \cdot x_j \cdot \mathbb{E}[m_j] \tag{10.9}$$

$$= \frac{1}{1 - p} \cdot x_j \cdot (1 - p) = x_j \tag{10.10}$$

10.4.1.3 Ensemble Interpretation

Dropout can be understood as training an ensemble of 2^n sub-networks (where n is the number of dropout units) that share parameters. Each forward pass with a different

mask effectively samples a different sub-network. At inference time, all sub-networks are averaged (which, by the linearity of the scaling, is equivalent to using the full network).

More precisely, a network with n dropout units has 2^n possible thinned networks. The dropout training objective minimizes:

$$\mathcal{L}_{\text{dropout}} = \mathbb{E}_{\mathbf{m}\sim\text{Bernoulli}(1-p)} \left[\ell\big(f(\mathbf{x}; \theta \odot \mathbf{m}), \mathbf{y}\big)\right] \tag{10.11}$$

This is equivalent to minimizing a *bagged ensemble* objective, which explains dropout's regularization effect through variance reduction.

10.4.1.4 Dropout as Approximate Bayesian Inference

Gal and Ghahramani [38] showed that dropout in a neural network is mathematically equivalent to approximate variational inference in a deep Gaussian process. Under this interpretation:

- Each dropout mask defines a sample from an approximate posterior $q(\theta)$
- The KL divergence between the approximate and true posterior is minimized implicitly
- Multiple stochastic forward passes at inference provide uncertainty estimates

This provides a principled explanation for why dropout provides not just regularization but also *model uncertainty* estimates—a crucial property for safety-critical applications.

10.4.1.5 Dropout Variants

Table 10.2 – Dropout Variants and Their Properties

Variant	Mechanism	Use Case
Standard Dropout	Randomly zero activations	Fully connected layers
Spatial Dropout	Drop entire feature maps	Convolutional layers
DropConnect	Randomly zero weights	Alternative to dropout
DropBlock	Drop contiguous regions	Convolutional layers
Attention Dropout	Drop attention weights	Transformer models
Feature Dropout	Drop input features	Input regularization

10.4.2 Weight Decay

10.4.2.1 L2 Regularization as MAP Estimation

Weight decay (L2 regularization) adds a penalty on the magnitude of model parameters:

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \frac{\lambda}{2}\|\mathbf{W}\|_F^2 \tag{10.12}$$

This has a elegant interpretation as *maximum a posteriori* (MAP) estimation. Assume a Gaussian prior on the weights $\mathbf{W} \sim \mathcal{N}(0, \sigma_w^2\mathbf{I})$. The MAP objective is:

$$\theta_{\text{MAP}} = \arg\max_{\theta} \log p(\mathcal{D}|\theta) + \log p(\theta) \tag{10.13}$$

$$= \arg\min_{\theta} \underbrace{-\sum_{i=1}^{N} \log p(\mathbf{y}_i|\mathbf{x}_i, \theta)}_{\mathcal{L}} + \underbrace{\frac{1}{2\sigma_w^2}\|\mathbf{W}\|_F^2}_{\lambda/2\cdot\|\mathbf{W}\|_F^2} \tag{10.14}$$

Thus, L2 regularization with coefficient λ is equivalent to MAP estimation with a Gaussian weight prior of variance $\sigma_w^2 = 1/\lambda$.

10.4.2.2 Weight Decay vs. L2 Regularization: AdamW

For adaptive optimizers like Adam, there is a subtle but important difference between L2 regularization and decoupled weight decay. In standard Adam with L2 regularization, the weight update becomes:

$$\mathbf{W}_{t+1} = \mathbf{W}_t - \alpha \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \odot (\nabla\mathcal{L}_t + \lambda\mathbf{W}_t) \tag{10.15}$$

The problem is that the weight decay term $\lambda\mathbf{W}_t$ is scaled by the adaptive learning rate $\frac{1}{\sqrt{\hat{\mathbf{v}}_t}+\epsilon}$, meaning parameters with large historical gradients receive *less* regularization—the opposite of what we want.

Loshchilov and Hutter [39] proposed **AdamW**, which decouples weight decay from the gradient-based update:

$$\mathbf{W}_{t+1} = \mathbf{W}_t - \alpha \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \odot \nabla\mathcal{L}_t - \alpha\lambda\mathbf{W}_t \tag{10.16}$$

$$= (1 - \alpha\lambda)\mathbf{W}_t - \alpha \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \odot \nabla\mathcal{L}_t \tag{10.17}$$

This ensures that weight decay is applied uniformly regardless of the adaptive learning rate, leading to significantly better generalization in practice.

10.4.3 Other Regularization Techniques

10.4.3.1 Gradient Clipping

For training very deep networks or recurrent networks, gradients can become extremely large, causing parameter updates that destabilize training. Gradient clipping

limits the gradient norm:

$$\text{If } \|\mathbf{g}\| > c : \quad \mathbf{g} \leftarrow \frac{c}{\|\mathbf{g}\|}\mathbf{g} \tag{10.18}$$

where c is a threshold (commonly 1.0 or 5.0 for Transformer models). Pascanu et al. [40] showed that gradient clipping is essential for training RNNs on long sequences.

10.4.3.2 Label Smoothing

Label smoothing [20] replaces hard one-hot labels with a softer distribution. Instead of target $\mathbf{y} = [0, \dots, 0, 1, 0, \dots, 0]$, we use:

$$y_k^{\text{smooth}} = (1 - \epsilon) \cdot y_k + \frac{\epsilon}{K} \tag{10.19}$$

where ϵ is the smoothing coefficient (typically 0.1) and K is the number of classes.

Why label smoothing works: It prevents the model from becoming overconfident. A model that outputs $p(\text{correct}) = 0.999$ on training data is likely memorizing rather than learning. Label smoothing penalizes this by requiring the model to distribute some probability mass to incorrect classes, which:

- Improves calibration (predicted probabilities better reflect true accuracy)
- Acts as implicit regularization (Müller et al., 2019 showed it is equivalent to adding KL divergence between the output distribution and a uniform prior)
- Improves BLEU scores in machine translation by 0.5–1.0

10.4.3.3 Stochastic Depth

Huang et al. [26] proposed randomly dropping entire residual blocks during training (analogous to dropout but at the block level). This allows training very deep networks (e.g., 1200 layers) by reducing effective depth during training:

$$\mathbf{y} = \begin{cases} \mathcal{F}_l(\mathbf{x}) + \mathbf{x} & \text{with probability } p_l \\ \mathbf{x} & \text{with probability } 1 - p_l \end{cases} \tag{10.20}$$

where p_l is the survival probability for layer l, often following a linear decay schedule from the first to the last layer.

10.5 Early Stopping

Early stopping halts training when validation performance stops improving. Despite its simplicity, it is one of the most effective and widely used regularization techniques.

10.5.1 Algorithm

1. Monitor validation loss (or a suitable metric) at the end of each epoch
2. Save the model checkpoint whenever the validation metric improves
3. If no improvement is observed for N epochs (the *patience*), stop training
4. Return the best checkpoint (not the final one)

Typical patience values: 10–20 epochs for small models, 5–10 epochs for large models.

10.5.2 Early Stopping as Regularization

Yao et al. showed that early stopping provides regularization effects equivalent to weight decay under certain conditions. The intuition is clear:

1. A model at epoch t has parameters θ_t that lie along the optimization trajectory
2. As training progresses, the model fits increasingly fine-grained patterns in the training data
3. Beyond a certain point, these patterns are noise (overfitting)
4. Early stopping selects parameters from the "sweet spot" where the model has learned signal but not noise

Formally, for gradient descent on a linear model with quadratic loss, early stopping is equivalent to Tikhonov regularization (L2 weight decay), with the number of gradient steps playing the role of the regularization strength.

10.6 Backpropagation

10.6.1 The Computational Graph Perspective

Backpropagation is the algorithm for efficiently computing gradients through a computational graph by applying the chain rule in reverse topological order. While the mathematical principle is straightforward—the chain rule—the computational efficiency comes from reusing intermediate results.

10.6.1.1 Forward Pass

Consider a simple computation: $z = f(g(x))$. The forward pass computes:

$$a = g(x) \tag{10.21}$$
$$z = f(a) \tag{10.22}$$

10.6.1.2 Backward Pass

The backward pass computes gradients in reverse order:

$$\frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial a} = \frac{\partial \mathcal{L}}{\partial z} \cdot f'(a) \tag{10.23}$$
$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial x} = \frac{\partial \mathcal{L}}{\partial a} \cdot g'(x) \tag{10.24}$$

10.6.2 Example: Two-Layer Network

For a two-layer network with loss $\mathcal{L}$:

$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) \tag{10.25}$$
$$\hat{\mathbf{y}} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2 \tag{10.26}$$
$$\mathcal{L} = \ell(\hat{\mathbf{y}}, \mathbf{y}) \tag{10.27}$$

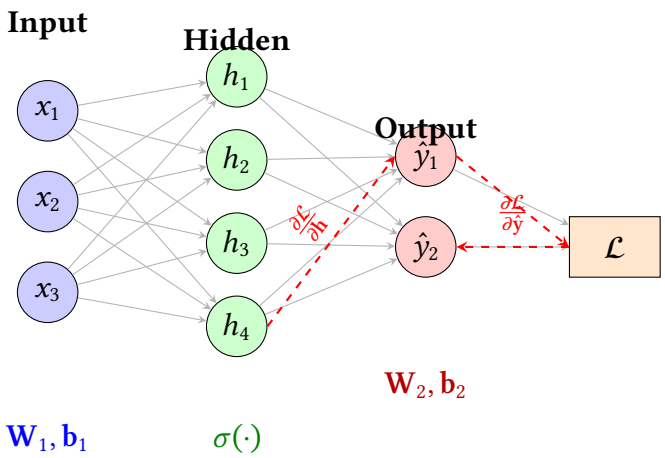


Figure 10.2 – Computational graph for a two-layer network. Solid arrows represent forward computation; dashed red arrows represent backward gradient flow.

The backward pass computes:

$$\frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}} = \nabla_{\hat{\mathbf{y}}} \ell \tag{10.28}$$
$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_2} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}} \mathbf{h}^\top, \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{b}_2} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}} \tag{10.29}$$
$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}} = \mathbf{W}_2^\top \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}} \tag{10.30}$$
$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}} \odot \sigma'(\mathbf{a}) \tag{10.31}$$
$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}} \mathbf{x}^\top, \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{b}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}} \tag{10.32}$$

where $\mathbf{a} = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1$ is the pre-activation.

10.6.3 Automatic Differentiation

Modern deep learning frameworks implement *reverse-mode automatic differentiation* (autodiff), which is distinct from both numerical differentiation and symbolic differentiation:

The key insight of reverse-mode autodiff is that computing *all* partial derivatives of a scalar-valued function with respect to n inputs costs only $\mathcal{O}(1)$ times the cost of the forward pass, regardless of n. This makes it ideal for neural networks, where the loss is scalar but there are millions of parameters.

Table 10.3 – Comparison of Differentiation Methods

Method	Accuracy	Speed	Usage
Numerical	Approximate	Slow (n evaluations)	Debugging only
Symbolic	Exact	Fast but memory-heavy	CAS (Mathematica)
Reverse autodiff	Exact (to machine ϵ)	$\mathcal{O}(1)\times$ forward pass	PyTorch, JAX

10.6.4 Common Training Pitfalls

Table 10.4 – Diagnosing Training Failures

Symptom	Likely Cause	Solution
Loss doesn't decrease	Learning rate too low	Increase LR or check data
Loss is NaN	Learning rate too high	Decrease LR, add grad clipping
Train loss decreases, val loss increases	Overfitting	Add regularization, early stopping
Loss oscillates	Batch size too small	Increase batch size or decrease LR
Very slow convergence	Learning rate too low	Use LR finder
Gradients vanish	Deep network, bad init	Use residual connections, ReLU

10.7 Model Checkpointing

Saving and restoring model state during training is essential for long-running experiments and production deployment.

- **Model architecture:** The network definition (layer sizes, types)
- **Weights:** The learned parameters θ
- **Optimizer state:** Momentum terms $\mathbf{m}$, adaptive learning rates $\mathbf{v}$ (required to resume training exactly)
- **Epoch and step counters:** Training progress
- **Best metric:** The validation metric for model selection
- **Random state:** RNG seed for reproducibility

10.8 Mixed Precision Training

10.8.1 Motivation

Standard training uses 32-bit floating point (FP32) for all computations. Mixed precision training uses 16-bit floating point (FP16) for most operations while maintaining a 32-bit master copy of weights, achieving:

- **2× memory reduction:** Activations and gradients stored in FP16
- **2–3× speedup:** Modern GPUs have 2× FP16 throughput vs FP32 (Tensor Cores on NVIDIA GPUs provide up to 8× for FP16 matmul)
- **No accuracy loss:** When implemented correctly with loss scaling

10.8.2 The Loss Scaling Problem

FP16 has a limited dynamic range ($\approx 6 \times 10^{-5}$ to 65504). Gradients smaller than 2^{-24} underflow to zero. Since gradients in deep networks can span many orders of magnitude, small gradients may be lost entirely.

Solution: Scale the loss by a factor S before backpropagation, then unscale the gradients before the optimizer step:

$$\hat{\mathcal{L}} = S \cdot \mathcal{L} \tag{10.33}$$

$$\mathbf{g}_{\text{FP32}} = \frac{1}{S} \cdot \nabla_{\theta_{\text{FP16}}} \hat{\mathcal{L}} \tag{10.34}$$

Dynamic loss scaling (Micikevicius et al., 2018) automatically adjusts S: increase S by a factor of 2 every N successful steps, and halve S whenever gradients overflow (contain NaN/Inf).

10.8.3 BF16: The Modern Alternative

Bfloat16 (BF16) uses the same 8 exponent bits as FP32 but only 7 mantissa bits (vs. 10 for FP16). This gives BF16 the same dynamic range as FP32 ($\approx 10^{-38}$ to 10^{38}), eliminating the need for loss scaling entirely. BF16 is supported on NVIDIA A100/H100 GPUs and is becoming the default for training large language models.

Table 10.5 – Floating Point Format Comparison

Property	FP32	FP16	BF16	TF32
Total bits	32	16	16	19
Exponent bits	8	5	8	8
Mantissa bits	23	10	7	10
Dynamic range	$\pm 3.4 \times 10^{38}$	± 65504	$\pm 3.4 \times 10^{38}$	$\pm 3.4 \times 10^{38}$
Loss scaling needed	No	Yes	No	No
Hardware support	Universal	Tensor Cores	A100+	A100+

10.9 PyTorch Framework

PyTorch is the most popular framework for deep learning research due to its eager execution model and Pythonic API.

Listing 10.1 – Complete Training Loop in PyTorch

```python
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader

# Model definition
model = nn.Sequential(
    nn.Linear(784, 512),
    nn.BatchNorm1d(512),
    nn.GELU(),
    nn.Dropout(0.2),
    nn.Linear(512, 256),
    nn.BatchNorm1d(256),
    nn.GELU(),
    nn.Dropout(0.1),
    nn.Linear(256, 10)
)

# Loss, optimizer, and scheduler
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
optimizer = optim.AdamW(model.parameters(), lr=1e-3,
                        weight_decay=0.01)
scheduler = optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=100, eta_min=1e-5)

# Training loop with mixed precision
scaler = torch.cuda.amp.GradScaler()
device = torch.device('cuda' if torch.cuda.is_available()
                      else 'cpu')
model = model.to(device)

best_val_loss = float('inf')
patience = 10
patience_counter = 0

for epoch in range(num_epochs):
    # --- Training ---
    model.train()
    train_loss = 0.0
    for data, target in train_loader:
        data, target = data.to(device), target.to(device)

        optimizer.zero_grad(set_to_none=True)

        # Mixed precision forward pass
        with torch.cuda.amp.autocast():
            output = model(data.view(data.size(0), -1))
            loss = criterion(output, target)

        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        scaler.step(optimizer)
        scaler.update()
```

```
        train_loss += loss.item()

    scheduler.step()

    # --- Validation ---
    model.eval()
    val_loss = 0.0
    correct = 0
    with torch.no_grad():
        for data, target in val_loader:
            data, target = data.to(device), target.to(device)
            with torch.cuda.amp.autocast():
                output = model(data.view(data.size(0), -1))
                val_loss += criterion(output, target).item()
            pred = output.argmax(dim=1)
            correct += pred.eq(target).sum().item()

    val_acc = 100. * correct / len(val_loader.dataset)
    avg_val_loss = val_loss / len(val_loader)

    # --- Checkpointing with early stopping ---
    if avg_val_loss < best_val_loss:
        best_val_loss = avg_val_loss
        patience_counter = 0
        torch.save({
            'epoch': epoch,
            'model_state_dict': model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'best_val_loss': best_val_loss,
        }, 'best_model.pt')
    else:
        patience_counter += 1
        if patience_counter >= patience:
            print(f'Early stopping at epoch {epoch}')
            break
```

10.10 CUDA and GPU Training

10.10.1 GPU Programming Model

GPUs excel at data-parallel operations through thousands of lightweight cores organized into streaming multiprocessors (SMs). Key considerations for efficient GPU utilization:

- **Data transfer:** CPU–GPU transfers (PCIe) are orders of magnitude slower than GPU computation. Minimize transfers by keeping data on GPU.
- **Kernel launch overhead:** Many small operations are slower than fewer large operations. Use fused kernels when possible.
- **Memory coalescing:** Adjacent threads should access adjacent memory addresses for optimal bandwidth.

- **Occupancy:** The ratio of active threads to maximum threads affects utilization but is not the only factor.

10.10.2 Distributed Training

For models too large for a single GPU, distributed training strategies are essential:

Table 10.6 – Distributed Training Strategies

Strategy	Mechanism	Memory	Speed
Data Parallel	Replicate model, split data	Linear in model	Near-linear scale
Model Parallel	Split model across GPUs	Reduced	Communication-bound
Pipeline Parallel	Split layers across GPUs	Reduced	Bubble overhead
Tensor Parallel	Split tensors within a layer	Reduced	All-reduce cost
FSDP	Shard parameters + gradients	Reduced	Communication cost

10.10.3 Gradient Accumulation

When the desired effective batch size exceeds GPU memory, gradient accumulation simulates larger batches by accumulating gradients over multiple micro-batches before each optimizer step:

$$\mathbf{g} = \frac{1}{K} \sum_{k=1}^{K} \nabla_\theta \mathcal{L}(\mathcal{B}_k) \tag{10.35}$$

where K is the number of accumulation steps and the effective batch size is $B_{\text{eff}} = B \times K$.

Listing 10.2 – Gradient Accumulation in PyTorch

```
accumulation_steps = 4  # Effective batch = 32 * 4 = 128
optimizer.zero_grad()

for i, (data, target) in enumerate(train_loader):
    output = model(data)
    loss = criterion(output, target) / accumulation_steps
    loss.backward()  # Accumulate gradients

    if (i + 1) % accumulation_steps == 0:
        optimizer.step()
        optimizer.zero_grad()
```

10.11 Training Diagnostics

Understanding what happens during training requires systematic monitoring of several indicators.

10.11.1 Loss Curves

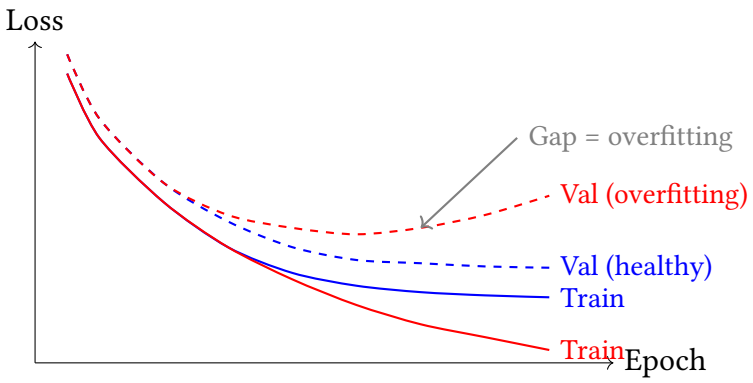


Figure 10.3 – Training loss curves: healthy training (blue) vs. overfitting (red). Solid lines: training loss. Dashed lines: validation loss.

10.11.2 Gradient Statistics

Monitoring gradient statistics helps diagnose training issues:

- **Gradient norm:** Should be stable. Spikes indicate instability; vanishing norm indicates dead neurons or saturated activations.
- **Gradient distribution:** Should be approximately Gaussian. Heavy tails may indicate outlier samples.
- **Update-to-weight ratio:** The ratio $\|\alpha \mathbf{g}\|/\|\mathbf{W}\|$ should be around 10^{-3}. Much larger values indicate instability; much smaller values indicate stagnation.
- **Activation statistics:** Mean and variance of layer activations should remain stable across training. BatchNorm helps with this.

10.12 Chapter Summary

1. Training involves iterative minibatch optimization over data, where gradient noise provides implicit regularization
2. The learning rate is the most critical hyperparameter; the LR range test provides a principled way to select it
3. Warmup is essential for large models with adaptive optimizers to stabilize early training
4. Dropout provides regularization through ensemble averaging and can be interpreted as approximate Bayesian inference
5. AdamW decouples weight decay from adaptive learning rates, leading to better generalization than L2 regularization with Adam
6. Early stopping prevents overfitting and is equivalent to regularization under certain conditions
7. Mixed precision training (FP16/BF16) provides 2–3× speedup with no accuracy loss when properly implemented

8. Gradient accumulation enables training with effective batch sizes larger than GPU memory
9. Systematic monitoring of loss curves, gradient statistics, and validation metrics is essential for diagnosing training issues

10.13 Further Reading

- Goodfellow et al., "Deep Learning" [6] — Chapter 7 (Regularization) and Chapter 8 (Optimization for Training Deep Models)
- [37] — Dropout: A Simple Way to Prevent Neural Networks from Overfitting
- [38] — Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning
- [39] — Decoupled Weight Decay Regularization (AdamW)
- [35] — Cyclical Learning Rates for Training Neural Networks (LR range test)
- [34] — On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima
- **goyal2017accurate** — Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour
- [20] — Rethinking the Inception Architecture for Computer Vision (Label Smoothing)
- [26] — Deep Networks with Stochastic Depth (Stochastic Depth)
- [40] — On the Difficulty of Training Recurrent Neural Networks (Gradient Clipping)
- Micikevicius et al., "Mixed Precision Training" (ICLR 2018)
- Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models" (SC 2020)

Chapter 11

Dataset Processing

11.1 Introduction

The quality of a machine learning model is fundamentally limited by the quality of its training data. As the saying goes, "garbage in, garbage out." This chapter covers the end-to-end pipeline of preparing data for neural network training: collection, annotation, splitting, augmentation, loading, normalization, and the increasingly important considerations of data quality and versioning.

While the techniques discussed apply broadly, we pay particular attention to the distinct requirements of different data modalities—images, text, and audio—since each presents unique processing challenges.

11.2 Data Collection

The foundation of any machine learning system is quality data. The dataset determines the ceiling of what your model can achieve.

11.2.1 Data Sources

Data can be obtained from multiple sources, each with its own quality and cost tradeoffs:

- **Public datasets:** Curated benchmark datasets (ImageNet, COCO, GLUE, SQuAD) provide standardized evaluation but may not match the target distribution.
- **Web scraping:** Automated collection from websites provides large volumes of data at low cost, but requires careful deduplication and quality filtering.
- **Sensors:** Cameras, microphones, medical imaging devices, accelerometers produce rich data but require careful calibration and preprocessing.
- **Human annotation:** Expert labeling provides high-quality ground truth but is expensive and slow. Crowdsourcing platforms (Amazon Mechanical Turk, Scale AI) trade annotation quality for scale and speed.
- **Synthetic data:** Data generated from simulations or generative models can augment limited real data, particularly valuable in medical imaging and robotics.

11.2.2 Data Quality Criteria

High-quality datasets share several properties:

1. **Correctness:** Labels accurately reflect ground truth. Errors introduce bias and limit model performance ceiling.
2. **Consistency:** Annotation guidelines are unambiguous, and annotators apply them uniformly.

3. **Representativeness:** The dataset distribution matches the target deployment domain. A model trained on daylight images will fail on nighttime scenes.
4. **Balancing:** Class frequencies and difficulty levels are appropriate for the task. Severe imbalance requires special handling.
5. **Deduplication:** Near-duplicate samples inflate effective dataset size and cause train-test leakage.

11.3 Data Annotation

Labels enable supervised learning. The annotation strategy depends on the task type.

11.3.1 Annotation Types

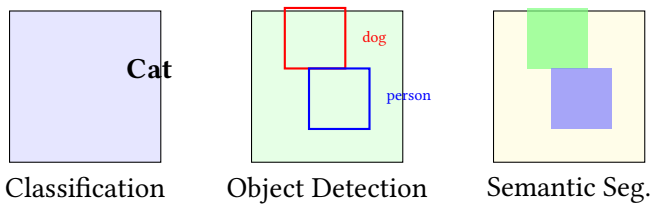


Figure 11.1 – Annotation granularity: from image-level labels to pixel-level segmentation masks.

- **Classification:** A single label per image (e.g., cat vs. dog). Simplest annotation task, prone to ambiguous cases.
- **Multi-label classification:** Multiple labels per image (e.g., "beach, sunset, people"). Each label is an independent binary decision.
- **Object detection:** Bounding boxes with class labels. Each box specifies a region and its class.
- **Semantic segmentation:** Pixel-wise class labels. Every pixel is assigned a class (or "background").
- **Instance segmentation:** Distinguishing between individual objects of the same class (e.g., each person is a separate instance).
- **Keypoint detection:** Points of interest (facial landmarks, human body joints, vehicle detection points).

11.3.2 Annotation Tools

- **LabelImg:** Open-source tool for bounding box annotation. Fast and lightweight.
- **CVAT:** Computer Vision Annotation Tool supports video, image, and text annotation with collaborative features.
- **Supervisely:** Full-featured platform with AI-assisted annotation (pre-labeling with existing models).

- **Amazon Mechanical Turk:** Crowdsourced human annotation at scale. Quality control via consensus/gold questions.
- **Label Studio:** Highly configurable open-source annotation tool supporting multiple data types.

11.4 Data Splitting

Proper data splitting is crucial for obtaining an unbiased estimate of model performance.

11.4.1 Basic Splits

- **Training set:** The samples used to update model parameters. Typically 70–90% of data.
- **Validation set:** Used for hyperparameter tuning and model selection. Never used for training.
- **Test set:** Used for final evaluation. Must never influence any training decisions.

Typical ratios:

- Small datasets (<10K): 70/10/20 or 80/10/10 (train/val/test)
- Medium datasets (10K–1M): 80/10/10 or 85/5/10
- Large datasets (>1M): 98/1/1 may suffice, but ensure test set is large enough for reliable estimates

Note 11.1 (Test Set Leakage)**.** Never use the test set for any decisions during training. This includes selecting hyperparameters, choosing architectures, or early stopping thresholds. Doing so leads to overfitting to the test set and unreliable performance estimates. The test set should be "locked away" until final evaluation.

11.4.2 Stratified Splitting

When class distributions are imbalanced, random splitting may produce unrepresentative splits. Stratified splitting maintains the class proportions in each split:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y
)
```

For multi-label classification, stratification is more complex. One approach is to treat each label combination as a separate class, though this can lead to rare combinations. Alternatively, iterate through labels and balance each.

11.4.3 Cross-Validation

For robust evaluation with limited data, K-fold cross-validation provides more reliable estimates:

Algorithm 3 K-Fold Cross-Validation

1. Shuffle data and split into K equal folds
2. For $i = 1, 2, \ldots, K$:
   (a) Train on folds $\{1, 2, \ldots, K\} \setminus \{i\}$
   (b) Validate on fold i
3. Report the average and standard deviation of the validation metrics

Common values: $K = 5$ (for moderate data) or $K = 10$ (for small data, larger variance reduction). Stratified K-fold maintains class proportions in each fold.

11.5 Data Augmentation

Data augmentation artificially increases dataset size by applying random transformations to existing samples, improving generalization without requiring additional data collection.

11.5.1 Image Augmentations

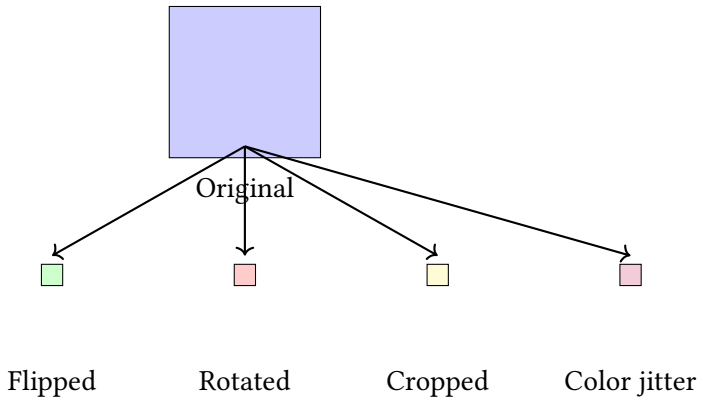


Figure 11.2 – Examples of image augmentations: horizontal flip, rotation, random crop, and color jitter.

Common geometric augmentations:

- **Rotation:** Random angle rotation (e.g., $\pm 15^\circ$). Should be within a range that preserves the label.
- **Flip:** Horizontal flip (right-left mirror). Safe for most classification tasks but not for text or asymmetric objects.

- **Random Crop:** Random crop to a smaller region, then resize to original size. Enforces scale invariance.
- **Scale:** Random scaling (zoom in/out). Same effect as random crop with different crop ratios.
- **Translation:** Random shifts in x and y directions. Equivalent to asymmetric cropping.

Color augmentations:

- **Color jitter:** Random adjustments to brightness, contrast, saturation, and hue.
- **Gaussian noise:** Injection of pixel-wise Gaussian noise for robustness.
- **Cutout/Random Erasing:** Randomly masking rectangular regions. Forces the model to use distributed features.
- **Grayscale/RGB permutation:** For multi-channel images.

Mixup and CutMix augmentations blend samples together:

11.5.2 Mixup: Convex Combinations of Samples

Mixup [41] creates virtual training examples by linearly interpolating between pairs of samples:

$$\tilde{\mathbf{x}} = \lambda \mathbf{x}_i + (1-\lambda)\mathbf{x}_j \tag{11.1}$$

$$\tilde{\mathbf{y}} = \lambda \mathbf{y}_i + (1-\lambda)\mathbf{y}_j \tag{11.2}$$

where $\lambda \sim \text{Beta}(\alpha, \alpha)$ for $\alpha \in (0, \infty)$. The mixing coefficient λ controls the relative weight of each sample.

Why Mixup works:

1. **Regularization:** The model sees smoothed decision boundaries, preventing overconfidence
2. **Data cleaning:** Mixing noisy labels with clean ones dilutes label noise
3. **Better calibration:** Predictions on mixed inputs encourage softer targets
4. **Label smoothing effect:** For classification, $\tilde{y}$ is a soft label even when y_i, y_j are one-hot

For a K-class classification problem with one-hot labels $\mathbf{y}_i, \mathbf{y}_j \in \{0,1\}^K$, the mixed label $\tilde{\mathbf{y}}$ is a probability distribution over classes. The Mixup loss becomes:

$$\mathcal{L}_{\text{mixup}} = -\sum_{k=1}^{K} \tilde{y}_k \log \hat{y}_k = -\lambda \sum_k y_{i,k} \log \hat{y}_k - (1-\lambda) \sum_k y_{j,k} \log \hat{y}_k \tag{11.3}$$

This is equivalent to minimizing the cross-entropy between the prediction and a mixture of the two one-hot labels.

11.5.3 CutMix: Pasting Patches

CutMix [42] pastes a rectangular region from one image into another instead of blending:

$$\tilde{\mathbf{x}} = \mathbf{x}_i \odot \mathbf{M} + \mathbf{x}_j \odot (1 - \mathbf{M}) \tag{11.4}$$

$$\tilde{\mathbf{y}} = \lambda \mathbf{y}_i + (1 - \lambda)\mathbf{y}_j \tag{11.5}$$

where $\mathbf{M} \in \{0, 1\}^{H \times W}$ is a binary mask and $\lambda = \frac{\text{area}(\mathbf{M})}{HW}$.

CutMix vs. Mixup: CutMix preserves the per-pixel nature of images (no blurring at the boundary) while still providing the regularization benefits of Mixup. The rectangular mask provides a clear contrast between regions, which can help models learn localized features.

Listing 11.1 – Mixup and CutMix in PyTorch

```
import torch
import numpy as np

def mixup_data(x, y, alpha=1.0):
    """Returns mixed inputs, pairs of targets, and lambda."""
    if alpha > 0:
        lam = np.random.beta(alpha, alpha)
    else:
        lam = 1

    batch_size = x.size(0)
    index = torch.randperm(batch_size).to(x.device)

    mixed_x = lam * x + (1 - lam) * x[index, :]
    y_a, y_b = y, y[index]
    return mixed_x, y_a, y_b, lam

def mixup_criterion(criterion, pred, y_a, y_b, lam):
    return lam * criterion(pred, y_a) + (1 - lam) * criterion(pred, y_b)

def cutmix_data(x, y, alpha=1.0):
    """CutMix: Cuts and pastes patches between images."""
    if alpha > 0:
        lam = np.random.beta(alpha, alpha)
    else:
        lam = 1

    batch_size = x.size(0)
    index = torch.randperm(batch_size).to(x.device)

    # Get bounding box
    W = x.size(3); H = x.size(2)
    cut_rat = np.sqrt(1. - lam)
    cut_w = int(W * cut_rat); cut_h = int(H * cut_rat)

    cx = np.random.randint(W)
    cy = np.random.randint(H)
    bbx1 = np.clip(cx - cut_w // 2, 0, W)
```

```
    bby1 = np.clip(cy - cut_h // 2, 0, H)
    bbx2 = np.clip(cx + cut_w // 2, 0, W)
    bby2 = np.clip(cy + cut_h // 2, 0, H)

    x[:, :, bby1:bby2, bbx1:bbx2] = x[index, :, bby1:bby2, bbx1:bbx2]
    lam = 1 - ((bbx2 - bbx1) * (bby2 - bby1) / (W * H))

    y_a, y_b = y, y[index]
    return x, y_a, y_b, lam
```

11.5.4 Advanced Augmentation Policies

Rather than manually designing augmentations, automated methods search for optimal augmentation policies:

- **AutoAugment** [43]: Learned policies from CIFAR-10/CIFAR-100. Discovered policies transfer well to other datasets but are computationally expensive.
- **RandAugment** [44]: Simplified search space with just two parameters (magnitude, number of operations). Nearly matches AutoAugment with much less computation.
- **TrivialAugment Wide muller2023 torchvision**: No search needed. Applies a randomly selected augmentation at a random magnitude. Surprisingly effective and simple.
- **Test-Time Augmentation (TTA):** Apply augmentations at inference and average predictions. Provides free accuracy improvement (typically 1–2%) at the cost of $K\times$ inference time.

11.6 Text Data Processing

Text data requires fundamentally different preprocessing than images, centered around the concept of tokenization.

11.6.1 Tokenization

Tokenization converts raw text strings into sequences of discrete tokens (word IDs or subword IDs) that the model can process.

11.6.2 Byte Pair Encoding (BPE)

BPE [45] is the most widely used subword tokenization algorithm in modern LLMs. It builds the vocabulary by iteratively merging the most frequent adjacent pairs of bytes/characters:

1. Start with a vocabulary of all individual characters plus an end-of-word token
2. Count all adjacent pairs in the corpus
3. Merge the most frequent pair into a single token
4. Repeat for N merge operations (where N is a hyperparameter)

Table 11.1 – Tokenization Strategies

Strategy	Example	Pros/Cons
Word-level	"hello world" → [12, 45]	Simple; large vocab, OOV issues
Character-level	"hello" → [104, 101, 108, 108, 111]	Small vocab; long sequences, no semantics
BPE	"unhappiness" → ["un", "happy", "ness"]	Good balance; learned from data
WordPiece	BERT tokenizer	Similar to BPE; masked LM training
SentencePiece	Unified tokenization	Language-agnostic; handles any text

The vocabulary size is a key hyperparameter: larger vocabularies represent more words as single tokens (faster inference) but require more embeddings and have higher memory footprint.

For a vocabulary of size V, BPE requires $\mathcal{O}(V)$ space. Modern LLMs typically use $V = 30{,}000–100{,}000$ tokens.

11.7 Audio Data Processing

Audio data is typically processed as spectrograms or mel-spectrograms.

11.7.1 Spectrogram Computation

The short-time Fourier transform (STFT) converts a time-domain signal $x[t]$ into a time-frequency representation:

$$X[m,k] = \sum_{n=0}^{N-1} x[n]w[n]e^{-j2\pi kn/N} \tag{11.6}$$

where $w[n]$ is a window function (Hann or Hamming) and N is the FFT size.

The **mel-spectrogram** maps frequencies to the mel scale, which approximates human auditory perception:

$$f_{\text{mel}} = 2595 \log_{10}\left(1 + \frac{f_{\text{Hz}}}{700}\right) \tag{11.7}$$

11.8 Data Normalization

Normalization standardizes input features, which stabilizes training by ensuring all inputs are on similar scales.

11.8.1 Image Normalization

For RGB images, pixel values $x \in [0, 255]$ are typically normalized to $[0, 1]$ or standardized to zero mean and unit variance:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma} \tag{11.8}$$

ImageNet normalization parameters:

- Mean: $[\,0.485,\ 0.456,\ 0.406\,]$
- Std: $[\,0.229,\ 0.224,\ 0.225\,]$

These values are computed from the ImageNet training set. For a custom dataset, compute dataset-specific statistics:

```
loader = DataLoader(dataset, batch_size=32)
mean = torch.zeros(3); std = torch.zeros(3)
for images, _ in loader:
    mean += images.mean(dim=[0, 2, 3])
    std += images.std(dim=[0, 2, 3])
mean /= len(loader.dataset)
std /= len(loader.dataset)
```

Note: When using pretrained models, always use the statistics the model was trained with, even if they are not optimal for your data.

11.9 Data Loading

Efficient data loading prevents GPU starvation and maximizes training throughput.

Listing 11.2 – Efficient Data Loading in PyTorch

```
from torch.utils.data import DataLoader, Dataset

class ImageDataset(Dataset):
    def __init__(self, data, labels, transform=None):
        self.data = data
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        image = self.data[idx]
        label = self.labels[idx]
        if self.transform:
            image = self.transform(image)
        return image, label

train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True,         # Important for SGD convergence
```

```
    num_workers=4,        # Parallel data loading
    pin_memory=True,      # Faster GPU transfer (CUDA pinning)
    drop_last=True,       # Consistent batch sizes
    persistent_workers=True  # Keep workers alive between epochs
)

# Prefetch next epoch while current epoch trains
val_loader = DataLoader(
    val_dataset,
    batch_size=64,
    shuffle=False,         # No need to shuffle validation
    num_workers=4,
    pin_memory=True
)
```

Properties 11.1 (DataLoader Optimization).

- **num_workers**: More workers = faster loading, but too many causes CPU-GPU contention. A good starting point is 4–8.
- **pin_memory**: Pins CPU memory so transfers to GPU are faster (avoids an extra memory copy).
- **drop_last**: Drops the last incomplete batch to avoid varying batch sizes (especially important for batch normalization).
- **persistent_workers**: Keeps worker processes alive between epochs, reducing per-epoch startup overhead.

11.10 Handling Data Imbalance

Class imbalance is ubiquitous in real-world datasets and can severely degrade model performance if not addressed.

11.10.1 Imbalance Ratios

Common imbalance ratios:

- Mild (1:10 to 1:100): Standard medical imaging, anomaly detection
- Severe (1:100 to 1:1000): Fraud detection, rare disease diagnosis
- Extreme (<1:1000): Scientific applications (particle physics, astronomy)

11.10.2 Techniques for Imbalanced Data

1. **Weighted loss function:** Assign higher loss weights to minority classes:

$$\mathcal{L}_{\text{weighted}} = -\sum_{k=1}^{K} w_k \cdot y_k \log \hat{y}_k \tag{11.9}$$

   where $w_k = N/(K \cdot N_k)$ and N_k is the number of samples in class k.

2. **Oversampling minority classes:** Replicate minority samples or generate synthetic samples (SMOTE for tabular data).

3. **Undersampling majority classes:** Randomly remove samples from majority classes to balance the dataset.
4. **Focal Loss** (Lin et al., 2017): Down-weights well-classified examples, focusing training on hard examples:

$$\mathcal{L}_{\text{focal}} = -(1 - p_t)^{\gamma} \log p_t \tag{11.10}$$

where p_t is the model's predicted probability for the true class and $\gamma \geq 0$ controls the strength of down-weighting. See Chapter 3 for a detailed derivation.

11.11 Data Quality and Versioning

As datasets grow in importance and complexity, systematic data management becomes essential.

11.11.1 Data Versioning

Data versioning tracks changes to datasets over time, enabling:

- Reproducibility: Training with the exact same data at different points in time
- Lineage tracking: Understanding which model was trained on which version of the data
- Rollback: Reverting to previous data versions if issues are discovered
- Experiment comparison: Testing hypotheses about data quality

Tools like **DVC** (Data Version Control) extend Git to handle large files and ML pipelines:

```
dvc init
dvc add data/
git add data.dvc
git commit -m "Add dataset v1"
# Later, update and version:
dvc add data/
git add data.dvc
git commit -m "Update to dataset v2"
```

DVC stores actual data in remote storage (S3, GCS, Azure Blob) while Git tracks metadata and hashes, enabling efficient collaboration.

11.11.2 Data Provenance

Understanding data provenance answers: Where did this sample come from? Who labeled it? When was it added?

Key provenance metadata:

- Source URL or collection method
- Timestamp of collection/annotation

- Annotator ID and their qualifications
- Annotation quality scores (if using consensus annotations)
- Processing pipeline version (augmentations, filters applied)

This metadata is crucial for debugging model failures, auditing for bias, and satisfying regulatory requirements in high-stakes domains (healthcare, finance, legal).

11.12 Evaluation Metrics

While Chapter 3 covers loss functions, this section addresses task-specific evaluation metrics.

11.12.1 Classification Metrics

- **Accuracy:** Fraction of correct predictions. Simple but misleading for imbalanced data.
- **Precision, Recall, F1:** For binary classification:

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{(of predicted positives, how many are correct?)} \tag{11.11}$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad \text{(of actual positives, how many are found?)} \tag{11.12}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{11.13}$$

- **Macro/Micro F1:** Macro-averages F1 per class then averages; micro-averages TP/FP/FN across classes then computes F1.
- **AUC-ROC:** Area under the ROC curve. Threshold-independent; measures ranking quality.

11.12.2 Segmentation Metrics

- **Intersection over Union (IoU):** $\text{IoU} = \frac{|A \cap B|}{|A \cup B|}$
- **Dice Coefficient:** $\text{Dice} = \frac{2|A \cap B|}{|A| + |B|} = \frac{2 \cdot \text{IoU}}{1 + \text{IoU}}$
- **Pixel Accuracy:** Fraction of correctly classified pixels.

Note: IoU and Dice are related: $\text{Dice} = \frac{2 \cdot \text{IoU}}{1 + \text{IoU}}$, so optimizing one approximately optimizes the other.

11.13 Chapter Summary

1. Data quality is the foundation of model performance; even the best architectures cannot overcome poor data
2. Proper train/val/test splitting prevents leakage and provides unbiased performance estimates
3. Cross-validation (K-fold) provides robust estimates when data is limited

4. Data augmentation artificially expands the effective dataset through random transformations
5. Mixup and CutMix create virtual training samples through convex combinations or patch pasting, providing strong regularization
6. Text data requires tokenization (BPE, WordPiece) before model input
7. Normalization stabilizes training by standardizing input scales
8. Efficient data loading with multiple workers and pinned memory maximizes GPU utilization
9. Class imbalance requires special handling through weighted losses or focal loss
10. Data versioning and provenance tracking are essential for reproducible ML research and production systems
11. Evaluation metrics should match the downstream task requirements; accuracy alone is insufficient for imbalanced or multi-class problems

11.14 Further Reading

- [6] — Chapter on Deep Learning, Section on Data Processing
- [41] — mixup: Beyond Empirical Risk Minimization
- [42] — CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features
- [43] — AutoAugment: Learning Augmentation Policies from Data
- [44] — RandAugment: Practical Automated Data Augmentation with a Reduced Search Space
- [45] — Neural Machine Translation of Rare Words with Subword Units
- [19] — Focal Loss for Dense Object Detection
- Chollet, "Deep Learning with Python" — Chapter 4 covers data preprocessing pipelines
- DVC Documentation: https://dvc.org/ — Data Version Control for ML projects

Chapter 12

Transformer Architecture

12.1 Introduction

The Transformer architecture, introduced by Vaswani et al. in 2017, fundamentally changed the landscape of sequence modeling and representation learning. By replacing recurrence with self-attention mechanisms, Transformers enable parallel computation across entire sequences, dramatically improving training efficiency and enabling the scaling to billions of parameters.

This chapter provides a comprehensive treatment of the Transformer architecture, from the fundamental attention mechanism to modern large language models.

12.2 The Attention Mechanism

12.2.1 Background: RNN Limitations

Before Transformers, sequence models relied on Recurrent Neural Networks (RNNs) and their variants (LSTM, GRU). While effective, RNNs suffer from:

- **Sequential computation:** Cannot parallelize across time steps
- **Long-range dependencies:** Gradient propagation through many steps causes vanishing/exploding gradients
- **Computational inefficiency:** Each time step requires loading the hidden state from memory

12.2.2 Scaled Dot-Product Attention

The fundamental building block is the scaled dot-product attention:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \tag{12.1}$$

where:

- $\mathbf{Q} \in \mathbb{R}^{n \times d_k}$: Query matrix
- $\mathbf{K} \in \mathbb{R}^{m \times d_k}$: Key matrix
- $\mathbf{V} \in \mathbb{R}^{m \times d_v}$: Value matrix
- d_k: Dimension of keys (also query dimension)

Note 12.1 (Why Scaled?)**.** The scaling factor $\sqrt{d_k}$ prevents the dot products from growing large in magnitude, which would push the softmax into regions with extremely small gradients. When d_k is large, the dot products tend to be large, causing softmax to saturate (output close to one-hot).

12.2.3 Self-Attention

Self-attention is attention applied to a sequence where queries, keys, and values all come from the same source:

$$\mathbf{X}_{\text{out}} = \text{Attention}(\mathbf{X}\mathbf{W}_Q, \mathbf{X}\mathbf{W}_K, \mathbf{X}\mathbf{W}_V) \tag{12.2}$$

where $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ are learned projection matrices.

Key properties:

- **All-to-all connections:** Each position can attend to all other positions
- **Constant path length:** Information flows in $O(1)$ steps regardless of distance
- **Computational complexity:** $O(n^2 \cdot d)$ for sequence length n and dimension d

12.3 Multi-Head Attention

Rather than performing a single attention function, we project the queries, keys, and values h times with learned linear projections:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}\left(\text{head}_1, \ldots, \text{head}_h\right) \mathbf{W}^O \tag{12.3}$$

where each head is:

$$\text{head}_i = \text{Attention}\left(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V\right) \tag{12.4}$$

Typical configuration: $h = 8$ heads, $d_k = d_v = d_{\text{model}}/h$.

Properties 12.1 (Multi-Head Attention Benefits).

- Each head can attend to different aspects of the representation
- Different heads may specialize: positional, syntactic, semantic information
- The model can jointly attend to information from different representation subspaces

12.4 Transformer Architecture

12.4.1 Feed-Forward Networks

Each layer includes a position-wise feed-forward network:

$$\text{FFN}(\mathbf{x}) = \mathbf{W}_2\, \sigma\left(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2 \tag{12.5}$$

Typically implemented as two linear transformations with a ReLU activation. The inner dimension is usually $4 \times d_{\text{model}}$.

12.4.2 Layer Normalization

Layer normalization stabilizes training by normalizing activations:

$$\mathbf{y} = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta \tag{12.6}$$

where $\mu = \frac{1}{d}\sum_{i=1}^{d} x_i$ and $\sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$.

Two variants exist:

- **Post-LN (Original):** LayerNorm after residual connections
- **Pre-LN:** LayerNorm before attention/FFN (more stable for deep networks)

12.4.3 Residual Connections

Each sub-layer has a residual connection:

$$\mathbf{x}_{\text{out}} = \mathbf{x} + \text{SubLayer}(\mathbf{x}) \tag{12.7}$$

This enables gradient flow through skip connections and allows the network to learn identity mappings when beneficial.

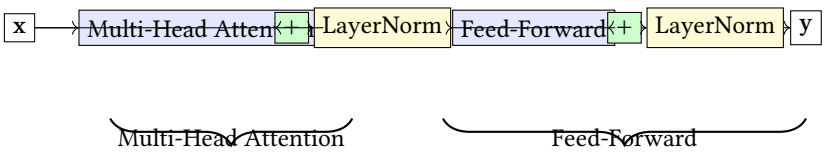


Figure 12.1 – Transformer layer with residual connections and layer normalization.

12.5 Positional Encoding

Since self-attention is permutation-invariant (invariant to input order), we must inject positional information:

$$\text{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{12.8}$$

$$\text{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{12.9}$$

where pos is the position and i is the dimension.

Properties 12.2 (Positional Encoding Properties).

- **Linear relationships:** $\text{PE}(pos+k)$ can be represented as a linear function of $\text{PE}(pos)$
- **Generalization:** Can encode positions beyond training sequence length
- **Relative positions:** Rotary Position Embedding (RoPE) encodes relative positions more directly

Alternative: Rotary Position Embedding (RoPE) used in LLaMA and GPT-NeoX, which encodes relative position directly into the attention computation.

12.6 Encoder-Decoder Architecture

The original Transformer uses an encoder-decoder structure:

- **Encoder:** N identical layers processing the input sequence

- **Decoder:** N identical layers with cross-attention to encoder outputs
- **Cross-attention:** Queries from decoder attend to keys/values from encoder

Listing 12.1 – PyTorch Multi-Head Attention Implementation

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0

        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads

        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def split_heads(self, x, batch_size):
        """Split the last dimension into num_heads."""
        x = x.view(batch_size, -1, self.num_heads, self.d_k)
        return x.permute(0, 2, 1, 3)  # (batch, heads, seq, d_k)

    def forward(self, query, key, value, mask=None):
        batch_size = query.size(0)

        # Linear projections
        Q = self.W_q(query)
        K = self.W_k(key)
        V = self.W_v(value)

        # Split heads
        Q = self.split_heads(Q, batch_size)
        K = self.split_heads(K, batch_size)
        V = self.split_heads(V, batch_size)

        # Scaled dot-product attention
        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)

        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)

        attention_weights = F.softmax(scores, dim=-1)
        attention_output = torch.matmul(attention_weights, V)

        # Concatenate heads
        attention_output = attention_output.permute(0, 2, 1, 3).contiguous()
        attention_output = attention_output.view(batch_size, -1, self.d_model)
```

```

        # Final linear projection
        output = self.W_o(attention_output)

        return output, attention_weights
```

12.7 Vision Transformer (ViT)

Vision Transformer applies the Transformer architecture to image classification by treating image patches as tokens:

1. Split image into 16×16 patches
2. Flatten each patch and project to d_{model} dimensions
3. Add [CLS] token (for classification) and positional embeddings
4. Process through standard Transformer encoder
5. Use [CLS] token output for classification

Computational complexity: $O(n^2)$ where $n = \frac{H \times W}{P^2}$ is the number of patches.

12.8 Large Language Models

12.8.1 GPT Architecture

GPT (Generative Pre-trained Transformer) uses a decoder-only architecture:

- Causal (masked) self-attention to prevent attending to future tokens
- Language modeling objective: predict next token
- Pre-training on large text corpus, then fine-tuning

GPT-3 scaled to 175 billion parameters, demonstrating emergent capabilities.

12.8.2 BERT Architecture

BERT uses an encoder-only architecture with bidirectional attention:

- Masked Language Modeling (MLM): predict masked tokens
- Next Sentence Prediction (NSP): predict if one sentence follows another
- Enables deep bidirectional representations

12.8.3 Modern LLMs

Modern large language models incorporate several innovations:

Key architectural differences:

- **RoPE (Rotary Position Embedding):** Encodes position in rotation matrix, better for relative positions

Table 12.1 – Comparison of Modern LLM Architectures

Model	Architecture	Position Encoding	Key Features
GPT-2	Decoder-only	Learned	Unigram tokenization
GPT-3	Decoder-only	Learned	In-context learning
LLaMA	Decoder-only	RoPE	Efficient architecture
ChatGLM	Prefix-LM	RoPE	Bilingual pre-training
Mistral	Decoder-only	RoPE	Grouped-query attention

- **Grouped-Query Attention (GQA):** Multiple query heads share key/value heads, reduces memory
- **SwiGLU activation:** Used in LLaMA, better than GeGLU
- **Flash Attention:** IO-aware exact attention algorithm, reduces memory from $O(n^2)$ to $O(n)$

12.9 Efficiency Considerations

12.9.1 Attention Complexity

Standard attention has $O(n^2)$ complexity in sequence length, which becomes prohibitive for long sequences:

- **Flash Attention:** Approximate attention with IO-awareness, 2-4x speedup
- **Sparse Attention:** Attend only to a subset of positions (Longformer, BigBird)
- **Linear Attention:** Kernel-based approximation with $O(n)$ complexity (Performer)
- **Low-rank attention:** Decompose attention matrix (Linformer)

12.9.2 Memory Optimization

For large models, memory becomes critical:

- **Gradient checkpointing:** Trade compute for memory
- **Mixed precision:** FP16/BF16 training reduces memory by 2x
- **Quantization:** INT8/INT4 for inference
- **Paged attention:** Memory paging for KV cache (vLLM)

12.10 Chapter Summary

1. Self-attention enables parallel computation and captures long-range dependencies with $O(1)$ path length

2. Scaled dot-product attention with $\sqrt{d_k}$ scaling prevents gradient saturation
3. Multi-head attention allows attending to different representation subspaces
4. Residual connections and layer normalization enable stable training of deep networks
5. Positional encoding injects order information since attention is permutation-invariant
6. Vision Transformer (ViT) applies the Transformer architecture to images via patch embeddings
7. Modern LLMs (GPT, BERT, LLaMA) scale Transformers to billions of parameters
8. Efficiency techniques (Flash Attention, sparse attention, quantization) enable practical deployment of large models

12.11 Further Reading

- [28] — Attention Is All You Need (Original Transformer paper)
- [46] — An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale (ViT)
- [47] — Language Models are Unsupervised Multitask Learners (GPT-2)
- [48] — Language Models are Few-Shot Learners (GPT-3)
- [49] — BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding
- [50] — LLaMA: Open and Efficient Foundation Language Models
- [29] — RoFormer: Enhanced Transformer with Rotary Position Embedding
- [51] — FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness
- [52] — Training Compute-Optimal Large Language Models (Chinchilla scaling laws)
- [53] — Emergent Abilities of Large Language Models

Chapter 13

Modern Training Techniques

13.1 Introduction

Training modern deep learning systems involves far more than gradient descent. This chapter covers advanced training techniques that enable training on massive datasets, transfer knowledge across tasks, and achieve state-of-the-art performance. These techniques are essential for anyone working with contemporary neural networks.

13.2 Pre-training and Fine-tuning

The dominant paradigm for building powerful models is pre-training followed by fine-tuning.

13.2.1 Pre-training Objectives

Pre-training learns general representations from large unlabeled data:

- **Language Modeling (CLM):** Predict the next token given previous tokens
- **Masked Language Modeling (MLM):** Predict randomly masked tokens (BERT-style)
- **Denoising Autoencoder:** Reconstruct corrupted input
- **Next Sentence Prediction (NSP):** Predict sentence ordering

For vision:

- **Contrastive Learning:** Maximize similarity between positive pairs
- **Masked Autoencoding (MAE):** Reconstruct masked patches
- **Self-distillation:** Student learns from teacher

13.2.2 Fine-tuning Strategies

After pre-training, models are adapted to specific tasks:

1. **Full fine-tuning:** Update all parameters on target task data
2. **Linear probing:** Train only a linear classifier on frozen features
3. **Adapter methods:** Add small trainable modules between layers
4. **LoRA (Low-Rank Adaptation):** Optimize low-rank decompositions of weight matrices

Note 13.1 (When to use each approach).

- **Abundant task data:** Full fine-tuning typically best
- **Limited task data:** LoRA/adapters prevent overfitting
- **Many tasks:** Adapter methods enable parameter-efficient multi-task learning

13.3 LoRA: Low-Rank Adaptation

LoRA reduces trainable parameters by decomposing weight updates:

$$\mathbf{W} \leftarrow \mathbf{W} + \Delta\mathbf{W}, \quad \Delta\mathbf{W} = \mathbf{B}\mathbf{A} \tag{13.1}$$

where $\mathbf{B} \in \mathbb{R}^{d\times r}$, $\mathbf{A} \in \mathbb{R}^{r\times k}$, and $r \ll \min(d, k)$.

During forward pass with input $\mathbf{x}$:

$$\mathbf{h} = \mathbf{W}\mathbf{x} + \frac{\alpha}{r}\mathbf{B}\mathbf{A}\mathbf{x} \tag{13.2}$$

where α is a scaling factor (typically equal to rank r).

Listing 13.1 – LoRA Implementation in PyTorch

```
import torch
import torch.nn as nn

class LoRALinear(nn.Module):
    def __init__(self, in_features, out_features, rank=4, alpha=4):
        super().__init__()
        self.rank = rank
        self.alpha = alpha
        self.scaling = alpha / rank

        # Original frozen weights
        self.weight = nn.Parameter(
            torch.randn(out_features, in_features),
            requires_grad=False
        )

        # Trainable low-rank matrices
        self.lora_A = nn.Parameter(torch.randn(rank, in_features))
        self.lora_B = nn.Parameter(torch.zeros(out_features, rank))

        # Initialize A with random Gaussian, B with zeros
        nn.init.normal_(self.lora_A, std=1.0 / rank)

    def forward(self, x):
        # Original forward pass
        h = F.linear(x, self.weight)
        # LoRA update
        h += self.scaling * (x @ self.lora_A.T @ self.lora_B.T)
        return h

# Usage in a model
class LoRALinearLayer(nn.Module):
    def __init__(self, linear_layer, rank=4, alpha=4):
        super().__init__()
        self.linear = linear_layer
        self.lora = LoRALinear(
            linear_layer.in_features,
            linear_layer.out_features,
            rank=rank,
```

```
            alpha=alpha
        )

    def forward(self, x):
        return self.lora(x)
```

Trainable parameters reduced from $d \times k$ to $r \times (d + k)$.

13.4 Transfer Learning

Transfer learning leverages knowledge from related tasks.

13.4.1 Feature Transfer

Features learned on one domain often transfer well:

- ImageNet features transfer to medical imaging, satellite imagery
- BERT features transfer across NLP tasks
- Key insight: Lower layers capture general features; higher layers capture task-specific patterns

13.4.2 Progressive Freezing

Gradually unfreeze layers during fine-tuning:

1. Train only the classifier head (frozen backbone)
2. Unfreeze last n layers, fine-tune
3. Optionally unfreeze remaining layers

This prevents catastrophic forgetting of pre-trained features.

13.5 Multi-Task Learning

Training a single model on multiple tasks simultaneously.

13.5.1 Hard Parameter Sharing

Share encoder weights across tasks:

$$\mathcal{L}_{\text{total}} = \sum_{t=1}^{T} \lambda_t \mathcal{L}_t(\theta_{\text{shared}}, \theta_t) \tag{13.3}$$

where λ_t is task weight and θ_t are task-specific heads.

13.5.2 Soft Parameter Sharing

Each task has its own parameters, with regularization encouraging similarity:

$$\mathcal{L}_{\text{total}} = \sum_{t=1}^{T} \mathcal{L}_t(\theta_t) + \lambda \sum_{i,j} \|\theta_i^{(l)} - \theta_j^{(l)}\|^2 \tag{13.4}$$

13.5.3 Task Balancing

Multi-task learning requires careful task weighting:

- **Equal weighting:** Simple but may not optimal
- **Uncertainty weighting:** Weight by inverse task uncertainty (Kendall et al., 2018)
- **Gradient balancing:** Balance gradient magnitudes across tasks
- **Dynamic weighting:** Adjust weights during training

13.6 Continual Learning

Training sequentially on new tasks without forgetting previous ones.

13.6.1 Catastrophic Forgetting

Neural networks tend to forget previous tasks when trained on new ones. Mitigation strategies:

1. **Experience Replay:** Store samples from previous tasks and replay during new training
2. **Elastic Weight Consolidation (EWC):** Protect important weights with regularization
3. **Progressive Networks:** Add new columns for new tasks while freezing old ones
4. **Architectural Growth:** Expand network capacity for new tasks

13.6.2 EWC Regularization

$$\mathcal{L}_{\text{EWC}} = \mathcal{L}_B(\theta) + \frac{\lambda}{2} \sum_i \mathcal{F}_i(\theta_i - \theta^*_{A,i})^2 \tag{13.5}$$

where $\mathcal{F}$ is the Fisher information matrix capturing parameter importance.

13.7 Self-Supervised Learning

Learning representations without labels using pretext tasks.

13.7.1 Contrastive Learning

Maximize agreement between differently augmented views of the same image:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \mathbf{1}_{[k \neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)} \tag{13.6}$$

where τ is temperature, $\mathbf{z}$ are normalized embeddings.

Popular contrastive learning frameworks:

- **SimCLR:** Simple framework with strong augmentations
- **MoCo:** Momentum contrast for large negative samples
- **SwAV:** Cluster assignments instead of pairwise comparison

13.7.2 Masked Image Modeling

BERT-style masking for images (MAE, BEiT):

- Mask random patches (75% for MAE)
- Decoder reconstructs masked patches from visible ones
- Encoder operates only on visible patches (highly efficient)

13.8 Distillation and Compression

Transferring knowledge from large models to smaller ones.

13.8.1 Knowledge Distillation

A smaller student model learns from a larger teacher:

$$\mathcal{L}_{\text{distill}} = \alpha \cdot \text{CE}(y, \text{softmax}(z_s/T)) + (1-\alpha) \cdot \text{CE}(y, \text{softmax}(z_t/T)) \tag{13.7}$$

where z_s and z_t are student and teacher logits, T is temperature.
Key insight: Soft targets from teacher contain more information than hard labels.

13.8.2 Self-Distillation

A model distills knowledge into itself:

- Train model A, then use A's predictions to train model B of same size
- BYOL, SimSiam: Two views with predictor network
- Shakes out noise and produces sharper predictions

13.9 Large Batch Training

Scaling training to thousands of GPUs requires careful batch size considerations.

13.9.1 Linear Scaling Rule

When increasing batch size B, scale learning rate proportionally:

$$\alpha_{\text{new}} = \alpha_{\text{old}} \times \frac{B_{\text{new}}}{B_{\text{old}}} \tag{13.8}$$

Goyal et al. (2017) achieved 256-way data parallelism on ImageNet with 8K batch size.

13.9.2 Warmup and Scaling

For very large batches (>16K):

- Gradual warmup to prevent early instability
- Layer-wise learning rate decay (LLRD): Lower LR for earlier layers
- Longer training (adjust schedule accordingly)

13.10 Efficient Training Techniques

13.10.1 Gradient Checkpointing

Trade compute for memory by recomputing activations during backward pass:

- Store activations only at selected checkpoints
- Recompute forward pass segments during backward
- Memory reduction: $O(\sqrt{n})$ with $O(n)$ compute overhead

13.10.2 Offloading

Move tensors between CPU and GPU memory:

- CPU offload: Keep parameters on CPU, load to GPU per layer
- ZeRO (Zero Redundancy Optimizer): Partition optimizer states across GPUs
- Activations offload: Pipeline recomputation

13.11 Debugging Training

13.11.1 Common Issues and Solutions

Table 13.1 – Common Training Issues and Solutions

Symptom	Possible Cause	Solution
Loss NaN	Large gradients	Reduce LR, gradient clipping
Loss not decreasing	Wrong learning rate	LR range test, reduce LR
Overfitting	Too few data, too many params	Regularization, augmentation
Underfitting	Model too simple	Larger model, more features
Catastrophic forgetting	Single-task training	EWC, replay, progressive
Instability at init	Poor initialization	Kaiming/He init, proper normalization

13.11.2 Training Recipes

Proven recipes for common scenarios:

Algorithm 4 Training Recipe for Vision Models

1. Start with small learning rate (1e-4 for Adam, 0.1 for SGD)
2. Use LR range test to find optimal LR
3. Apply cosine annealing schedule with warmup
4. Use strong augmentation (RandAugment, Mixup, CutMix)
5. Label smoothing (0.1 for classification)
6. Weight decay (0.01 for AdamW)
7. Exponential moving average of weights (EMA, decay=0.9999)
8. Early stopping on validation loss

Algorithm 5 Training Recipe for LLMs

1. Pre-train on large corpus (trillion+ tokens for GPT-3 scale)
2. Use mixed precision (BF16) with gradient checkpointing
3. Apply gradient clipping (1.0)
4. Use learning rate warmup (2000-10000 steps)
5. Cosine decay to small LR (typically 10% of peak)
6. Weight decay (0.1 for LLaMA-2)
7. Attention dropout during fine-tuning
8. LoRA for efficient fine-tuning on downstream tasks

13.12 Chapter Summary

1. Pre-training on large datasets followed by fine-tuning is the dominant paradigm
2. LoRA enables parameter-efficient fine-tuning by decomposing weight updates
3. Transfer learning allows leveraging representations across domains
4. Multi-task learning requires careful task balancing strategies
5. Continual learning techniques prevent catastrophic forgetting
6. Self-supervised learning (contrastive, masked) learns from unlabeled data
7. Knowledge distillation transfers capabilities from large to small models
8. Large batch training benefits from linear learning rate scaling and warmup
9. Systematic debugging and proven recipes accelerate development

13.13 Further Reading

- [54] — Distilling the Knowledge in a Neural Network

- [49] — BERT: Pre-training of Deep Bidirectional Transformers
- [55] — Multi-Task Learning Using Uncertainty to Weigh Losses
- [56] — Overcoming Catastrophic Forgetting in Neural Networks
- [1] — LoRA: Low-Rank Adaptation of Large Language Models
- [22] — A Simple Framework for Contrastive Learning (SimCLR)
- [57] — Masked Autoencoders Are Scalable Vision Learners (MAE)
- **goyal2017accurate** — Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour
- [58] — High-Resolution Image Synthesis with Latent Diffusion Models
- [59] — Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Bibliography

[1] E. J. Hu, Y. Shen, P. Wallis, *et al.*, "LoRA: Low-rank adaptation of large language models," 2022. arXiv: 2106.09685 [cs.CL].

[2] A. Choromanska, M. Henaff, M. Mathieu, G. B. Arous, and Y. LeCun, "The loss surfaces of multilayer networks," *Proceedings of the 18th International Conference on Artificial Intelligence and Statistics*, pp. 192–204, 2015.

[3] C. E. Shannon, "A mathematical theory of communication," *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948. DOI: 10.1002/j.1538-7305.1948.tb01338.x.

[4] N. Tishby and N. Zaslavsky, "Deep learning and the information bottleneck principle," 2015, IEEE Information Theory Workshop (ITW) 2015. arXiv: 1503.02406 [cs.LG].

[5] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," 2014, Originally presented at the NIPS 2013 Workshop on Deep Learning. arXiv: 1312.6114 [stat.ML].

[6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning* (Adaptive Computation and Machine Learning). MIT Press, 2016, ISBN: 978-0262035613.

[7] C. M. Bishop, *Pattern Recognition and Machine Learning* (Information Science and Statistics). Springer, 2006, ISBN: 978-0387310732.

[8] G. Strang, *Introduction to Linear Algebra*, 4th ed. Wellesley, MA: Wellesley-Cambridge Press, 2009, ISBN: 978-0980232714.

[9] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Hoboken, NJ: Wiley-Interscience, 2006, ISBN: 978-0471241959.

[10] K. Hornik, "Approximation capabilities of multilayer feedforward networks," *Neural Networks*, vol. 4, no. 2, pp. 251–257, 1991. DOI: 10.1016/0893-6080(91)90009-T.

[11] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems 25*, Red Hook, NY: Curran Associates, Inc., 2012, pp. 1097–1105.

[12] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proceedings of the IEEE International Conference on Computer Vision*, IEEE, 2015, pp. 1026–1034. DOI: 10.1109/ICCV.2015.123.

[13] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, IEEE, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90.

[14] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proceedings of the 32nd International Conference on Machine Learning*, Lille, France: PMLR, 2015, pp. 448–456. DOI: 10.5555/3045118.3045167.

[15] D. Misra, "Mish: A self regularized non-monotonic neural activation function," 2019, arXiv preprint. arXiv: 1908.08681 [cs.LG].

[16] U. Maulik, X. Ma, D. Patel, and Y. Zhang, "StarReLU: Are self-gated activations better than their sibling ReLU for transformers?" In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, IEEE, 2024. DOI: 10.1109/CVPR52700.2024.01549.

[17] K. Clark, "A unified view of GELU activation and dropout," in *arXiv preprint*, 2022. arXiv: 2209.03448 [cs.LG].

[18] P. J. Huber, "Robust estimation of a location parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.

[19] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE International Conference on Computer Vision*, IEEE, 2017, pp. 2980–2988. DOI: 10.1109/ICCV.2017.324.

[20] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," pp. 2818–2826, 2016. DOI: 10.1109/CVPR.2016.308.

[21] A. v. d. Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," 2018. arXiv: 1807.03748 [cs.LG].

[22] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proceedings of the 37th International Conference on Machine Learning*, PMLR, 2020, pp. 1597–1607.

[23] F. Milletari, N. Navab, and S.-A. Ahmadi, "V-net: Fully convolutional neural networks for volumetric medical image segmentation," pp. 565–571, 2016. DOI: 10.1109/3DV.2016.79.

[24] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, IEEE, 2022, pp. 16 000–16 009. DOI: 10.1109/CVPR52688.2022.01553.

[25] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He, "Aggregated residual transformations for deep neural networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, IEEE, 2017, pp. 1492–1500. DOI: 10.1109/CVPR.2017.156.

[26] G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Q. Weinberger, "Deep networks with stochastic depth," in *Proceedings of the European Conference on Computer Vision*, Springer, 2016, pp. 646–661. DOI: 10.1007/978-3-319-46493-0_39.

[27] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, IEEE, 2017, pp. 4700–4708. DOI: 10.1109/CVPR.2017.243.

[28] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, "Attention is all you need," 2017, NeurIPS 2017. arXiv: 1706.03762 [cs.CL].

[29] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *arXiv preprint arXiv:2104.09864*, 2021. arXiv: 2104.09864 [cs.CL].

[30] O. Press, N. A. Smith, and M. Lewis, "Train short, test long: Attention with linear biases enables input length extrapolation," in *Proceedings of the 10th International Conference on Learning Representations*, 2022. arXiv: 2108.12409 [cs.LG].

[31] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, IEEE, 2018, pp. 7132–7141. DOI: 10.1109/CVPR.2018.00745.

[32] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional block attention module," in *Proceedings of the European Conference on Computer Vision*, Springer, 2018, pp. 3–19. DOI: 10.1007/978-3-030-01234-2_1.

[33] Z. Liu, Y. Lin, Y. Cao, *et al.*, "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, IEEE, 2021, pp. 10 012–10 022. DOI: 10.1109/ICCV48922.2021.00971.

[34] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang, "On large-batch training for deep learning: Generalization gap and sharp minima," *International Conference on Learning Representations (ICLR)*, 2017. arXiv: 1609.04836 [cs.LG].

[35] L. N. Smith, "Cyclical learning rates for training neural networks," in *Proceedings of the IEEE Winter Conference on Applications of Computer Vision*, IEEE, 2017, pp. 464–472. DOI: 10.1109/WACV.2017.58.

[36] E. Hoffer, I. Hubara, and D. Soudry, "Train longer, generalize better: Closing the generalization gap in large batch training of neural networks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[37] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.

[38] Y. Gal and Z. Ghahramani, "Dropout as a bayesian approximation: Representing model uncertainty in deep learning," in *Proceedings of the 33rd International Conference on Machine Learning*, PMLR, 2016, pp. 1050–1059.

[39] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *International Conference on Learning Representations (ICLR)*, 2019. arXiv: 1711.05101 [cs.LG].

[40] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," *International Conference on Machine Learning (ICML)*, pp. 1310–1318, 2013.

[41] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "Mixup: Beyond empirical risk minimization," *International Conference on Learning Representations (ICLR)*, 2018. arXiv: 1710.09412 [cs.LG].

[42] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "Cutmix: Regularization strategy to train strong classifiers with localizable features," *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 6023–6032, 2019. arXiv: 1905.04899 [cs.CV].

[43] E. D. Cubuk, B. Zoph, D. Mane, V. Vasudevan, and Q. V. Le, "Autoaugment: Learning augmentation policies from data," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 113–123, 2019. arXiv: 1805.09501 [cs.CV].

[44] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, "Randaugment: Practical automated data augmentation with a reduced search space," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3008–3017, 2020. arXiv: 1909.13719 [cs.CV].

[45] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 1715–1725, 2016. arXiv: 1508.07909 [cs.CL].

[46] A. Dosovitskiy, L. Beyer, A. Kolesnikov, *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *9th International Conference on Learning Representations (ICLR 2021)*, 2021. arXiv: 2010.11929 [cs.CV].

[47] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," 2019, OpenAI Technical Report GPT-2.

[48] T. B. Brown, B. Mann, N. Ryder, *et al.*, "Language models are few-shot learners," pp. 1877–1901, 2020. arXiv: 2005.14165 [cs.CL].

[49] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," pp. 4171–4186, 2019. DOI: 10.18653/v1/N19-1423.

[50] H. Touvron, T. Lavril, G. Izacard, *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023. arXiv: 2302.13971 [cs.CL].

[51] T. Dao, "Flashattention: Fast and memory-efficient exact attention with io-awareness," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[52] J. Hoffmann, S. Borgeaud, A. Mensch, *et al.*, "Training compute-optimal large language models," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[53] J. Wei, Y. Tay, R. Bommasconi, *et al.*, "Emergent abilities of large language models," *arXiv preprint arXiv:2206.07682*, 2022. arXiv: 2206.07682 [cs.CL].

[54] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015. arXiv: 1503.02531 [stat.ML].

[55] A. Kendall, Y. Gal, and R. Cipolla, "Multi-task learning using uncertainty to weigh losses for scene geometry and semantics," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7482–7491, 2018.

[56] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, *et al.*, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017. DOI: 10.1073/pnas.1611835114.

[57] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16 000–16 009, 2022. arXiv: 2111.06377 [cs.CV].

[58] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10 684–10 695, 2022. arXiv: 2112.10752 [cs.CV].

[59] P. Wei and K. Mckeown, "Symbolic knowledge distillation: From general language models to commonsense models," *arXiv preprint arXiv:2210.04678*, 2022. arXiv: 2210.04678 [cs.CL].

[60] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958. DOI: 10.1037/h0042519.

[61] B. Widrow and M. E. Hoff, "Adaptive "adaline" neuron using chemical "memistors"," vol. 3, pp. 1554–1556, 1960, Reprinted in [62].

[62] B. Widrow and S. D. Stearns, *Adaptive Signal Processing*. Prentice-Hall, Englewood Cliffs, NJ, 1985.

[63] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986. DOI: 10.1038/323533a0.

[64] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504–507, 2006. DOI: 10.1126/science.1127647.

[65] Y. Bengio and Y. LeCun, "Scaling learning algorithms towards AI," in *Large-Scale Kernel Machines*, L. Bottou, O. Chapelle, D. DeCoste, and J. Weston, Eds., Cambridge, MA: MIT Press, 2007, pp. 321–359.

[66] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015. DOI: 10.1038/nature14539.

[67] Y. LeCun, B. Boser, J. Denker, *et al.*, "Backpropagation applied to handwritten zip code recognition," *Neural Computation*, vol. 1, no. 4, pp. 541–551, 1989. DOI: 10.1162/neco.1989.1.4.541.

[68] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. DOI: 10.1109/5.726791.

[69] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *3rd International Conference on Learning Representations (ICLR 2015)*, arXiv:1409.1556, 2015.

[70] C. Szegedy, W. Liu, Y. Jia, *et al.*, "Going deeper with convolutions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, IEEE, 2015, pp. 1–9. DOI: 10.1109/CVPR.2015.7298594.

[71] K. He, X. Zhang, S. Ren, and J. Sun, "Identity mappings in deep residual networks," in *Computer Vision – ECCV 2016*, Cham: Springer, 2016, pp. 630–645. DOI: 10.1007/978-3-319-46493-0_38.

[72] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. DOI: 10.1162/neco.1997.9.8.1735.

[73] K. Cho, B. Van Merriënboer, D. Bahdanau, and Y. Bengio, "On the properties of neural machine translation: Encoder-decoder approaches," in *Proceedings of SSST-8, 8th Workshop on Syntax, Semantics and Structure in Statistical Translation*, Stroudsburg, PA: Association for Computational Linguistics, 2014, pp. 103–111. DOI: 10.3115/v1/W14-4012.

[74] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," in *NIPS 2014 Workshop on Recurrent Neural Networks*, arXiv:1412.3555, 2014.

[75] H. Robbins and S. Monro, "A stochastic approximation method," *The Annals of Mathematical Statistics*, vol. 22, no. 3, pp. 400–407, 1951. DOI: 10.1214/aoms/1177729586.

[76] T. Tieleman and G. Hinton, *Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude*, COURSERA: Neural Networks for Machine Learning, 2012.

[77] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2015, Originally published as arXiv:1412.6980 (2014). arXiv: 1412.6980 [cs.LG].

[78] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2019, Originally published as arXiv:1711.05101 (2017). arXiv: 1711.05101 [cs.LG].

[79] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," 2016. arXiv: 1607.06450 [cs.NE].

[80] D. Ulyanov, A. Vedaldi, and V. Lempitsky, "Instance normalization: The missing ingredient for fast stylization," 2016. arXiv: 1607.08022 [cs.NE].

[81] J. Deotte *et al.*, "Gridmask data augmentation," 2020. arXiv: 2001.04086 [cs.CV].

[82] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, IEEE, 2014, pp. 580–587. DOI: 10.1109/CVPR.2014.81.

[83] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems 28*, Curran Associates, Inc., 2015, pp. 91–99.

[84] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, IEEE, 2016, pp. 779–788. DOI: 10.1109/CVPR.2016.91.

[85] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, IEEE, 2017, pp. 2117–2125. DOI: 10.1109/CVPR.2017.106.

[86] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, *et al.*, "Generative adversarial networks," in *Advances in Neural Information Processing Systems 27*, Curran Associates, Inc., 2014, pp. 2672–2680.

[87] D. J. Rezende, S. Mohamed, and D. Wierstra, "Stochastic backpropagation and approximate inference in deep generative models," in *Proceedings of the 31st International Conference on Machine Learning*, Beijing, China: PMLR, 2014, pp. 1278–1286. DOI: 10.5555/3044805.3044940.

[88] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, IEEE, 2015, pp. 3431–3440. DOI: 10.1109/TPAMI.2016.2572683.

[89] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI 2015)*, Cham: Springer, 2015, pp. 234–241. DOI: 10.1007/978-3-319-24574-4_28.

[90] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proceedings of the IEEE International Conference on Computer Vision*, IEEE, 2017, pp. 2961–2969. DOI: 10.1109/ICCV.2017.322.

[91] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," 2014. arXiv: 1409.0473 [cs.CL].

[92] K. Xu, J. Ba, R. Kiros, *et al.*, "Show, attend and tell: Neural image caption generation with visual attention," in *Proceedings of the 32nd International Conference on Machine Learning*, Lille, France: PMLR, 2015, pp. 2048–2057. DOI: 10.5555/3045118.3045259.

[93] X. Wang, R. Girshick, A. Gupta, and K. He, "Non-local neural networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, IEEE, 2018, pp. 7794–7803. DOI: 10.1109/CVPR.2018.00813.

[94] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers and distillation through attention," in *Proceedings of the 38th International Conference on Machine Learning*, Virtual: PMLR, 2021, pp. 10 347–10 357. DOI: 10.5555/3454284.3455222.

[95] I. Loshchilov and F. Hutter, "Sgdr: Stochastic gradient descent with warm restarts," 2017, Originally published as ICLR 2017 (workshop paper from ICLR 2016). arXiv: 1608.03983 [cs.LG].

[96] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, Sardinia, Italy: PMLR, 2010, pp. 249–256.

[97] A. Paszke, S. Gross, F. Massa, *et al.*, "Pytorch: An imperative style, high-performance deep learning library," 2019.

[98] M. Abadi, P. Barham, J. Chen, *et al.*, "Tensorflow: A system for large-scale machine learning," in *12th USENIX Symposium on Operating Systems Design and Implementation*, Savannah, GA: USENIX Association, 2016, pp. 265–283.

[99] K. P. Murphy, *Machine Learning: A Probabilistic Perspective* (Adaptive Computation and Machine Learning). MIT Press, 2012, ISBN: 978-0262018029.

[100] S. O. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Pearson, 2009, ISBN: 978-0131471399.

[101] C. Shorten and T. M. Khoshgoftaar, "A survey on deep learning: Concepts, architectures, and algorithms," 2019. arXiv: 1908.08994 [cs.LG].

[102] X. Cui *et al.*, *Cutting catalog impact: A comparison of neural network approaches on medium-size datasets*, arXiv preprint, 2020.

[103] C. Eckart and G. Young, "The approximation of one matrix by another of lower rank," *Psychometrika*, vol. 1, no. 3, pp. 211–218, 1936. DOI: 10.1007/BF02288367.

[104] N. Shazeer, "GLU variants improve transformer," in *arXiv preprint arXiv:2002.05202*, 2020. arXiv: 2002.05202 [cs.CL].

[105] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994. DOI: 10.1109/72.279181.

[106] W. Luo, Y. Li, R. Urtasun, and R. Zemel, "Understanding the effective receptive field in deep convolutional neural networks," in *Advances in Neural Information Processing Systems 29*, 2016. arXiv: 1701.04128 [cs.CV].

[107] Y. Wu and K. He, "Group normalization," in *Proceedings of the European Conference on Computer Vision*, Springer, 2018, pp. 3–19. DOI: 10.1007/978-3-030-01234-2_1.

[108] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," *arXiv preprint arXiv:1706.05587*, 2017. arXiv: 1706.05587 [cs.CV].

[109] A. G. Howard, M. Zhu, B. Chen, *et al.*, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017. arXiv: 1704.04861 [cs.CV].

[110] M. D. Zeiler, "ADADELTA: An adaptive learning rate method," *arXiv preprint arXiv:1212.5701*, 2012. arXiv: 1212.5701 [cs.LG].

[111] S. J. Reddi, S. Kale, and S. Kumar, "On the convergence of Adam and beyond," in *6th International Conference on Learning Representations (ICLR 2018)*, 2018. arXiv: 1904.09237 [cs.LG].

[112] L. N. Smith, "A disciplined approach to neural network hyper-parameters: Part 1 – learning rate, batch size, momentum, and weight decay," in *NeurIPS Workshop*, 2018. arXiv: 1803.09820 [cs.LG].

[113] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," pp. 4510–4520, 2018. DOI: 10.1109/CVPR.2018.00474.

[114] M. Tan and Q. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *Proceedings of the 36th International Conference on Machine Learning*, PMLR, 2019, pp. 6105–6114.

[115] X. Zhang, X. Zhou, M. Lin, and J. Sun, "ShuffleNet: An extremely efficient convolutional neural network for mobile devices," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, IEEE, 2018, pp. 6848–6856. DOI: 10.1109/CVPR.2018.00716.

[116] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A convnet for the 2020s," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, IEEE, 2022, pp. 11 976–11 986. DOI: 10.1109/CVPR52688.2022.01167.

[117] J. Kaplan, S. McCandlish, T. Henighan, *et al.*, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020. arXiv: 2001.08361 [cs.LG].

[118] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," in *3rd International Conference on Learning Representations (ICLR 2016)*, 2016. arXiv: 1510.00149 [cs.CV].

[119] P. Micikevicius, S. Narang, J. Alben, *et al.*, "Mixed precision training," in *6th International Conference on Learning Representations (ICLR 2018)*, 2018. arXiv: 1710.03740 [cs.LG].

[120] T. Chen, B. Xu, C. Zhang, and C. Guestrin, "Training deep nets with sublinear memory cost," *arXiv preprint arXiv:1604.06174*, 2016. arXiv: 1604.06174 [cs.LG].

[121] S. G. Müller and F. Hutter, "Torchvision's mobile vision application: The effect of trivial augmentations on image classification," *arXiv preprint arXiv:2305.10813*, 2023. arXiv: 2305.10813 [cs.CV].

Chapter A

Mathematical Reference

A.1 Linear Algebra

A.1.1 Matrix Operations

- $\mathbf{AB} \neq \mathbf{BA}$ (non-commutative in general)
- $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$
- $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$
- $\det(\mathbf{AB}) = \det(\mathbf{A})\det(\mathbf{B})$
- $\mathrm{tr}(\mathbf{AB}) = \mathrm{tr}(\mathbf{BA})$

A.1.2 Norms

For a vector $\mathbf{x} \in \mathbb{R}^n$:

$$\|\mathbf{x}\|_1 = \sum_{i=1}^{n} |x_i| \quad \text{(L1 norm)} \tag{A.1}$$

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2} \quad \text{(L2 / Euclidean norm)} \tag{A.2}$$

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p} \quad \text{(L}p\text{ norm)} \tag{A.3}$$

$$\|\mathbf{x}\|_\infty = \max_i |x_i| \quad \text{(L}\infty\text{ norm)} \tag{A.4}$$

For a matrix $\mathbf{A}$:

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i,j} A_{ij}^2} \quad \text{(Frobenius norm)} \tag{A.5}$$

$$\|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A}) \quad \text{(Spectral norm = largest singular value)} \tag{A.6}$$

A.1.3 Eigenvalue Decomposition

A square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ can be decomposed (if diagonalizable):

$$\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^{-1} \tag{A.7}$$

where $\mathbf{\Lambda} = \mathrm{diag}(\lambda_1, \dots, \lambda_n)$ are eigenvalues and $\mathbf{Q}$ contains eigenvectors.

Properties:

- $\det(\mathbf{A}) = \prod_{i=1}^{n} \lambda_i$
- $\mathrm{tr}(\mathbf{A}) = \sum_{i=1}^{n} \lambda_i$
- $\mathbf{A}$ is positive definite iff $\lambda_i > 0$ for all i

- $\mathbf{A}^k = \mathbf{Q}\mathbf{\Lambda}^k\mathbf{Q}^{-1}$

A.1.4 Singular Value Decomposition (SVD)

Any matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ can be decomposed:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top \tag{A.8}$$

where $\mathbf{U} \in \mathbb{R}^{m\times m}$ (left singular vectors), $\mathbf{\Sigma} \in \mathbb{R}^{m\times n}$ (diagonal singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$), $\mathbf{V} \in \mathbb{R}^{n\times n}$ (right singular vectors).

Applications in deep learning:

- Low-rank approximation: $\mathbf{A}_k = \mathbf{U}_k\mathbf{\Sigma}_k\mathbf{V}_k^\top$ (keep top k singular values)
- Principal Component Analysis (PCA)
- Understanding weight matrices: $\sigma_{\max}(\mathbf{W})$ governs gradient flow through a layer

A.2 Calculus

A.2.1 Common Derivatives

$$\frac{d}{dx}x^n = nx^{n-1} \tag{A.9}$$

$$\frac{d}{dx}e^x = e^x \tag{A.10}$$

$$\frac{d}{dx}\log(x) = \frac{1}{x} \tag{A.11}$$

$$\frac{d}{dx}\sin(x) = \cos(x) \tag{A.12}$$

$$\frac{d}{dx}\cos(x) = -\sin(x) \tag{A.13}$$

$$\frac{d}{dx}\sigma(x) = \sigma(x)(1-\sigma(x)) \quad \text{(sigmoid)} \tag{A.14}$$

$$\frac{d}{dx}\text{ReLU}(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases} \tag{A.15}$$

$$\frac{d}{dx}\tanh(x) = 1 - \tanh^2(x) \tag{A.16}$$

A.2.2 Chain Rule

For $y = f(g(x))$:

$$\frac{dy}{dx} = f'(g(x)) \cdot g'(x) \tag{A.17}$$

For multivariable functions, the chain rule becomes:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \tag{A.18}$$

where $\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$ is the **Jacobian matrix** with entries $J_{ij} = \frac{\partial y_i}{\partial x_j}$.

A.2.3 Jacobian and Hessian

Jacobian of $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$ at $\mathbf{x}$:

$$\mathbf{J}_f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \in \mathbb{R}^{m \times n} \tag{A.19}$$

Hessian of $f : \mathbb{R}^n \to \mathbb{R}$ at $\mathbf{x}$:

$$\mathbf{H}_f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix} \in \mathbb{R}^{n \times n} \tag{A.20}$$

Uses in deep learning:

- Jacobians describe gradient flow through layers (critical for RNN vanishing gradient analysis)
- The Hessian's eigenvalues determine the loss landscape curvature (sharp vs flat minima)
- Second-order methods (Newton's method) use $\mathbf{H}^{-1}$ for parameter updates

A.2.4 Taylor Expansion

For a smooth function $f(\mathbf{x})$ around $\mathbf{x}_0$:

$$f(\mathbf{x}) = f(\mathbf{x}_0) + \nabla f(\mathbf{x}_0)^\top (\mathbf{x} - \mathbf{x}_0) + \frac{1}{2}(\mathbf{x} - \mathbf{x}_0)^\top \mathbf{H}(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0) + \cdots \tag{A.21}$$

A.2.5 Lagrange Multipliers

For constrained optimization $\min_{\mathbf{x}} f(\mathbf{x})$ subject to $g(\mathbf{x}) = 0$:

$$\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda \cdot g(\mathbf{x}) \tag{A.22}$$

Solution: $\nabla_{\mathbf{x}} \mathcal{L} = \mathbf{0}$, $\nabla_\lambda \mathcal{L} = 0$.

A.3 Probability and Statistics

A.3.1 Bayes' Theorem

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} \tag{A.23}$$

A.3.2 Distributions

$$\text{Gaussian:} \quad p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \tag{A.24}$$

$$\text{Bernoulli:} \quad P(X = k) = p^k(1-p)^{1-k}, \quad k \in \{0, 1\} \tag{A.25}$$

$$\text{Categorical:} \quad P(X = k) = p_k, \quad \sum_k p_k = 1 \tag{A.26}$$

A.3.3 Expectation and Variance

$$\mathbb{E}[X] = \sum_x x \cdot P(X = x) \quad \text{(discrete)} \tag{A.27}$$

$$\mathbb{E}[X] = \int x \cdot p(x)\, dx \quad \text{(continuous)} \tag{A.28}$$

$$\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 \tag{A.29}$$

$$\mathrm{Var}(aX + b) = a^2\mathrm{Var}(X) \tag{A.30}$$

$$\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y] \quad \text{if } X \perp Y \tag{A.31}$$

A.3.4 KL Divergence and Cross-Entropy

$$\mathrm{KL}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)} \quad \text{(KL divergence, } \geq 0\text{)} \tag{A.32}$$

$$H(p, q) = -\sum_x p(x) \log q(x) \quad \text{(cross-entropy)} \tag{A.33}$$

$$H(p, q) = H(p) + \mathrm{KL}(p\|q) \tag{A.34}$$

A.3.5 Information Theory

$$H(X) = -\sum_x P(X = x) \log P(X = x) \quad \text{(entropy)} \tag{A.35}$$

$$I(X; Y) = H(X) - H(X|Y) \quad \text{(mutual information)} \tag{A.36}$$

Chapter B

Quick Reference Tables

B.1 Activation Functions

Table B.1 – Activation Function Reference

Name	Formula	Range	Use Case
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	$(0, 1)$	Binary output
Tanh	$\tanh(x)$	$(-1, 1)$	RNN hidden
ReLU	$\max(0, x)$	$[0, \infty)$	Default (hidden)
Leaky ReLU	$\max(\alpha x, x)$	$(-\infty, \infty)$	Avoid dead neurons
GELU	$x \cdot \Phi(x)$	$(-\infty, \infty)$	Transformers
Swish	$x \cdot \sigma(\beta x)$	$(-\infty, \infty)$	Modern CNNs
Softmax	$\frac{e^{x_i}}{\sum_j e^{x_j}}$	$(0, 1)$	Classification output

B.2 Loss Functions

Table B.2 – Loss Function Reference

Loss	Formula	Use Case
MSE	$\frac{1}{N}\sum_i (y_i - \hat{y}_i)^2$	Regression
MAE	$\frac{1}{N}\sum_i \lvert y_i - \hat{y}_i \rvert$	Regression (robust)
Cross-Entropy	$-\sum_c y_c \log \hat{y}_c$	Classification
Binary CE	$-y \log \hat{y} - (1-y)\log(1-\hat{y})$	Binary classif.
Huber	$\begin{cases} \frac{1}{2}(y-\hat{y})^2 & \lvert y-\hat{y}\rvert \le \delta \\ \delta(\lvert y-\hat{y}\rvert - \frac{\delta}{2}) & \text{else} \end{cases}$	Robust regression
Focal	$-(1-p_t)^\gamma \log p_t$	Imbalanced data
KL Divergence	$\sum_c p_c \log \frac{p_c}{q_c}$	Distribution matching
Triplet	$\max(0, d^+ - d^- + m)$	Metric learning

B.3 Common Datasets

Table B.3 – Benchmark Datasets

Dataset	Task	Size	Classes	Year
MNIST	Classification	70K	10	1998
CIFAR-10	Classification	60K	10	2009
CIFAR-100	Classification	60K	100	2009
ImageNet-1K	Classification	1.2M	1000	2009
COCO	Detection/Seg	330K	80	2014
LibriSpeech	ASR	281H	–	2015
SQuAD	QA	150K	–	2016
GLUE	NLP tasks	varies	varies	2018

Chapter C

PyTorch Quick Reference

C.1 Common Operations

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

# Creation
x = torch.randn(3, 4)  # Random normal
x = torch.zeros(3, 4)  # Zeros
x = torch.ones(3, 4)   # Ones
x = torch.tensor([1, 2, 3])  # From data
x = torch.arange(0, 10)  # 0, 1, ..., 9

# Shape operations
y = x.view(1, 12)     # Reshape
y = x.reshape(2, 6)   # Reshape (may copy)
y = x.permute(1, 0, 2) # Transpose dimensions
y = x.flatten()        # Flatten to 1D
y = x.squeeze(0)       # Remove dim of size 1
y = x.unsqueeze(0)     # Add dim at position 0

# Math operations
y = x.sum(dim=0)
y = x.mean(dim=1)
y = x.max(dim=1)       # Returns (values, indices)
y = x.softmax(dim=-1)
y = torch.argmax(x, dim=-1)

# GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
x = x.to(device)
x = x.cuda()
x = x.cpu()
```

Listing C.1 – Common PyTorch tensor operations

C.2 Model Creation

```python
class ConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch, stride=1):
        super().__init__()
        self.conv = nn.Conv2d(in_ch, out_ch, 3, stride, 1, bias=False)
        self.bn = nn.BatchNorm2d(out_ch)
        self.relu = nn.ReLU(inplace=True)

    def forward(self, x):
        return self.relu(self.bn(self.conv(x)))

```

```python
class MyModel(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            ConvBlock(3, 64),
            nn.MaxPool2d(2),
            ConvBlock(64, 128),
            nn.AdaptiveAvgPool2d(1)
        )
        self.classifier = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.features(x)
        x = x.flatten(1)
        return self.classifier(x)
```

Listing C.2 – PyTorch model definition patterns

C.3 Training Utilities

```python
# Gradient clipping
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

# Learning rate scheduler
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=100, eta_min=1e-6)
scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer, max_lr=0.01, total_steps=10000)

# Mixed precision
scaler = torch.amp.GradScaler('cuda')
with torch.amp.autocast('cuda'):
    output = model(data)
    loss = criterion(output, target)
scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()

# Model summary
from torchsummary import summary
summary(model, (3, 224, 224))

# Save and load
torch.save(model.state_dict(), 'model.pth')
model.load_state_dict(torch.load('model.pth'))
```

Listing C.3 – Common training utilities

Chapter D

Glossary

D.1 Architecture Terms

ANN Artificial Neural Network — the broadest class of neural networks

CNN Convolutional Neural Network — specialized for grid-like data (images, audio spectrograms)

RNN Recurrent Neural Network — processes sequential data with hidden state recurrence

LSTM Long Short-Term Memory — gated RNN that solves the vanishing gradient problem

GRU Gated Recurrent Unit — simplified LSTM with two gates (reset, update)

Transformer Attention-based architecture that processes sequences in parallel

ViT Vision Transformer — applies the Transformer architecture to image patches

GAN Generative Adversarial Network — generator-discriminator framework for generative modeling

VAE Variational Autoencoder — probabilistic generative model with encoder-decoder structure

Autoencoder Self-supervised model that learns to compress and reconstruct data

U-Net Encoder-decoder architecture with skip connections for segmentation

D.2 Training Terms

Epoch One complete pass through the entire training dataset

Batch A subset of samples processed together in one forward/backward pass

Batch Size Number of samples per batch (e.g., 32, 64, 256)

Backpropagation Algorithm for computing gradients via reverse-mode automatic differentiation

Learning Rate (LR) Step size for parameter updates during optimization

Loss Function Objective function that the model minimizes during training

Gradient Partial derivative of the loss with respect to model parameters

Convergence State where the loss stops decreasing (model has learned the data)

Overfitting Model performs well on training data but poorly on unseen data

Generalization Model's ability to perform well on unseen data

Regularization Techniques to prevent overfitting (dropout, weight decay, etc.)

Early Stopping Stop training when validation performance degrades

Fine-tuning Adapting a pretrained model to a new task/dataset

Pre-training Training a model on a large dataset before task-specific adaptation

Transfer Learning Using knowledge from one task to improve another

D.3 Layer and Operation Terms

Convolution Operation that slides a kernel across input to extract local features

Pooling Spatial downsampling operation (max pooling, average pooling)

Stride Step size for the sliding window in convolution/pooling

Padding Adding zeros (or values) around input borders

Kernel / Filter Learnable weights applied in convolution

Feature Map Output of a convolutional layer

Activation Function Nonlinear function applied after linear transformation

Dropout Randomly zeroing activations during training for regularization

BatchNorm Batch Normalization — normalizes features across the batch dimension

LayerNorm Layer Normalization — normalizes features across the feature dimension

Softmax Converts logits to probability distribution (sums to 1)

Embedding Dense vector representation of discrete tokens (words, patches)

Attention Mechanism that computes weighted sums of values based on query-key similarity

Self-Attention Attention where queries, keys, and values come from the same sequence

Cross-Attention Attention where queries and keys/values come from different sequences

Positional Encoding Encoding of position information for sequence models

Residual Connection Skip connection that adds input to output ($\mathbf{y} = \mathbf{x} + \mathcal{F}(\mathbf{x})$)

D.4 Performance Metrics

FLOPs Floating Point Operations — measure of computational cost

MACs Multiply-Accumulate operations ($\approx$ half of FLOPs)

GFLOPs Giga FLOPs (10^9 FLOPs)

Top-1 Accuracy Fraction of samples where the highest-confidence prediction is correct

Top-5 Accuracy Fraction of samples where the correct class is in the top 5 predictions

mAP Mean Average Precision — standard metric for object detection

IoU Intersection over Union — overlap measure for segmentation/detection

BLEU Bilingual Evaluation Understudy — metric for text generation quality

Perplexity Exponential of cross-entropy; lower is better for language models

Throughput Number of samples processed per second (samples/sec)

Latency Time to process a single sample (ms)

D.5 Other Common Abbreviations

SGD Stochastic Gradient Descent

Adam Adaptive Moment Estimation optimizer

AdamW Adam with decoupled weight decay

ReLu Rectified Linear Unit activation function

GELU Gaussian Error Linear Unit activation function

MSE Mean Squared Error loss

MAE Mean Absolute Error loss

KL Kullback-Leibler divergence

PCA Principal Component Analysis

SVD Singular Value Decomposition

AMP Automatic Mixed Precision

NAS Neural Architecture Search

TTA Test-Time Augmentation

QA Question Answering

NLP Natural Language Processing

CV Computer Vision

Acknowledgments

> "The important thing in science is not so much to obtain new facts as to discover new ways of thinking about them."
>
> — William Lawrence Bragg

We hope this textbook has provided you with a solid foundation in neural networks and deep learning. The field is constantly evolving, and we encourage you to stay curious and continue exploring.

Happy learning!